COMP3711: Design and Analysis of Algorithms

WU Jinjun

November 10, 2026

Contents

1 Prerequisites: Asymptotic Notations **6**

2 Divide-and-Conquer and Randomized Algorithms **9**
2.1 Divide and Conquer . . . 9
2.1.1 Binary Search . . . 9
2.1.2 Mergesort & Inversion Count . . . 14
2.1.3 Maximum Subarray . . . 19
2.1.4 Multiplication and Master Theorem . . . 21
2.1.5 Matrix Multiplication and Strassen's Algorithm . . . 25
2.2 Randomized Algorithms . . . 26
2.2.1 Why Randomization? . . . 26
2.2.2 Probability Review . . . 26
2.2.3 Linearity of Expectation . . . 26
2.2.4 Example: Hiring Problem . . . 27
2.2.5 More Indicator-Variable Examples . . . 27
2.2.6 Generating a Uniform Random Permutation . . . 28
2.2.7 Geometric Distribution and Coupon Collector . . . 28
2.2.8 Case Study: The Majority Element . . . 30

3 Balanced Trees, B+ Trees and Amortized Analysis **32**
3.1 AVL Trees, Splay Trees and Amortized Analysis . . . 32
3.1.1 AVL Trees . . . 32
3.1.2 Insertion: the Four Rotations . . . 32
3.1.3 Worked Example: Building an AVL Tree . . . 33
3.1.4 Splay Trees . . . 34
3.1.5 Amortized Analysis . . . 35
3.1.6 Amortized Analysis of Splaying . . . 36
3.2 Red-Black Trees and B+ Trees . . . 38
3.2.1 Red–Black Trees . . . 38
3.2.2 Insertion . . . 38
3.2.3 Deletion . . . 39
3.2.4 B+ Trees . . . 40
3.3 Inverted File Index . . . 42
3.3.1 Definition . . . 42

3.3.2 Terms and Their Normalisation . . . 42
3.3.3 Accessing Terms . . . 42
3.3.4 Distributed Indexing . . . 43
3.3.5 Threshold . . . 43
3.3.6 Evaluating a Search Engine . . . 43

4 Priority Queues: Leftist, Skew, Binomial Heaps 45
4.1 Leftist Heaps and Skew Heaps . . . 45
4.1.1 Leftist Heaps . . . 45
4.1.2 Merging Two Leftist Heaps . . . 45
4.1.3 Skew Heaps . . . 46
4.2 Binomial Queues . . . 48
4.2.1 Binomial Trees . . . 48
4.2.2 Operations . . . 49
4.2.3 Amortized Analysis . . . 49

5 Sorting and External Sorting 51
5.1 Quicksort . . . 51
5.1.1 Deterministic Quicksort (Lomuto Partition) . . . 51
5.1.2 Quicksort Analysis . . . 52
5.1.3 Randomized Selection (Quickselect) . . . 53
5.1.4 Hoare's Partition (Alternative) . . . 54
5.2 Heapsort . . . 55
5.2.1 Priority Queue Motivation . . . 55
5.2.2 Heap Data Structure . . . 55
5.2.3 Max-Heapify (Bubble-Down) . . . 56
5.2.4 Build Max-Heap . . . 57
5.2.5 Heapsort Algorithm . . . 58
5.2.6 Priority Queue Operations (Applications of Heaps) . . . 59
5.2.7 Applications . . . 60
5.3 Linear-Time Sorting . . . 62
5.3.1 The Decision-Tree Model and the Sorting Lower Bound . . . 62
5.3.2 Counting Sort . . . 64
5.3.3 Radix Sort . . . 64
5.3.4 Bucket Sort . . . 66
5.4 External Sorting . . . 69
5.4.1 The Model . . . 69
5.4.2 Replacement Selection: Longer Initial Runs . . . 69
5.4.3 Polyphase Merge: Fewer Tapes . . . 70
5.4.4 Optimising the Merge Order: Huffman Merging . . . 71
5.4.5 Parallel Merging . . . 71

6 Design Paradigms: Greedy, Huffman, Backtracking and Dynamic Programming 72
6.1 Greedy Algorithms . . . 72
6.1.1 Warm-up: Change-Making . . . 72
6.1.2 Warm-up: Max Sum of Non-Adjacent Elements . . . 73

6.1.3 Interval Scheduling . . . 73
6.1.4 Exercise: Interval Covering . . . 74
6.1.5 Fractional Knapsack . . . 75
6.1.6 Interval Partitioning . . . 76
6.2 Huffman Coding . . . 78
6.2.1 Motivation: Variable-Length Encoding . . . 78
6.2.2 Prefix-Free Codes . . . 78
6.2.3 The Encoding Problem . . . 78
6.2.4 Tree Correspondence and Weighted External Path Length . . . 79
6.2.5 Huffman's Greedy Algorithm . . . 79
6.2.6 Worked Example . . . 79
6.2.7 Correctness Proof . . . 80
6.2.8 Application: Optimal Two-Way Merge . . . 81
6.3 Backtracking . . . 83
6.3.1 The Eight Queens Problem . . . 83
6.3.2 The Turnpike Reconstruction Problem . . . 84
6.3.3 Game Trees and Minimax . . . 84
6.3.4 Alpha–Beta Pruning . . . 85
6.4 Dynamic Programming . . . 86
6.4.1 Warm-up: Stair Climbing and Fibonacci . . . 86
6.4.2 The Rod Cutting Problem . . . 87
6.4.3 Exercise: Maximum Sum of Non-Adjacent Elements . . . 89
6.4.4 Weighted Interval Scheduling . . . 90
6.4.5 Exercise: Highway Billboards . . . 92
6.4.6 Kadane's Algorithm Derived from DP . . . 92
6.4.7 Minimum Number of Coins . . . 93
6.4.8 The 0/1 Knapsack Problem . . . 94
6.4.9 Longest Common Subsequence . . . 95
6.4.10 Exercise: Choosing m Numbers with a Sliding-Window Constraint . . . 98
6.4.11 DP over Intervals: Longest Palindromic Substring . . . 98
6.4.12 Longest Palindromic Subsequence . . . 100
6.4.13 Matrix-Chain Multiplication . . . 101
6.4.14 Optimal Binary Search Trees . . . 103
6.4.15 RNA Secondary Structure . . . 106

7 Graph Search and Fundamentals 108
7.1 Graph Algorithms . . . 108
7.1.1 Graph Basics . . . 108
7.1.2 Graph Representation . . . 110
7.1.3 Paths, Connectivity, and Trees . . . 110
7.1.4 Breadth-First Search . . . 112
7.1.5 Connected Components and Reachability . . . 113
7.1.6 Worked BFS Example . . . 115
7.1.7 Depth-First Search . . . 117
7.1.8 Worked DFS Example . . . 117
7.1.9 Classifying Edges . . . 118
7.1.10 Cycle Detection . . . 119

7.1.11 Topological Sorting . . . 121

8 Graph Optimization: MST, Shortest Paths, Max Flow, Matchings and Stable Marriage 123

8.1 Minimum Spanning Trees . . . 123
8.1.1 Useful Facts about Connected Graphs . . . 123
8.1.2 The Minimum Spanning Tree Problem . . . 123
8.1.3 Prim's Algorithm . . . 124
8.1.4 Correctness: the Cut Lemma . . . 124
8.1.5 Implementation of Prim's Algorithm . . . 125
8.1.6 Worked Example for Prim and Kruskal . . . 126
8.1.7 Kruskal's Algorithm . . . 127
8.1.8 The Union-Find Data Structure . . . 128
8.1.9 Correctness of Kruskal's Algorithm . . . 130
8.1.10 Exercises on Minimum Spanning Trees . . . 130
8.2 Shortest Paths . . . 132
8.2.1 The Shortest Path Problem . . . 132
8.2.2 Subpath Optimality and Relaxation . . . 132
8.2.3 Bellman–Ford: Relax Everything $|V| - 1$ Times . . . 133
8.2.4 Shortest Paths in a DAG . . . 134
8.2.5 Dijkstra's Algorithm . . . 135
8.2.6 All-Pairs Shortest Paths . . . 136
8.2.7 Floyd–Warshall . . . 137
8.2.8 Exercises on Shortest Paths . . . 138
8.3 Maximum Flow . . . 140
8.3.1 Flows and Cuts . . . 140
8.3.2 Ford–Fulkerson . . . 140
8.3.3 Running Time and Edmonds–Karp . . . 141
8.3.4 Applications of Maximum Flow . . . 142
8.3.5 Exercises on Maximum Flow . . . 142
8.4 Matchings in Bipartite Graphs . . . 144
8.4.1 Definitions . . . 144
8.4.2 Reduction to Maximum Flow . . . 144
8.4.3 Augmenting Paths and Berge's Theorem . . . 145
8.4.4 Hall's Theorem and Vertex Covers . . . 146
8.4.5 Worked Applications . . . 146
8.4.6 Exercises on Matchings . . . 147
8.5 Stable Marriage . . . 148
8.5.1 The Problem . . . 148
8.5.2 The Propose-and-Reject Algorithm . . . 149
8.5.3 Correctness . . . 149
8.5.4 An $O(n^2)$ Implementation . . . 149
8.5.5 Optimality for the Proposing Side . . . 150
8.5.6 Exercises on Stable Marriage . . . 151

9 Intractability, Approximation and Local Search 152

9.1 Intractability . . . 152
9.1.1 Decision Problems versus Optimisation Problems . . . 152
9.1.2 Polynomial Time and the Class P . . . 152

9.1.3 Verification and the Class NP . . . 152
9.1.4 Reductions and NP-Completeness . . . 153
9.1.5 The First NP-Complete Problems . . . 154
9.1.6 A Catalogue of NP-Complete Problems . . . 154
9.2 Approximation Algorithms . . . 156
9.2.1 Approximation Ratio and Schemes . . . 156
9.2.2 Bin Packing . . . 156
9.2.3 Knapsack . . . 157
9.2.4 The k-Center Problem . . . 157
9.2.5 Vertex Cover . . . 158
9.2.6 The Travelling Salesman Problem . . . 158
9.3 Local Search . . . 160
9.3.1 Local Search for Vertex Cover . . . 160
9.3.2 Hopfield Neural Networks . . . 160
9.3.3 The Maximum Cut Problem . . . 160

10 Parallel Algorithms **163**
10.1 Parallel Algorithms . . . 163
10.1.1 The PRAM Model and Work–Depth . . . 163
10.1.2 The Summation Problem . . . 163
10.1.3 Prefix Sums . . . 164
10.1.4 Merging and Ranking . . . 164
10.1.5 Maximum Finding . . . 164
10.1.6 Sorting by Merging . . . 165

1 Prerequisites: Asymptotic Notations

O, Ω, Θ are used to describe the asymptotic behavior of functions.

Definitions of Asymptotic Notations

(a) **Big-O (Upper Bound):** $f(n) = O(g(n))$ means that there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

(b) **Big-Omega (Lower Bound):** $f(n) = \Omega(g(n))$ means that there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} > 0$$

(c) **Big-Theta (Tight Bound):** $f(n) = \Theta(g(n))$ means that there exist positive constants c_1, c_2 and n_0 such that $c_1 g(n) \leq f(n) \leq c_2 g(n)$ for all $n \geq n_0$.

$$0 < \lim_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

(d) **Not Comparable:** $f(n)$ and $g(n)$ are not comparable if $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ does not exist.

Asymptotic Growth Hierarchy (Memorize This Order)

constant < logarithmic < polynomial < exponential

$$O(1) \subset O(\log\log n) \subset O(\log n) \subset O(\sqrt{n}) \subset O(n) \subset O(n\log n)$$
$$\subset O(n^2) \subset O(n^k) \subset O(2^n) \subset O(n!) \subset O(n^n)$$

Common Mistakes with Asymptotic Notation

- $O(g(n))$ describes an **upper bound**, not necessarily tight. $n = O(n^2)$ is technically true but not useful.
- $\Omega(g(n))$ describes a **lower bound**. Best-case $\neq$ lower bound!
- $\Theta(g(n))$ requires **both** upper and lower bounds to match.
- Always drop constant factors and lower-order terms: $3n^2 + 5n + 7 = \Theta(n^2)$, <u>not</u> $\Theta(3n^2)$.

2024–25 Spring, Problem 2 (Asymptotic complexity, 5 points)

Problem. For each case fill in **A** (algorithm A eventually always faster), **B** (algorithm B faster) or **U** (information insufficient).

Case	$T_A(n)$	$T_B(n)$	**Faster**	**Reason**
0	$\Theta(n)$	$\Theta(n^2)$	A	$n < n^2$
1	$\Theta\big(\sum_{i=1}^{n} i\sqrt{n}\big)$	$\Theta(n^2)$	B	$\sqrt{n} \cdot \frac{n(n+1)}{2} = \Theta(n^{2.5})$
2	$O\big((\log n)^{10}\big)$	$\Omega(n^{0.1})$	A	polylog beats any positive power
3	$\Theta(2^n)$	$\Theta(2^{n/2})$	A	ratio $2^{n/2} \to \infty$
4	$O\big(\sum_{i=1}^{n} 2^{-i}\big)$	$\Omega(\log n)$	A	geometric sum is $O(1)$
5	$\Theta\big(\sum_{i=1}^{n} i\sqrt{i}\big)$	$\Theta(n^{4/3})$	B	$\sum i^{1.5} = \Theta(n^{2.5})$

Solution. Cases 3 and 5 are the traps: (3) is exponential vs. exponential, and the bases decide — 2^n grows like $(2^{1/2})^{2\cdot}$ times faster; (5) needs the integral bound $\sum_{i=1}^{n} i^p = \Theta(n^{p+1})$ for $p > -1$. Case 2 is the one place where "O vs. Ω" is decisive: a polylogarithmic upper bound is eventually below any $\Omega(n^\epsilon)$ lower bound, so A is provably faster. Case 0–1 illustrate that O/Θ claims must be simplified before comparing. ■

2025–26 Spring, Problem 1(a) (Asymptotics table, 5 points)

Problem. Fill in A / B / U.

Case	$T_A(n)$	$T_B(n)$	**Answer**	**Reason**
0	$\Theta(n)$	$\Theta(n^2)$	A	given as example
1	$\Theta\big(n^2/(\log n)^{10}\big)$	$\Theta(n^{1.5})$	B	$\frac{T_A}{T_B} = \frac{n^{0.5}}{(\log n)^{10}} \to \infty$
2	$\Theta\big(2^{\sqrt{n}}\big)$	$\Theta\big(10^{\sqrt[3]{n}}\big)$	B	$2^{n^{1/2}}$ vs. $2^{O(n^{1/3})}$
3	$O(\sqrt{\log n})$	$\Omega(\log\log n)$	U	O vs. Ω: no comparison possible
4	$\Theta(5^n)$	$\Theta(7^n)$	A	ratio $\left(\frac{5}{7}\right)^n \to 0$
5	$\Theta(n^2)$	$T_B(n) = 2T_B\left(\frac{n}{2}\right) + n$	B	$T_B = \Theta(n \log n)$

Solution. Case 1 rewards simplifying before comparing: a polylogarithmic factor never competes with any n^ϵ. Case 2 is exponential-vs-exponential in disguise — take $\log_2$ of both: $\sqrt{n}$ versus $(\log_2 10)\, n^{1/3}$, and $n^{1/2} > n^{1/3}$. Case 3 is the only U: one bound is an upper bound and the other a lower bound, so no ordering is justified (e.g. T_A could be constant). Case 5 needs the Master Theorem applied to B first. ■

What the Midterms Repeatedly Test

- **Recursion trees**: always write the number of nodes per level, the cost per node, and the level sum. When the log contains the level index (e.g. $n \log_4(n/4^i)$), expand it into an arithmetic series before summing.
- **Lower bounds**: count the number of possible outputs ($\binom{n}{k} k!$, $n!$) and take the log. When one side is O and the other is Ω, answer "U" unless the bounds are provably separated.
- **Heaps**: insertions follow the bubble-up path; for merging m heaps and bottom-up build-up use $O(k2^{m-k})$ and $\sum_k k/2^{k+1} \leq 1$.
- **Randomization**: always split into geometric stages ($\mathbb{E}[\text{good tries}] = 2$) and sum a shrinking geometric series. Remember coupon collector $= \Theta(n)$ for $\frac{n}{2}$ coupons and $\Theta(n \log n)$ for all of them.
- **Sorted array with few distinct values**: binary-search for "the last position holding this value", giving $O(\log^2 n)$.

2 Divide-and-Conquer and Randomized Algorithms

2.1 Divide and Conquer

Solve a problem of size n by breaking it into one or more smaller problems of size less than n. Solve the smaller problems recursively and combine their solutions to solve the large problem.

Three Steps of D&C

1. **Divide:** Split the problem into subproblems.
2. **Conquer:** Solve subproblems recursively (or directly if small enough).
3. **Combine:** Merge subproblem solutions into the original solution.

2.1.1 Binary Search

Given a sorted array A and a target value x, find the index of x (or -1 if not found).

Algorithm 1 Binary Search

```
1:  function BinarySearch(A, x)
2:      low ← 0
3:      high ← length(A) − 1
4:      while low ≤ high do
5:          mid ← ⌊(low + high) / 2⌋
6:          if A[mid] = x then
7:              return mid
8:          else if A[mid] < x then
9:              low ← mid + 1
10:         else
11:             high ← mid − 1
12:         end if
13:     end while
14:     return −1
15: end function
```

Binary Search Analysis

Let $T(n)$ be the number of comparisons on an array of size n.

- Recurrence: $T(n) \leq T\left(\frac{n}{2}\right) + 2$, with $T(1) = T(0) = 1$.
- Expand: $T(n) \leq T\left(\frac{n}{2}\right) + 2 \leq T\left(\frac{n}{4}\right) + 4 \leq \cdots \leq T(1) + 2\log_2 n = 1 + 2\log_2 n$.
- Therefore: $T(n) = O(\log n)$.

Algorithm 2 Binary Search (Recursive Version)

```
1: function BinarySearch(A, l, r, x)
2:     if l > r then
3:         return nil                                   ▷ x not present
4:     end if
5:     m ← ⌊(l + r)/2⌋
6:     if A[m] = x then
7:         return m
8:     else if x < A[m] then
9:         return BinarySearch(A, l, m − 1, x)
10:    else
11:        return BinarySearch(A, m + 1, r, x)
12:    end if
13: end function
```

Worked example: search for $x = 42$ in

$$A = [4, 7, 10, 15, 19, 20, 42, 54, 87, 90].$$

Call	**Range**	m	$A[m]$ **vs.** 42
1	$[1, 10]$	5	$19 < 42 \Rightarrow$ go right
2	$[6, 10]$	8	$54 > 42 \Rightarrow$ go left
3	$[6, 7]$	6	$20 < 42 \Rightarrow$ go right
4	$[7, 7]$	7	$A[7] = 42$ **found**!

Recursion-Tree View and the $\Omega(\log n)$ Lower Bound

The recursion tree is a single path: exactly one subproblem per level, costing 2 comparisons per internal level and 1 at the leaf, giving $2\log_2 n + 1$ comparisons — equivalent to expansion but more visual.

- Upper bound: each level halves the range, so at most $\lfloor \log_2 n \rfloor + 1$ levels $\Rightarrow T(n) = O(\log n)$.
- Lower bound: an element sitting at a deepest leaf (or a failed search) forces traversing the full path of length $\log_2 n$, so $T(n) = \Omega(\log n)$.

Hence $T(n) = \Theta(\log n)$. More generally, the decision-tree model proves no search algorithm on a sorted array can beat $\Omega(\log n)$ comparisons (there are $n + 1$ possible answers: n positions plus "not found").

Variant: Find the Boundary (lower_bound) Consider the variant with **no equality test**:

Algorithm 3 Binary Search without Equality Test

```
1: function FindBoundary(A, l, r, x)
2:     if l = r then
3:         return l
4:     end if
5:     m ← ⌊(l + r) / 2⌋
6:     if A[m] < x then
7:         return FindBoundary(A, m + 1, r, x)
8:     else
9:         return FindBoundary(A, l, m, x)
10:    end if
11: end function
```

What Does This Variant Return?

- It returns the index of the **first** element $\geq x$ — the insertion position even if x is absent (C++ `std::lower_bound`, Python `bisect.bisect_left`).
- Changing $A[m] < x$ to $A[m] \leq x$ returns the first element **strictly greater** than x (`upper_bound`, `bisect_right`); the two answers differ exactly when x occurs in A.
- Running time is still $O(\log n)$: every call halves a non-empty range.

Tail recursion into a loop. Because the recursive call is the very last action (tail recursion), it rewrites directly as a `while` loop:

```
l = 1; r = n;
while (l < r) {
    m = (l + r) / 2;
    if (A[m] < x) l = m + 1;
    else          r = m;
}
return l;
```

The iterative form avoids function-call overhead, uses $O(1)$ instead of $O(\log n)$ space, and cannot overflow the call stack — this is why library routines are written this way.

When to Use Binary Search

Binary search works whenever you can discard half the search space with one comparison. The array need not be literally sorted—it just needs a "monotonic" property that lets you decide which half contains the answer.

Exercise 1: Rotated Sorted Array Let $A[1..n]$ be a sorted array of n distinct numbers that has been rotated $n - k$ steps for some unknown integer $k \in [1, n-1]$. That is, $A[1..k]$ is sorted increasing, $A[k+1..n]$ is sorted increasing, and $A[n] < A[1]$.

Example: $n = 16$, $k = 10$:

$$A = [9, 13, 16, 18, 19, 23, 28, 31, 37, 42, 0, 1, 2, 5, 7, 8]$$

(1) Find the rotation index k in $O(\log n)$. ($A[k]$ is the maximum element.)

Algorithm 4 Find Rotated Index

```
1: function FindRotatedIndex(A, l, r)
2:     mid ← ⌊(l + r)/2⌋
3:     if A[mid] > A[mid + 1] then
4:         return mid
5:     else if A[mid] ≥ A[l] then
6:         return FindRotatedIndex(A, mid + 1, r)
7:     else
8:         return FindRotatedIndex(A, l, mid − 1)
9:     end if
10: end function
```

(2) Search for x in $O(\log n)$.

Algorithm 5 Search in Rotated Sorted Array

```
1: function SearchRotated(A, x, n)
2:     k ← FindRotatedIndex(A, 1, n)
3:     if x ≥ A[1] then
4:         return BinarySearch(A[1..k], x)
5:     else
6:         return BinarySearch(A[k + 1..n], x)
7:     end if
8: end function
```

Exercise 2: Finding the Last 0 Array $A[1..n]$ contains 0s followed by 1s.

(1) Standard $O(\log n)$ solution.

Algorithm 6 Find Last 0

```
1: function FindLast0(A, l, r)
2:     mid ← ⌊(l + r)/2⌋
3:     if A[mid] = 0 and A[mid + 1] = 1 then
4:         return mid
5:     else if A[mid] = 0 then
6:         return FindLast0(A, mid + 1, r)
7:     else
8:         return FindLast0(A, l, mid − 1)
9:     end if
10: end function
```

(2) $O(\log k)$ solution when $k \ll n$. Exponential search (gallop): double the index until we overshoot, then binary search.

Algorithm 7 Find Last 0 with $O(\log k)$

$i \leftarrow 1$
while $i < n \wedge A[i] = 0$ **do**
  $i \leftarrow \min\{2i, n\}$
end while
if $A[i] = 0$ **then**
  return i ▷ all zeros: last 0 is n
else if $i = 1$ **then**
  return -1 ▷ no zero
else
  return FindLast0$(A, \left\lfloor \frac{i}{2} \right\rfloor, i)$
end if

2024–25 Spring, Problem 6 (Searching in a sorted array with few distinct values, 20 points)

Problem. $A[1..n]$ is non-decreasing and contains $\Theta(\log n)$ distinct numbers, but the count is not given. Output every distinct value together with its frequency in $O(\log^2 n)$ time (a left-to-right scan costs $O(n)$ and is too slow). Example: $A =$ `10 10 12 12 12 15 15 20 20 30` $\Rightarrow$ `(10,2),(12,3),(15,2),(20,2),(30,1)`.

Solution (binary search for the last occurrence of each run). A decomposes into $m = \Theta(\log n)$ maximal constant runs $A[1..k_1], A[k_1+1..k_2], \ldots, A[k_{m-1}+1..n]$.

function LastOcc(A, p, r, x) ▷ precondition: $A[p] = x$, returns the last index of x
  if $p = r$ **or** $A[r] = x$ **then**
    return r
  end if
  $q \leftarrow \left\lfloor \frac{(p+r)}{2} \right\rfloor$
  if $A[q] = x$ **and** $A[q+1] > x$ **then**
    return q
  end if
  if $A[q] = x$ **then**
    return LastOcc$(A, q+1, r, x)$
  else
    return LastOcc$(A, p, q-1, x)$
  end if
end function

Start with $p = 1$; repeatedly $k_i \leftarrow$ LastOcc$(A, p, n, A[p])$, output $(A[p], k_i - p + 1)$, then set $p \leftarrow k_i + 1$. Stop once $k_i = n$.

Correctness. Each call is given $A[p] = x$, so the returned k satisfies $A[k] = x$ and $A[k+1] > x$ (or $k = n$): the run of x is exactly $A[p..k]$, giving the correct frequency; the next run starts at $k+1$, so no value is skipped or counted twice.

Running time. LastOcc halves the range each call — $O(\log n)$. There are $m = \Theta(\log n)$ runs, so the total is $O(\log^2 n)$. ■

2025–26 Fall, Problem 4 (Divide and conquer: k-th of two sorted arrays, 10 points)

Problem. $A[1..m]$, $B[1..n]$ are sorted increasing with distinct values; given $k \in [1, m+n]$, find the k-th smallest of $A \cup B$ in $O(\log k)$ time. Example: $A = [2, 5, 7]$, $B = [3, 6]$, $k = 3 \Rightarrow 5$.

Solution. Search for the k-th element of $A[p..q] \cup B[r..s]$, maintaining the invariant that the answer is the k-th smallest of the current union.

- If $k = 1$, return $\min\{A[p], B[r]\}$.

- Otherwise let $h = \left\lfloor \frac{k}{2} \right\rfloor \geq 1$ and compare $A[p+h-1]$ with $B[r+h-1]$ (a probe that runs past the end of an array is treated as $+\infty$). At most one probe overflows because $k \leq (q-p+1)+(s-r+1)$.
- If $A[p+h-1] < B[r+h-1]$: exactly h elements of A are $\leq A[p+h-1]$, and at most $h-1$ elements of B are, so its rank is at most $2h-1 \leq k-1$. Hence $A[p..p+h-1]$ contains no element of rank k and is discarded, with $k \leftarrow k-h$. Symmetrically, if $B[r+h-1] < A[p+h-1]$ discard $B[r..r+h-1]$.

Correctness. Each step removes exactly those elements that provably cannot be the k-th smallest, and reduces the rank by the number removed; the invariant "the answer is the k'-th smallest of the current union" is preserved until $k' = 1$.

Time. The rank goes $k \to k - \left\lfloor \frac{k}{2} \right\rfloor = \left\lceil \frac{k}{2} \right\rceil$, halving each step, so there are $O(\log k)$ calls of $O(1)$ work: $O(\log k)$.

Worked example $A = [2,5,7]$, $B = [3,6]$, $k = 3$: $h = 1$, compare $A[1] = 2$ with $B[1] = 3$; $2 < 3$, so discard $A[1]$ and set $k \leftarrow 2$. Now $A = [5,7]$, $B = [3,6]$, $k = 2$: $h = 1$, compare 5 with 3; $3 < 5$, discard $B[1]$ and set $k \leftarrow 1$. Finally $k = 1$ returns $\min\{5,6\} = 5$. ■

2025–26 Spring, Problem 3 (Searching with $\Theta(\log n)$ distinct values, 20 points)

Problem. Identical to 2024–25 Spring Problem 6: output the distinct values of a non-decreasing array and their frequencies in $O(\log^2 n)$ time, without being given the number of distinct values.

Solution. Reuse the LastOcc divide-and-conquer from that problem: for each run start p, binary search the last index k with $A[k] = A[p]$ in $O(\log n)$, emit $(A[p], k-p+1)$, and continue at $k+1$. There are $m = \Theta(\log n)$ runs, so the total is $O(\log^2 n)$.

Why a plain binary search is not enough. Ordinary binary search finds an index of x; we need the right end of the run. The precondition $A[p] = x$ guarantees termination at $A[k+1] > x$, and the "$A[r] = x$" shortcut handles an all-equal suffix in $O(1)$. ■

2.1.2 Mergesort & Inversion Count

Warm-up: Tower of Hanoi Three pegs, n disks of decreasing size. Move all disks from source to target peg.

Hanoi Recurrence

Let $T(n)$ = minimum moves for n disks.

- Basis: $T(1) = 1$.
- Recursive step: move $n-1$ disks source→aux, move largest source→target, move $n-1$ aux→target.

$$T(n) = 2T(n-1) + 1$$

- Solution: $T(n) = 2^n - 1$. (Expand: $T(n) = 2^k T(n-k) + (2^k - 1)$, set $k = n-1$.)

Algorithm 8 Hanoi Tower

```
1: function HANOI(n, A, B, C)
2:     if n = 1 then
3:         Move disk from A to C
4:     else
5:         HANOI(n − 1, A, C, B)
6:         Move disk from A to C
7:         HANOI(n − 1, B, A, C)
8:     end if
9: end function
```

Merge: Combining Two Sorted Halves Given two sorted subarrays $A[p..q]$ and $A[q+1..r]$, merge them into $A[p..r]$ sorted.

Algorithm 9 Merge

```
1: function MERGE(A, p, q, r)
2:     n1 ← q − p + 1;    n2 ← r − q
3:     Create arrays L[1..n1 + 1], R[1..n2 + 1]
4:     for i = 1 to n1 do
5:         L[i] ← A[p + i − 1]
6:     end for
7:     for j = 1 to n2 do
8:         R[j] ← A[q + j]
9:     end for
10:    L[n1 + 1] ← ∞;    R[n2 + 1] ← ∞                    ▷ sentinels
11:    i ← 1;    j ← 1
12:    for k = p to r do
13:        if L[i] ≤ R[j] then
14:            A[k] ← L[i];    i ← i + 1
15:        else
16:            A[k] ← R[j];    j ← j + 1
17:        end if
18:    end for
19: end function
```

Merge Time Complexity

Merge runs in $\Theta(n)$ time on $n = r - p + 1$ elements: two copy loops ($n_1 + n_2 = n$) plus one comparison loop (n iterations). Sentinels avoid boundary checks — clean but not cache-friendly in practice.

Worked example: merge the sorted halves $L = [2, 4, 5, 7, \infty]$ and $R = [1, 2, 3, 6, \infty]$:

k	**compare**	**winner**	$A[p \dots r]$ **so far**
1	$L[1] = 2$ vs $R[1] = 1$	$R[1]$	$[1]$
2	$L[1] = 2$ vs $R[2] = 2$	$L[1]$ ($\leq$ keeps it stable)	$[1, 2]$
3	$L[2] = 4$ vs $R[2] = 2$	$R[2]$	$[1, 2, 2]$
4	$L[2] = 4$ vs $R[3] = 3$	$R[3]$	$[1, 2, 2, 3]$
5	$L[2] = 4$ vs $R[4] = 6$	$L[2]$	$[1, 2, 2, 3, 4]$
6	$L[3] = 5$ vs $R[4] = 6$	$L[3]$	$[1, 2, 2, 3, 4, 5]$
7	$L[4] = 7$ vs $R[4] = 6$	$R[4]$	$[1, 2, 2, 3, 4, 5, 6]$
8	$L[4] = 7$ vs ∞	$L[4]$	$[1, 2, 2, 3, 4, 5, 6, 7]$

Each step advances the head pointer (i or j) of the list whose element was copied; exactly one element is placed per iteration.

Algorithm 10 Mergesort

```
1: function MERGESORT(A, p, r)
2:     if p < r then
3:         q ← ⌊(p + r)/2⌋                          ▷ Divide
4:         MERGESORT(A, p, q)                       ▷ Conquer left
5:         MERGESORT(A, q + 1, r)                   ▷ Conquer right
6:         MERGE(A, p, q, r)                        ▷ Combine
7:     end if
8: end function
```

Mergesort Algorithm

Mergesort Analysis

Recurrence for n elements (power of 2 for simplicity):

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1. \end{cases}$$

Solution: $T(n) = \Theta(n \log n)$. (Master Theorem: $a = 2, b = 2, f(n) = \Theta(n)$, case 2.)

Worked example: $A = [5, 2, 4, 7, 1, 3, 2, 6]$.

Stage	**Array state**
input	$[5, 2, 4, 7 \mid 1, 3, 2, 6]$
after sorting left half recursively	$[2, 4, 5, 7 \mid 1, 3, 2, 6]$
after sorting right half recursively	$[2, 4, 5, 7 \mid 1, 2, 3, 6]$
after $\Theta(n)$ merge	$[1, 2, 2, 3, 4, 5, 6, 7]$

Solving the Recurrence by Expansion / Recursion Tree

Expanding directly:

$$\begin{aligned} T(n) &= 2T(n/2) + n = 2^2 T(n/4) + 2n = 2^3 T(n/8) + 3n \\ &= 2^i T(n/2^i) + in = nT(1) + n \log_2 n = n \log n + n. \end{aligned}$$

On the recursion tree, level i has 2^i nodes, each paying $n/2^i$, so the cost of every level is $2^i \cdot (n/2^i) = n$; with $\log_2 n + 1$ levels the total is $n(\log_2 n + 1)$.

Why $\Omega(n \log n)$ Too?

The merge step costs $\Theta(n)$ on every input — there is no lucky case that skips work. Replacing each O by Ω in the analysis is still valid, so $T(n) = \Omega(n \log n)$ and overall $T(n) = \Theta(n \log n)$: every input is a worst-case input for mergesort.

Mergesort Properties

- **Stable:** yes (preserves order of equal keys if $L[i] \leq R[j]$ uses $\leq$, not $<$).
- **In-place:** no (needs auxiliary arrays L and R; $\Theta(n)$ extra space).
- **Best / Worst / Average:** all $\Theta(n \log n)$ (unlike Quicksort).

Counting Inversions An **inversion** is a pair (i, j) with $i < j$ and $A[i] > A[j]$. Count the total number of inversions.

Brute Force is Too Slow

Checking all $\binom{n}{2}$ pairs takes $\Theta(n^2)$ time. With D&C we can do $\Theta(n \log n)$ by **piggybacking on Mergesort**.

Algorithm 11 Count Inversions via Mergesort

```
1: function CountInv(A, p, r)
2:     if p ≥ r then
3:         return 0
4:     end if
5:     q ← ⌊(p + r)/2⌋
6:     left ← CountInv(A, p, q)
7:     right ← CountInv(A, q + 1, r)
8:     cross ← MergeCount(A, p, q, r)
9:     return left + right + cross
10: end function
11: function MergeCount(A, p, q, r)
12:     ... same as Merge, but whenever R[j] is copied before L[i] is exhausted:
13:     add (n_1 − i + 1) to cross (all remaining L[i..n_1] are > R[j])
14:     return cross
15: end function
```

Why Cross-Inversions Work

When copying $R[j]$ before $L[i]$ is empty, **every remaining element in L forms an inversion with $R[j]$** because L indices $< R$ indices (by array split) and $L[i..n_1] > R[j]$ (since we picked $R[j]$ first). This adds exactly $n_1 - i + 1$ inversions in $O(1)$.

Applications. The inversion number measures (i) how "sorted" an array is ($0 =$ sorted, $\binom{n}{2} =$ reverse sorted), and (ii) the similarity between two rankings: map one ranking to the identity permutation and count inversions in the other.

Worked example: $A = [4, 8, 10, 2, 1, 5, 12, 11, 3, 7, 6, 9]$.

Split into left half $[4, 8, 10, 2, 1, 5]$ and right half $[12, 11, 3, 7, 6, 9]$:

- 5 inversions inside the left half and 8 inside the right half (found recursively).
- 9 cross inversions: $(5, 3)$, $(4, 3)$, $(8, 6)$, $(8, 3)$, $(8, 7)$, $(10, 6)$, $(10, 9)$, $(10, 3)$, $(10, 7)$.
- Total: $5 + 8 + 9 =$ **22**.

Counting cross inversions during the merge. With 1-indexed arrays split at m, when $R[j]$ is copied while the L-pointer is at i, the blue elements already passed ($L[1..i-1]$) are $\leq R[j]$; every remaining blue element $L[i..m]$ is $> R[j]$, so the contribution of $R[j]$ is

$$I[j] = m - i + 1 \quad \big(\text{or, with the slide's indexing, } m - l - i + 2\big).$$

Tiny trace: sorted halves $L = [1, 3, 5, 7]$, $R = [2, 4, 6, 8]$:

Element copied from R	**pointer i**	**$L[i..]$ greater than it**	$I[j]$
2	1 ($L[1] = 1 \leq 2$ passed, then $i = 2$)	$3, 5, 7$	3
4	3	$5, 7$	2
6	4	7	1
8	past end	—	0

Cross inversions $= 3+2+1+0 = 6$; the merge simultaneously produces $[1,2,3,4,5,6,7,8]$, preserving the sortedness invariant needed by the parent call.

Recursion-tree accounting. Each leaf returns 0; each internal node returns its cross count from the merge, and a node reports the sum of its two children plus its own cross count. For the 12-element example the two size-6 root children return 5 (blue half) and 8 (green half), and the root merge contributes 9:

root: $5 + 8 + 9 = 22$
left child `Sort-and-Count`$[4,8,10,2,1,5] \to 5$
right child `Sort-and-Count`$[12,11,3,7,6,9] \to 8$
(their values are sums of cross counts at lower levels; leaves contribute 0)

Since both the counting and the merging are linear scans, `Merge-and-Count` costs $\Theta(n)$ and the overall recurrence is again $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n)$.

Algorithm 12 Merge-and-Count (precondition: $A[l..m]$ and $A[m+1..r]$ sorted)

copy $A[l..m]$ into $L[1..n_1]$, $A[m+1..r]$ into $R[1..n_2]$, append ∞ sentinels
$i \leftarrow 1$; $j \leftarrow 1$; $c \leftarrow 0$
for $k = l$ **to** r **do**
  if $L[i] \le R[j]$ **then**
    $A[k] \leftarrow L[i]$; $i \leftarrow i + 1$
  else
    $A[k] \leftarrow R[j]$; $j \leftarrow j + 1$
    $c \leftarrow c + (n_1 - i + 1)$ ▷ all remaining L elements invert $R[j]$
  end if
end for
return c

2025–26 Fall, Problem 1(a) (Recursion tree, 4 points)

Problem. For n a power of 4 and

$$T(n) = 4T(n/4) + n \log_4 n, \qquad T(1) = 1,$$

derive the exact asymptotic solution by the recursion tree method, showing for levels $0, 1, 2$ the level number, per-node cost and level sum, then the general level i, the number of leaves and the sum of leaf costs.

Solution. Level i has 4^i nodes of size $n/4^i$, each costing

$$\frac{n}{4^i} \log_4 \frac{n}{4^i} = \frac{n}{4^i}\big(\log_4 n - i\big).$$

Level	# nodes	Size per node	Cost per node	Level sum
0	1	n	$n \log_4 n$	$n \log_4 n$
1	4	$\frac{n}{4}$	$\frac{n}{4}(\log_4 n - 1)$	$n(\log_4 n - 1)$
2	4^2	$n/4^2$	$\frac{n}{4^2}(\log_4 n - 2)$	$n(\log_4 n - 2)$
i	4^i	$n/4^i$	$\frac{n}{4^i}(\log_4 n - i)$	$n(\log_4 n - i)$

The tree stops at level $L = \log_4 n$, so there are $4^L = n$ leaves, each costing $T(1) = 1$: leaf total $= n$. Summing levels $i = 0, \ldots, L-1$:

$$T(n) = n + \sum_{i=0}^{L-1} n(L - i) = n + n\frac{L(L+1)}{2} = \Theta\big(n(\log n)^2\big),$$

i.e. $\alpha = 1$, $\beta = 2$. Note the contrast with mergesort: here the per-level cost decreases ($n \log_4 n$, $n(\log_4 n - 1)$, ...) and the leaves are the dominant term class, but the arithmetic series still contributes an extra $\log n$ factor. ■

2.1.3 Maximum Subarray

Given an array $A[1..n]$, find indices i, j maximizing $\sum_{t=i}^{j} A[t]$ (the empty subarray is not allowed). (Useful for best-buy/best-sell stock problem.)

Motivating example. Yearly profits (M$) $[3, 2, 1, -7, 5, 2, -1, 3, -1]$: the best span is years 5–8 with $5 + 2 - 1 + 3 = 9$ M$.

Two Simpler Solutions First

1. **Brute force $\Theta(n^3)$:** enumerate all pairs (i, j) and re-sum $A[i..j]$ with an inner loop. The j- and k-loops average $\frac{n}{2}$ and $\frac{n}{3}$ iterations.
2. **Data reuse $\Theta(n^2)$:** exploit $V(i, j) = V(i, j-1) + A[j]$ — extend the running sum instead of restarting, removing the inner loop.

The D&C solution improves this to $\Theta(n \log n)$, and Kadane's DP reaches $\Theta(n)$.

D&C Strategy

For a subarray $A[low..high]$ with midpoint mid, the maximum subarray is **exactly one** of:

1. Entirely in $A[low..mid]$ (recurse left).
2. Entirely in $A[mid + 1..high]$ (recurse right).
3. **Crossing the midpoint:** one end in left half, one end in right half (compute directly).

Return the maximum of the three.

Algorithm 13 Maximum Subarray (D&C)

```
1: function MaxSubarray(A, low, high)
2:     if low = high then
3:         return (low, high, A[low])
4:     end if
5:     mid ← ⌊(low + high)/2⌋
6:     (left_lo, left_hi, left_sum) ← MaxSubarray(A, low, mid)
7:     (right_lo, right_hi, right_sum) ← MaxSubarray(A, mid + 1, high)
8:     (cross_lo, cross_hi, cross_sum) ← MaxCrossing(A, low, mid, high)
9:     if left_sum ≥ right_sum and left_sum ≥ cross_sum then
10:        return (left_lo, left_hi, left_sum)
11:    else if right_sum ≥ left_sum and right_sum ≥ cross_sum then
12:        return (right_lo, right_hi, right_sum)
13:    else
14:        return (cross_lo, cross_hi, cross_sum)
15:    end if
16: end function
17: function MaxCrossing(A, low, mid, high)
18:    left_sum ← −∞;    sum ← 0
19:    for i = mid downto low do
20:        sum ← sum + A[i]
21:        if sum > left_sum then
22:            left_sum ← sum;    max_left ← i
23:        end if
24:    end for
25:    right_sum ← −∞;    sum ← 0
26:    for j = mid + 1 to high do
27:        sum ← sum + A[j]
28:        if sum > right_sum then
29:            right_sum ← sum;    max_right ← j
30:        end if
31:    end for
32:    return (max_left, max_right, left_sum + right_sum)
33: end function
```

Max Subarray Analysis

Recurrence:

$$T(n) = \begin{cases} \Theta(1) & n = 1, \\ 2T(n/2) + \Theta(n) & n > 1. \end{cases}$$

Same as Mergesort: $T(n) = \Theta(n \log n)$.

Kadane's Algorithm (iterative DP, not D&C): $O(n)$ time, $O(1)$ space.

$$\text{ending_here}_i = \max\big(A[i],\ \text{ending_here}_{i-1} + A[i]\big),$$
$$\text{best}_i = \max\big(\text{best}_{i-1},\ \text{ending_here}_i\big).$$

Kadane trace on $A = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$ (start $V = V_{\max} = -\infty$):

i	$A[i]$	$V = \max(V + A[i], A[i])$ **(best ending at i)**	$V_{\max}$	**best window so far**
1	-2	-2	-2	$[-2]$
2	1	1 (restart)	1	$[1]$
3	-3	-2	1	$[1]$
4	4	4 (restart)	4	$[4]$
5	-1	3	4	$[4]$
6	2	5	5	$[4, -1, 2]$
7	1	6	**6**	$[4, -1, 2, 1]$
8	-5	1	6	$[4, -1, 2, 1]$
9	4	5	6	$[4, -1, 2, 1]$

Answer: 6, achieved by $[4, -1, 2, 1]$. The "restart or extend" choice at every step is the DP decision.

Application: Maximizing Stock Profit Given prices $p[1..n]$, buy once and sell later ($i < j$) to maximize $p[j] - p[i]$ (a trade is mandatory).

Direct D&C: split at m. The optimal pair is either (1) entirely in the left half, (2) entirely in the right half, or (3) **crossing**: buy at the minimum price in $p[l..m]$ and sell at the maximum price in $p[m+1..r]$, found by two linear scans. Same recurrence $2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n)$.

Reduction to Maximum Subarray ($O(n)$ preprocessing)

Build the difference array of consecutive-day changes:

$$\text{Profit}[i] = p[i+1] - p[i].$$

Every interval telescopes:

$$\sum_{t=i}^{j-1} \text{Profit}[t] = \big(p[i+1] - p[i]\big) + \big(p[i+2] - p[i+1]\big) + \cdots + \big(p[j] - p[j-1]\big) = p[j] - p[i].$$

So the maximum-profit trade is exactly the maximum subarray of `Profit`, solvable by Kadane in $\Theta(n)$.

Example: prices $[3, 6, 10, 1, 8, 2, 6, 5, 10, 3, 1, 9]$ give differences

$$[3, 4, -9, 7, -6, 4, -1, 5, -7, -2, 8],$$

whose maximum subarray identifies the best buy/sell pair.

2.1.4 Multiplication and Master Theorem

Integer Multiplication (Karatsuba) Given two n-bit integers x and y, compute $x \cdot y$.

Grade-School Multiplication

The standard pencil-and-paper algorithm is $\Theta(n^2)$: multiply each bit of y by x (shift & add), giving n rows of $O(n)$ bits each. Example: 163×97:

$$\begin{array}{rrr} & & 163 \\ \times & & 97 \\ \hline & 1141 & (163 \times 7) \\ + \quad 1467 & & (163 \times 9, \text{ shifted}) \\ \hline & & 15811 \end{array}$$

In binary, 10100011×01100001 similarly produces n shifted partial products: $\Theta(n^2)$ single-bit multiplications and $\Theta(n^2)$ binary additions.

First D&C Attempt: Four Half-Size Products Split $a = 2^{n/2}a_1 + a_0$, $b = 2^{n/2}b_1 + b_0$ (high/low $\frac{n}{2}$ bits):

$$a \cdot b = 2^n a_1 b_1 + 2^{n/2}(a_1 b_0 + a_0 b_1) + a_0 b_0.$$

Multiplication by 2^k is just a k-bit left shift (e.g. $12 = 00001100$, then $12 \cdot 2^3 = 96 = 01100000$), so the four products $a_1b_1, a_1b_0, a_0b_1, a_0b_0$ are computed recursively and combined with $\Theta(n)$ additions:

$$T(n) = 4T(n/2) + \Theta(n), \qquad T(1) = 1.$$

Analysis: The Naive D&C Is Still $\Theta(n^2)$

Expanding: $T(n) = 4^i T(n/2^i) + n(2^{i-1} + \cdots + 2 + 1)$; at $i = \log_2 n$,

$$T(n) = n^2 + n(2^{\log_2 n} - 1) = n^2 + n(n-1) = \Theta(n^2).$$

On the recursion tree, level i costs $4^i \cdot n/2^i = n2^i$, so costs double each level and the leaves dominate ($4^{\log_2 n} = n^2$). Essentially the method still multiplies every bit of a by every bit of b. The contrast with mergesort: one problem spawns **4** subproblems of size $\frac{n}{2}$, not 2.

Karatsuba Split

Write $x = a \cdot 2^{n/2} + b$, $y = c \cdot 2^{n/2} + d$ (split into high/low halves). Then:

$$x \cdot y = ac \cdot 2^n \; + \; \big[(a+b)(c+d) - ac - bd\big] \cdot 2^{n/2} \; + \; bd.$$

Instead of 4 half-size multiplications (ac, ad, bc, bd), we use **3**: ac, bd, and $(a+b)(c+d)$, subtracting to recover $ad + bc$.

Algorithm 14 Karatsuba Multiplication

function MULTIPLY(x, y, n)
  if $n = 1$ **then**
    return $x \cdot y$
  end if
  $a \leftarrow$ high $\frac{n}{2}$ bits of x; $\quad b \leftarrow$ low $\frac{n}{2}$ bits of x
  $c \leftarrow$ high $\frac{n}{2}$ bits of y; $\quad d \leftarrow$ low $\frac{n}{2}$ bits of y
  $ac \leftarrow$ MULTIPLY$(a, c, \frac{n}{2})$
  $bd \leftarrow$ MULTIPLY$(b, d, \frac{n}{2})$
  $ab_cd \leftarrow$ MULTIPLY$(a + b, c + d, \frac{n}{2})$
  return $ac \ll n \;+\; (ab_cd - ac - bd) \ll \left(\frac{n}{2}\right) \;+\; bd$
end function

Karatsuba Recurrence

$$T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n).$$

By Master Theorem case 1 ($\log_b a = \log_2 3 \approx 1.585 > 1$):

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}),$$

asymptotically better than $\Theta(n^2)$.

Decimal illustration ($n = 2$ digits, no recursion needed). Multiply 23×14: $a_1 = 2, a_0 = 3$, $c_1 = 1, d_0 = 4$.

$$ac = 2 \cdot 1 = 2, \qquad bd = 3 \cdot 4 = 12,$$
$$(a+b)(c+d) = 5 \cdot 5 = 25, \qquad ad + bc = 25 - 2 - 12 = 11,$$
$$23 \cdot 14 = 2 \cdot 100 + 11 \cdot 10 + 12 = 200 + 110 + 12 = \mathbf{322}.$$

Only **3** small multiplications $(2, 12, 25)$ instead of 4.

History and Practice

Karatsuba (1960) was the first algorithm to beat $\Theta(n^2)$. Splitting into r parts and using fewer than r^2 products gives the Toom–Cook family: Toom-3 $\Rightarrow \Theta(n^{\log_3 5}) = \Theta(n^{1.465})$. FFT-based methods reached $\Theta(n \log n \log\log n)$; a $\Theta(n \log n)$ integer-multiplication algorithm was proven by Harvey & van der Hoeven in 2019. Real big-integer libraries combine methods by input size: textbook for tiny inputs, Karatsuba/Toom for medium, FFT for very large.

Master Theorem for Recurrences For recurrences of the form:

$$T(n) = a\,T\Big(\frac{n}{b}\Big) + f(n), \quad a \geq 1,\ b > 1.$$

Compare $f(n)$ with $n^{\log_b a}$ (the "leaf work").

Master Theorem — Three Cases

1. **Case 1 (Leaves dominate):** If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$,

$$T(n) = \Theta(n^{\log_b a}).$$

2. **Case 2 (Balanced):** If $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$,
$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n).$$
3. **Case 3 (Root dominates):** If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and the **regularity condition** $a\, f\left(\frac{n}{b}\right) \leq c\, f(n)$ holds for some $c < 1$ and large n,
$$T(n) = \Theta(f(n)).$$

Common D&C Recurrences (Memorize!)

Algorithm	Recurrence	Solution
Binary search	$T(n) = T\left(\frac{n}{2}\right) + \Theta(1)$	$\Theta(\log n)$ (Case 2, $k = 0$)
Mergesort	$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$	$\Theta(n \log n)$ (Case 2, $k = 0$)
Karatsuba	$T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n)$	$\Theta(n^{\log_2 3})$ (Case 1)
Strassen matrix	$T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$	$\Theta(n^{\log_2 7})$ (Case 1)
Bad D&C (naive 4-mult)	$T(n) = 4T\left(\frac{n}{2}\right) + \Theta(n)$	$\Theta(n^2)$ (Case 1)

Why the Master Theorem Works (Recursion-Tree Intuition)

For the simplified form $T(n) = aT\left(\frac{n}{b}\right) + n$: level i contains a^i subproblems of size n/b^i and costs $n\left(\frac{a}{b}\right)^i$; there are $\log_b n$ levels and the leaves alone contribute $a^{\log_b n} = n^{\log_b a}$. Total:
$$T(n) = n^{\log_b a} + n \sum_{i=0}^{\log_b n - 1} \left(\frac{a}{b}\right)^i.$$
The geometric ratio $\frac{a}{b}$ decides everything:

- $a > b$: increasing geometric series, **leaves dominate**, $T(n) = \Theta(n^{\log_b a})$.
- $a = b$: every term is 1, each level costs n, $T(n) = \Theta(n \log n)$.
- $a < b$: decreasing geometric series is $\Theta(1)$, **the root dominates**, $T(n) = \Theta(n)$.

2024–25 Spring, Problem 3 (Recurrence, 5 points)

Problem. Solve $T(n) = 2T(\sqrt{n}) + \log_2 n$, with $T(c) = \Theta(1)$ for any convenient constant $c \geq 1$.
Solution (substitution to fit the Master Theorem). Let $k = \log_2 n$, i.e. $n = 2^k$, and define $S(k) = T(2^k)$. Then $\sqrt{n} = 2^{k/2}$ and
$$S(k) = 2S\left(\frac{k}{2}\right) + k.$$
Master Theorem, case 2 ($a = 2$, $b = 2$, $f(k) = k = \Theta(k)$): $S(k) = \Theta(k \log k)$. Substituting back,
$$T(n) = \Theta\big(\log n \cdot \log\log n\big).$$
Sanity check by expansion: at depth i the argument is $n^{1/2^i}$ and there are 2^i nodes of cost $\log(n^{1/2^i}) = (\log n)/2^i$ each, so every level costs exactly $\log n$. The recursion stops when $n^{1/2^i} = 2$, i.e. $i = \log\log n$, giving $\log n \cdot \log\log n$ — the same

answer without the Master Theorem. ■

2025–26 Spring, Problem 1(b) (Recurrence by expansion, 5 points)

Problem. For n a power of 4, solve $T(n) = 64\,T\left(\frac{n}{4}\right) + n^3$, $T(1) = 1$, without the Master Theorem, and write the answer as $\Theta(n^\alpha (\log n)^\beta)$.
Solution. Since $64 = 4^3$, unrolling k times gives

$$T(n) = 4^{3k} T(n/4^k) + k\,n^3.$$

Indeed each level contributes n^3: level j has $64^j = 4^{3j}$ nodes of size $n/4^j$, total $4^{3j}(n/4^j)^3 = n^3$. The base case is reached at $k = \log_4 n$, where $4^{3k} = n^3$ and $T(1) = 1$:

$$T(n) = n^3 + (\log_4 n)\,n^3 = \Theta\big(n^3 \log n\big),$$

so $\alpha = 3$, $\beta = 1$. This is exactly Master Theorem case 2: $a = 64$, $b = 4$, $n^{\log_b a} = n^3$ matches $f(n) = n^3$. ■

2.1.5 Matrix Multiplication and Strassen's Algorithm

Given two $n \times n$ matrices, compute $C = AB$ with $c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$. The brute-force triple loop costs $\Theta(n^3)$ (n^2 entries, each a length-n dot product).

Naive D&C: partition each matrix into four $\frac{n}{2} \times \frac{n}{2}$ blocks:

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix},$$

requiring **8** block multiplications (e.g. $C_{11} = A_{11}B_{11} + A_{12}B_{21}$) and $\Theta(n^2)$ block additions:

$$T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$$

(Master case 1: $\log_2 8 = 3$, and $n^{\log_2 8} = n^3$ dominates n^2). No improvement.

Strassen's Idea: 7 Block Products

Strassen (1969) computes the 2×2 block product with only **7** recursive multiplications plus $\Theta(n^2)$ additions/subtractions:

$$\begin{aligned}
&P_1 = A_{11}(B_{12} - B_{22}), && P_2 = (A_{11} + A_{12})B_{22}, && P_3 = (A_{21} + A_{22})B_{11},\\
&P_4 = A_{22}(B_{21} - B_{11}), && P_5 = (A_{11} + A_{22})(B_{11} + B_{22}), &&\\
&P_6 = (A_{12} - A_{22})(B_{21} + B_{22}), && P_7 = (A_{11} - A_{21})(B_{11} + B_{12}), &&
\end{aligned}$$

then

$$\begin{aligned}
C_{11} &= P_5 + P_4 - P_2 + P_6, & C_{12} &= P_1 + P_2,\\
C_{21} &= P_3 + P_4, & C_{22} &= P_5 + P_1 - P_3 - P_7.
\end{aligned}$$

Hence

$$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) = \Theta(n^{2.807\ldots}),$$

the first algorithm to beat cubic matrix multiplication. It is practical only for large dense matrices ($n \gtrsim 100$): the extra additions and recursion overhead make it slower on small matrices.

2.2 Randomized Algorithms

2.2.1 Why Randomization?

A randomized algorithm flips coins during execution. Worst-case analysis becomes **expected**-case analysis, often beating deterministic worst-case bounds.

Two Types of Randomized Algorithms

- **Las Vegas:** always correct; running time is a random variable (e.g. randomized Quicksort). We bound the expected runtime.
- **Monte Carlo:** always fast; has a small probability of being wrong (e.g. randomized min-cut / Karger). Repeat to reduce error.

How We Measure Randomized Running Time

We take the **expected running time on the worst-case input**: $\max_{I:|I|=n} \mathbb{E}_R[T(I,R)]$, where the expectation is only over the algorithm's internal randomness R (random pivots, random permutations, ...). Randomized algorithms are often simpler and faster in practice; whether randomization can give an asymptotic advantage over deterministic algorithms remains open for some problems (e.g. $O(1)$-time hash tables with $O(n)$ space).

2.2.2 Probability Review

For a discrete random variable X taking values with probabilities $\Pr[X = i]$:

$$\mathbb{E}[X] = \sum_i i \cdot \Pr[X = i].$$

Dice warm-ups:

- One fair 6-sided die: $\mathbb{E}[X] = \dfrac{1+2+\cdots+6}{6} = 3.5$.
- Sum of two dice: by linearity of expectation, $\mathbb{E}[X_1 + X_2] = 3.5 + 3.5 = \mathbf{7}$.
- Maximum of two dice: $\Pr[\max = k] = \dfrac{(k^2 - (k-1)^2)}{36} = \dfrac{(2k-1)}{36}$, so

$$\mathbb{E}[\max] = 1 \cdot \frac{1}{36} + 2 \cdot \frac{3}{36} + 3 \cdot \frac{5}{36} + \cdots + 6 \cdot \frac{11}{36} \approx 4.47.$$

Two useful facts: $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$ only when X, Y are **independent** (but linearity of expectation needs no independence — that is why it is so powerful). For a non-negative integer-valued variable:

$$\mathbb{E}[X] = \sum_{i \geq 1} \Pr[X \geq i]$$

(the "tail-sum" formula, handy for waiting times).

2.2.3 Linearity of Expectation

For any random variables $X_1, X_2, \ldots, X_n$ (they need **not** be independent):

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i].$$

This is the single most useful tool for analyzing randomized algorithms.

Indicator Variables

To count "how many events happened", set:

$$X_i = \mathbf{1}\{\text{event } i \text{ occurs}\} = \begin{cases} 1 & \text{if event } i \text{ happens,} \\ 0 & \text{otherwise.} \end{cases}$$

Then $\mathbb{E}[X_i] = \Pr(\text{event } i)$, and $X = \sum X_i$ gives the total count with $\mathbb{E}[X] = \sum \Pr(\text{event } i)$.

2.2.4 Example: Hiring Problem

Interview candidates in random order; hire whenever candidate is better than all previous. How many hires (expected)?

Let $X_i = 1$ if candidate i is hired (i.e. best among first i candidates). In random order this occurs with probability $\frac{1}{i}$.

$$\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} \frac{1}{i} = H_n = \Theta(\log n).$$

So the expected number of hires is the n-th harmonic number, $\ln n + O(1)$.

2.2.5 More Indicator-Variable Examples

Guessing Shuffled Cards Shuffle n cards and guess each card as it is revealed.

- **No memory:** each turn guess a uniformly random card from the full deck. Let $X_i = \mathbf{1}\{\text{guess } i \text{ correct}\}$; $\mathbb{E}[X_i] = \frac{1}{n}$ regardless of other guesses, so $\mathbb{E}[X] = \sum_i \frac{1}{n} = \mathbf{1}$.
- **With memory:** guess uniformly among the unseen cards. At turn i there are $n-i+1$ candidates, so $\mathbb{E}[X_i] = \frac{1}{(n-i+1)}$ and

$$\mathbb{E}[X] = \frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{1} = H_n = \Theta(\log n).$$

Note that the events are dependent in both games — linearity of expectation applies anyway.

Hash Table with Chaining A hash function $h : U \to \{0, \ldots, m-1\}$ maps n keys into a table of m slots; collisions are resolved by a linked list per slot. Under the **simple uniform hashing assumption** ($h(x)$ uniform and mutually independent across keys), define $I_y = \mathbf{1}\{h(y) = h(x)\}$ for each stored key $y \neq x$. Then $\mathbb{E}[I_y] = \frac{1}{m}$ and

$$\mathbb{E}[\text{keys colliding with } x] = \sum_{y \neq x} \mathbb{E}[I_y] \leq \frac{n}{m}.$$

With load factor $\alpha = \frac{n}{m}$, insert/delete/query take $O(1+\alpha)$ expected time; choosing $m = \Theta(n)$ makes this $O(1)$. (This is the idea behind C++ `unordered_map`.)

Birthday Paradox With k people and $n = 365$ equiprobable birthdays, let $X_{ij} = \mathbf{1}\{i, j \text{ share a birthday}\}$ ($\mathbb{E}[X_{ij}] = \frac{1}{n}$):

$$\mathbb{E}[X] = \binom{k}{2}\frac{1}{n} = \frac{k(k-1)}{2n}.$$

Expecting ≥ 1 collision means $k\frac{(k-1)}{(2n)} \geq 1$, i.e. $k \gtrsim \sqrt{2n} + 1$ — about **28** people for birthdays, far below the intuitive $\frac{365}{2}$. (This threshold also dictates when an m-slot hash table starts seeing collisions.)

2.2.6 Generating a Uniform Random Permutation

The hiring analysis assumes a uniformly random candidate order; the Fisher–Yates shuffle produces one in place:

Algorithm 15 Fisher–Yates Shuffle

```
1: function RandomPermute(A)
2:     for i = 1 to n do
3:         swap A[i] ↔ A[Random(1, i)]
4:     end for
5: end function
```

$O(n)$ time and $O(1)$ extra space.

Correctness: Every Permutation Has Probability $\frac{1}{n}!$

Induct on i: after iteration i, every permutation of $A[1..i]$ occurs with probability $\frac{1}{i}!$. For target $(a_1, \ldots, a_i)$, let j be the position of the original $A[i]$ in this target. The target appears iff (i) after iteration $i-1$ the other $i-1$ elements had the required order (probability $\frac{1}{(i-1)}!$ by induction), and (ii) $\text{Random}(1, i) = j$ (probability $\frac{1}{i}$). Multiplying: $\frac{1}{(i-1)}! \cdot \frac{1}{i} = \frac{1}{i}!$.

2.2.7 Geometric Distribution and Coupon Collector

Waiting for a success. If each coin flip succeeds with probability p, the number X of flips to the first head is geometric:

$$\Pr[X = j] = (1-p)^{j-1}p, \qquad \mathbb{E}[X] = \sum_{i \geq 1}\Pr[X \geq i] = \sum_{i \geq 1}(1-p)^{i-1} = \frac{1}{p}.$$

Coupon collector. Each cereal box contains one of n coupon types uniformly at random; how many boxes until all types are seen? Split the process into stages: after collecting i distinct coupons, the chance the next box is new is $\frac{(n-i)}{n}$, so stage i is

geometric with mean $\dfrac{n}{(n-i)}$:

$$\mathbb{E}[X] = \sum_{i=0}^{n-1} \frac{n}{n-i} = n \sum_{j=1}^{n} \frac{1}{j} = nH_n = \Theta(n \log n).$$

2024–25 Spring, Problem 5 (Randomization: coupon collector variants, 15 points)

Problem. Each cereal box holds one of n coupon types uniformly at random. (a) Find the expected number of boxes needed to collect $\dfrac{n}{2}$ distinct coupons, in the form $\Theta(n^{\alpha}(\log n)^{\beta})$, with both bounds. (b) After opening n boxes, how many distinct coupons do we expect to have? Give an exact formula (no asymptotics).

Solution (a). Let X_i be the number of boxes to go from i to $i+1$ distinct coupons. When i are collected, a new coupon appears with probability $\dfrac{(n-i)}{n}$, so X_i is geometric and $\mathbb{E}[X_i] = \dfrac{n}{n-i}$. Hence

$$\mathbb{E}\Big[\sum_{i=0}^{n/2-1} X_i\Big] = n\left(\frac{1}{n} + \frac{1}{n-1} + \cdots + \frac{1}{n/2+1}\right).$$

Bound the bracket:

$$\underbrace{\frac{1}{n} + \cdots + \frac{1}{n/2+1}}_{\text{each term} \,\geq\, 1/n} \;\geq\; \frac{1}{2}, \qquad \frac{1}{n} + \cdots + \frac{1}{n/2+1} \;\leq\; \frac{n/2}{n/2} = 1.$$

So the bracket is $\Theta(1)$ and the answer is $\Theta(n)$ — i.e. $\alpha = 1$, $\beta = 0$. (Compare: collecting all n coupons costs $nH_n = \Theta(n \log n)$.)

Solution (b). Let $X_i = \mathbf{1}\{\text{coupon } i \text{ is never seen}\}$. The i-th type is missed iff all n boxes contain something else, so by independence

$$\mathbb{E}[X_i] = \Pr[X_i = 1] = \left(1 - \frac{1}{n}\right)^n.$$

Linearity (no independence needed between the X_i) gives the expected number missed $= n\left(1 - \dfrac{1}{n}\right)^n$, hence

$$\boxed{\mathbb{E}[\text{distinct}] = n - n\left(1 - \frac{1}{n}\right)^n}$$

which is $\approx \left(1 - \dfrac{1}{e}\right) n \approx 0.63n$ for large n. ■

2025–26 Fall, Problem 3 (Randomization: searching with random-access queries, 10 points)

Problem. A database of n records with distinct floating-point keys supports

$$\text{RANDOM-ACCESS}(l, r):$$

return a uniformly random record whose key lies in (l, r) (null if none), at cost equal to the number of keys inside (l, r); $l = -\infty$, $r = +\infty$ are allowed. Retrieve the record

with key x (or report absence) and bound the expected total cost.
Solution (randomized binary search). Keep $l = -\infty$, $r = +\infty$. Repeat: $R \leftarrow$ RANDOM-ACCESS(l, r); if $R =$ null report "not found"; if key$(R) = x$ return R; if key$(R) < x$ set $r \leftarrow$ key(R), else set $l \leftarrow$ key(R).
Expected cost $O(n)$. Call a query **good** if the returned key lies between the 25th and 75th percentile of the current range — probability $\frac{1}{2}$, and then at least $\frac{1}{4}$ of the candidates are discarded forever. After i good calls the current range has at most $\left(\frac{3}{4}\right)^i n$ keys, and the cost of a call is at most the current range size. Let X_i be the number of calls needed to obtain the $(i+1)$-st good call; X_i is geometric with $\mathbb{E}[X_i] = 2$. Then

$$\mathbb{E}[\text{total cost}] \le \sum_{i \ge 0} \mathbb{E}[X_i] \left(\frac{3}{4}\right)^i n = 2n \sum_{i \ge 0} \left(\frac{3}{4}\right)^i \le 8n = O(n).$$

Note the query cost is not $O(1)$: a single call can inspect $\Theta(n)$ keys, so the argument must charge each call against the current range size — exactly the "geometric stages" template of Quickselect. ■

2025–26 Spring, Problem 2 (Randomization: coupon collector, 10 points)

Problem. (a) Expected boxes to collect $\frac{n}{2}$ distinct coupons, as $\Theta(n^\alpha (\log n)^\beta)$, with both bounds. (b) Expected number of distinct coupons after n boxes, exact formula.
Solution. (a) With X_i geometric, $\mathbb{E}[X_i] = \frac{n}{(n-i)}$, the total is $n\left(\frac{1}{n} + \cdots + \frac{1}{n/2+1}\right)$, and since each of the $\frac{n}{2}$ terms lies in $\left[\frac{1}{n}, \frac{2}{n}\right]$ the bracket is in $\left[\frac{1}{2}, 1\right]$: the answer is $\Theta(n)$. (b) By indicators, $\mathbb{E}[\text{missed}] = n\left(1 - \frac{1}{n}\right)^n$, so $\mathbb{E}[\text{distinct}] = n - n\left(1 - \frac{1}{n}\right)^n \approx 0.63n$.
This is the same paper question as 2024–25 Spring Problem 5; see the full derivation there. The examinable difference: part (a) is $\Theta(n)$ because the harmonic tail saturates after $\frac{n}{2}$ terms, while collecting all n coupons is $\Theta(n \log n)$ — the $\log n$ comes only from the last few rare coupons. ■

2.2.8 Case Study: The Majority Element

Find the element appearing $> \frac{n}{2}$ times, assuming it exists.

Method	Time	Space	Idea
Sort and count	$O(n \log n)$	$O(1)$	majority, if present, occupies the middle position
Divide and conquer	$O(n \log n)$	$O(\log n)$	winners of halves, equality tests only
Hashing	$O(n)$ expected	$O(n)$	frequency map
Try-and-check	$O(n)$ expected	$O(1)$	sample a random element, verify by counting; retry
Boyer–Moore vote	$O(n)$ worst-case	$O(1)$	one cancellation pass, equality tests only

Algorithm 16 Boyer–Moore Majority Vote

```
1: c ← 0
2: for i = 1 to n do
3:     if c = 0 then
4:         m ← A[i];   c ← 1
5:     else if m = A[i] then
6:         c ← c + 1
7:     else
8:         c ← c − 1                ▷ cancel A[i] against one stored copy of m
9:     end if
10: end for
11: return m
```

Why the Vote Works

Invariant: after processing $A[1..i]$, the prefix equals "c copies of m plus elements already cancelled in pairs." Each cancellation removes two different items, so any single value can lose at most $\frac{n}{2}$ copies. A true majority ($> \frac{n}{2}$ copies) can never be completely cancelled — it must survive as m. (If a majority is not guaranteed, a second $O(n)$ verification pass is required.)

3 Balanced Trees, B+ Trees and Amortized Analysis

3.1 AVL Trees, Splay Trees and Amortized Analysis

Why balance? A binary search tree supports SEARCH, INSERT and DELETE in $O(h)$ time, where h is the height. Inserting the keys $1, 2, \ldots, n$ in increasing order into an initially empty tree produces a chain of height $n-1$, so every operation degenerates to $\Theta(n)$. A balanced search tree keeps $h = O(\log n)$ at all times, so all operations become $O(\log n)$.

3.1.1 AVL Trees

Definition of an AVL Tree

An **AVL tree** is a binary search tree in which, for every node X, the heights of the left and right subtrees of X differ by at most 1. The **balance factor** of X is

$$BF(X) = h(\text{left subtree of } X) - h(\text{right subtree of } X) \in \{-1, 0, 1\},$$

where the height of an empty tree is -1 and the height of a leaf is 0.

Height Bound and the Minimum Number of Nodes

Let n_h be the minimum number of nodes in an AVL tree of height h. To minimise the node count while keeping every balance factor legal, the two subtrees of the root must have heights $h-1$ and $h-2$, hence

$$n_h = n_{h-1} + n_{h-2} + 1, \qquad n_0 = 1,\ n_1 = 2.$$

With the Fibonacci numbers $F_0 = 0$, $F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$, the solution is

$$n_h = F_{h+3} - 1.$$

Since $F_k \approx \varphi^k/\sqrt{5}$ with $\varphi = \dfrac{(1+\sqrt{5})}{2} \approx 1.618$, an AVL tree with n nodes satisfies $n \geq F_{h+3} - 1 = \Omega(\varphi^h)$, i.e.

$$h \leq 1.44 \log_2(n+2) - 0.328 = O(\log n).$$

For example $n_0 = F_3 - 1 = 2 - 1 = 1$, $n_1 = F_4 - 1 = 3 - 1 = 2$, $n_2 = F_5 - 1 = 5 - 1 = 4$, $n_3 = F_6 - 1 = 8 - 1 = 7$, and $n_6 = F_9 - 1 = 34 - 1 = 33$.

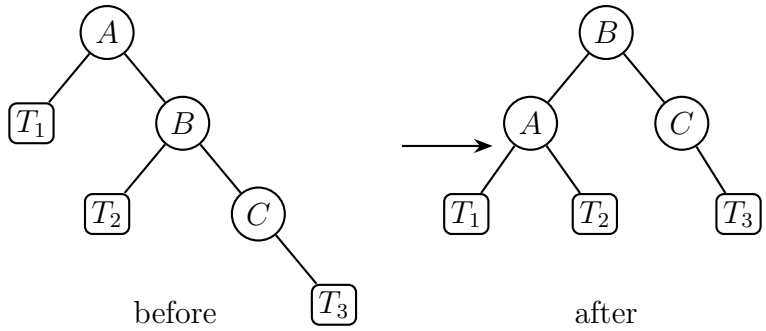
3.1.2 Insertion: the Four Rotations

Insert the new key as an ordinary BST leaf, then walk back to the root updating heights. Let A be the first (lowest) node whose balance factor becomes ± 2; only A's subtree needs repair, and one rotation (single or double) restores the AVL property for the whole tree. The four cases are named after the position of the inserted node relative to A:

Case	Structure	Repair
LL	left child of A, left side	single right rotation
RR	right child of A, right side	single left rotation
LR	left child of A, right side	left–right double rotation
RL	right child of A, left side	right–left double rotation

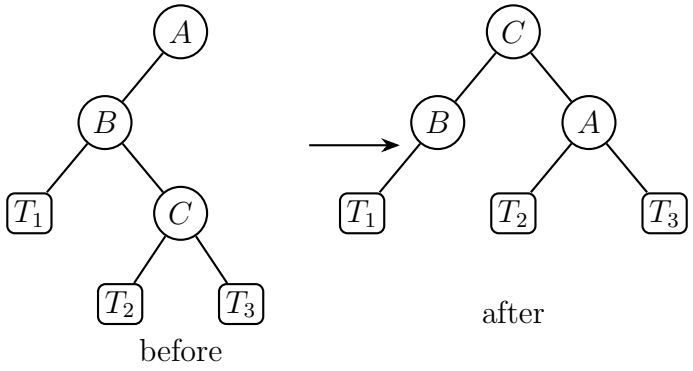
Single rotation (RR case). Write the three involved nodes as A (the unbalanced node), B (its right child) and C (the right child of B); let T_1, T_2, T_3 be the subtrees shown

below. The keys satisfy $A < T_2 < B < T_3 < C$, so promoting B to the root of this subtree preserves the BST order:



The mirror case LL is symmetric: promote the left child of A with a single right rotation. A single rotation costs $O(1)$ pointer changes.

Double rotation (LR case). When the insertion happens in the right subtree of the left child (zig-zag shape), a single rotation cannot fix the balance because the height is asymmetric in the opposite direction. Rotate the grandchild C up in two steps: first a left rotation about B, then a right rotation about A.



The RL case is the mirror image: right rotation about the right child, then left rotation about A.

How Many Rotations?

- **Insertion** needs at most **2** rotations (one single, or one double). After the rotation the subtree has exactly the same height as before the insertion, so no ancestor can become unbalanced.
- **Deletion** may need $\Theta(\log n)$ rotations, because repairing the lowest unbalanced node decreases the height of that subtree and the imbalance can propagate all the way to the root.

3.1.3 Worked Example: Building an AVL Tree

Insert $2, 1, 4, 5, 9, 3, 6, 7$ into an initially empty AVL tree.

Insert	Tree shape	Action
2	2	root
1	2, left child 1	$BF(2) = +1$
4	2 with $1, 4$	balanced
5	4 gets right child 5	RR: rotate left; root 4
9	4's right subtree $5 \to 9$	RR: rotate left; root 5
3	left child of 4	balanced
6	right child of 4	RL: double rotate; root 4
7	right child of 6	RR: rotate left; root 5

The final tree has root 5, left child 2 (with children 1 and 4, and 4 has left child 3), and right child 9 (with left child 6, whose right child is 7). Checking the statements of the classic exercise: 4 is not the root, 3 and 7 are not siblings, 2 and 6 are not siblings, and 9 is not the parent of 7; the false statement is "3 and 7 are siblings".

Exercise: Minimum Nodes for a Given Depth

Problem. If the depth of an AVL tree is 6 (the depth of an empty tree is -1), what is the minimum possible number of nodes?
Solution. Depth 6 means height 6, so $n_6 = F_9 - 1 = 34 - 1 = 33$. ■

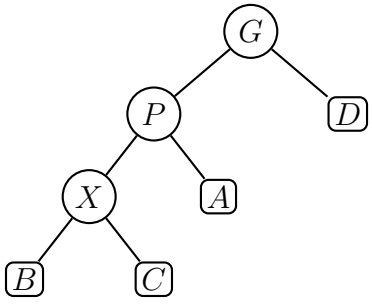
3.1.4 Splay Trees

An AVL tree pays for lookups with extra height information and rotations on every update. A **splay tree** is an ordinary BST with no balance information at all; instead, every access moves the accessed node to the root by a sequence of rotations called splaying. The tree may be badly unbalanced after a single operation, but the amortized cost of every operation is $O(\log n)$.

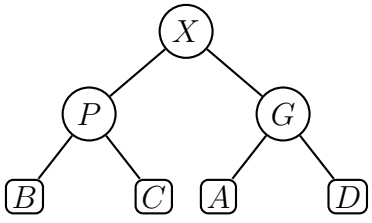
Splaying a node X. Let P be the parent of X and G the grandparent. Repeat the following until X is the root:

Case	Configuration	Operation
zig	P is the root	single rotation X–P
zig-zig	X, P both left or both right children	rotate P about G, then X about P
zig-zag	X, P on opposite sides	rotate X about P, then about G

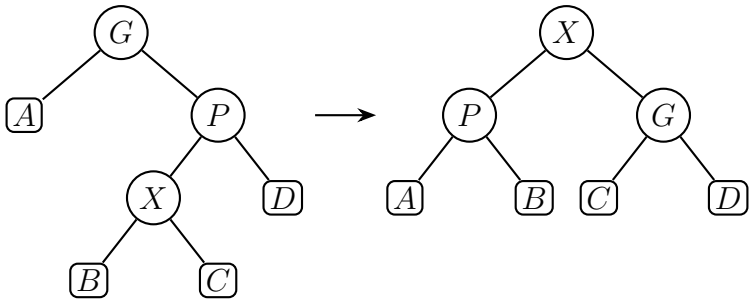
The crucial difference from a naive "rotate to the top" strategy is the order in zig-zig: the parent is rotated first, so the path is halved instead of merely shortened by one level.



zig-zig, before



zig-zig, after



zig-zag, before zig-zag, after

Why splay? After accessing X, nodes visited recently lie near the root, so a sequence of accesses to the same locality is cheap. Insertion splays the new leaf; deletion splays the parent of the removed node (or joins the two subtrees by splaying the maximum of the left subtree and attaching the right subtree to it).

Facts Worth Memorising

- Splaying an accessed node costs $O(h)$ worst case, $O(\log n)$ amortized.
- A splay tree with n nodes has $\Omega(n)$ possible shapes; a single access can cost $\Theta(n)$.
- Insert and delete are both search + one splay, so their amortized cost is $O(\log n)$.
- Repeated access to the same node is $O(1)$ amortized after the first access.

3.1.5 Amortized Analysis

Amortized analysis bounds the average cost of an operation over a worst-case sequence of operations, without averaging over inputs. Three standard methods:

Aggregate, Accounting and Potential Methods

- **Aggregate method.** Show that any sequence of n operations costs at most $T(n)$ in total; then the amortized cost per operation is $T\dfrac{(n)}{n}$.
- **Accounting (banker's) method.** Assign an amortized cost $\hat{c}_i$ to operation i. If $\hat{c}_i > c_i$ the surplus is saved as credit; if $\hat{c}_i < c_i$ the deficit is paid from previously saved credit. The invariant is that the total credit is never negative, so $\sum_{i=1}^{n} \hat{c}_i \geq \sum_{i=1}^{n} c_i$.
- **Potential method.** Define a potential function Φ on the data structure with $\Phi_{\text{start}} = 0$ and $\Phi \geq 0$, and set $\hat{c}_i = c_i + \Delta\Phi_i$. Then

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} c_i + \Phi_{\text{end}} - \Phi_{\text{start}} \geq \sum_{i=1}^{n} c_i.$$

A good potential function assumes its minimum at the start of the sequence, not its maximum; this is the single most common trap in exam questions.

Example: dynamic table (buffer doubling). A buffer holds k items. When it is full and a new item arrives, the buffer size doubles and all k old items are copied, so that insertion costs $k + 1$ instead of 1. Starting from size 1, the i-th insertion (with $i = 2^j$) costs i, all others cost 1, hence

$$T(n) = n + (1 + 2 + 4 + \cdots + 2^{\lfloor \lg n \rfloor}) < n + 2n = 3n = O(n),$$

so the aggregate amortized cost is $O(1)$ per insertion.

With the potential method, take Φ to be the negative number of free cells: $\Phi = -(\text{size} - \text{count})$. If the buffer is full with k items, an insertion costs $k+1$ and changes Φ from 0 to $-(k-1)$, giving $\hat{c} = (k+1) - (k-1) = 2 = O(1)$. If the buffer is not full, $\hat{c} = 1 + 1 + 1 = 3$. The sign matters: using $+$(free cells) would give $\hat{c} = \Theta(k)$.

3.1.6 Amortized Analysis of Splaying

Define the **rank** of a node i by $R(i) = \log S(i)$, where $S(i)$ is the number of nodes in the subtree rooted at i (including i), and let

$$\Phi(T) = \sum_{i \in T} R(i).$$

For a splay step that transforms a subtree with ranks R_1 into one with ranks R_2, the accounting cost is

$$\hat{c}_i = 1 + \sum_i \big(R_2(i) - R_1(i)\big).$$

The Access Lemma

For every splay step performed while splaying a node X,

$$\hat{c}_i \le 1 + 3\big(R_2(X) - R_1(X)\big).$$

Summing over the whole splay, the intermediate ranks of X telescope, and because X becomes the root, $R_2(X) = R(T)$ and $R_1(X)$ is the initial rank of X:

$$\hat{c} \le 3\big(R(T) - R(X)\big) + 1 = O(\log n).$$

Proof sketch. The key inequality is: if $a + b \le c$ with $a, b > 0$, then $\log a + \log b \le 2\log c - 2$.

- **zig:** $\hat{c} = 1 + R_2(X) + R_2(P) - R_1(X) - R_1(P)$. Since $R_1(P) \ge R_2(P)$, $\hat{c} \le 1 + R_2(X) - R_1(X) \le 1 + 3(R_2(X) - R_1(X))$.
- **zig-zig:** $R_1(G) = R_2(X)$, so $\hat{c} = 2 + R_2(P) + R_2(G) - R_1(X) - R_1(P) \le 2 + R_2(X) + R_2(G) - 2R_1(X)$. Applying the lemma to $S_2(G) + S_1(X) \le S_2(X)$ gives $R_2(G) + R_1(X) \le 2R_2(X) - 2$, hence $\hat{c} \le 3(R_2(X) - R_1(X))$.
- **zig-zag:** $R_2(X) = R_1(G)$ and applying the lemma to $S_2(P) + S_2(G) \le S_2(X)$ gives $\hat{c} \le 2R_2(X) - 2R_1(X)$.

Each of the three cases satisfies the claim. ■

Exercise: Splay Access Sequence

Problem. Starting from the BST below, access the keys $3, 9, 1, 5$ in this order. Which statement is false: (A) 5 is the root; (B) 1 and 9 are siblings; (C) 6 and 10 are siblings; (D) 3 is the parent of 4?

Solution. The tree is 10 (root) with children 3 and 11; 3 has children $2, 4$; 2 has child 1; 4 has children 6 (right of 4 is 6, whose children are $5, 8$, and 8 has children $7, 9$) and $12, 13$ on the right of 11. Splaying 3 makes it the root with children 2 and 10. Splaying 9 (a zig-zig path through 6 and 8) then 1 then 5 ends with 5 at the root, 1 and 9 as its children, and 6 and 10 as siblings; 3 ends up deeper than 4, so statement (D) is false. ■

Exercise: Buffer Potential Function

Problem. In the buffer-doubling problem, assume the buffer is full after all N items are placed. Which potential function gives $O(1)$ amortized cost: (A) number of

items; (B) negative number of items; (C) number of available blocks; (D) negative number of available blocks?
Solution. Only (D). With k items and buffer size $2k$ after doubling, $c_i = k + 1$ and $\Delta\Phi = -(k-1) - 0$, so $\hat{c}_i = 2$. Options (A),(C) give $\Theta(k)$; (B) gives k. ■

Exercise: Amortized-Cost Statements

Problem. Which statement about amortized analysis is false: (A) aggregate analysis gives amortized cost $T\dfrac{(n)}{n}$; (B) a good potential function assumes its maximum at the start; (C) the accounting method saves credit when $\hat{c}_i > c_i$; (D) the accounting method allows different operations to have different amortized costs?
Solution. (B) is false: the potential should assume its minimum at the start, otherwise the telescoping sum can decrease and the bound fails. ■

3.2 Red-Black Trees and B+ Trees

AVL trees keep the height balanced by storing a height and repairing every imbalance. A **red–black tree** replaces height information by one bit of colour per node and guarantees $O(\log n)$ height with a small, uniform set of repair rules. B+ trees generalise the same idea to multi-way search trees and are the standard index structure of databases and file systems.

3.2.1 Red–Black Trees

Definition

A **red–black tree** is a binary search tree in which every node is coloured red or black and:

1. every node is red or black;
2. the root is black;
3. every leaf (NIL) is black;
4. if a node is red, then both its children are black (no red–red edge);
5. for every node, all paths from that node down to a NIL leaf contain the same number of black nodes.

Property 5 defines the **black height** $bh(X)$ of a node X: the number of black nodes on any path from X (excluding X, including the NIL leaf) to a leaf.

Height Bound

A red–black tree with n internal nodes has height at most $2\lg(n+1)$.

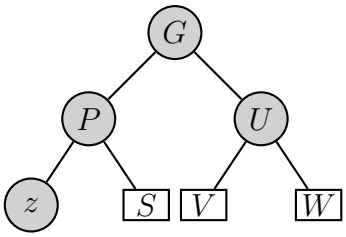
Proof. First, the subtree rooted at any node X contains at least $2^{bh(X)} - 1$ internal nodes: by induction on the height, each child of X has black height $bh(X)$ or $bh(X) - 1$, so the subtree has at least $2 \cdot (2^{bh(X)-1} - 1) + 1 = 2^{bh(X)} - 1$ nodes. Second, no path from the root to a leaf contains two consecutive red nodes, so at least half of the nodes on any root–leaf path are black; hence $bh(\text{root}) \geq \dfrac{h}{2}$. Combining, $n \geq 2^{h/2} - 1$, i.e. $h \leq 2\lg(n+1)$. ■

3.2.2 Insertion

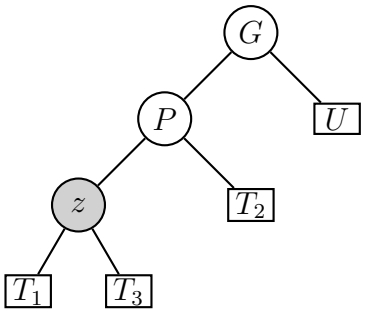
Insert the new node z as an ordinary BST leaf and colour it **red**. Only property 4 can now be violated, and only along the path from z to the root. Let y be the uncle of z (the sibling of z's parent). Three cases arise; if z reaches the root, simply recolour the root black.

Case	Condition	Repair
1	uncle y red	recolour $z.p, y$; recurse on $z.p.p$
2	y black, z zig-zag	rotate z about $z.p$; go to case 3
3	y black, z zig-zig	rotate $z.p$ about $z.p.p$; done

The mnemonic is "three reds recolour, two reds rotate": case 1 has three red nodes (z, $z.p$, y) and only needs recolouring; cases 2 and 3 have two red nodes (z, $z.p$) and need one or two rotations.



case 1: uncle U red $\Rightarrow$ recolour



case 3: zig-zig $\Rightarrow$ single rotation

Running time. Case 1 may repeat $O(\log n)$ times while climbing, but cases 2 and 3 terminate immediately. Hence an insertion performs at most **2** rotations and $O(\log n)$ recolourings.

Exercise: Inserting into a Red–Black Tree

Problem. Insert $41, 38, 31, 12, 19, 8$ successively into an initially empty red–black tree. Which statement is false: (A) 38 is the root; (B) 19 and 41 are siblings and both red; (C) 12 and 31 are siblings and both black; (D) 8 is red?
Solution. Inserting $41, 38, 31$ makes 38 the black root with red children 31 and 41. Inserting 12 (left of 31) is case 1: $31, 12$'s uncle 41 are red, so 31 and 41 become black and 38 becomes red; then 38 is the root, so it is recoloured black. Inserting 19 (right of 12) is a zig-zag repaired by a double rotation about 31, after which 19 is a black node with red children 12 and 31. Inserting 8 gives a red node under 12. The resulting tree has 19 as the left child of 38, and 41 as the right child of 38; 19 and 41 are not siblings, so (B) is false. ■

3.2.3 Deletion

Deleting a node from a red–black tree is the hardest of the standard operations. Let x denote the node actually removed.

1. x **is a leaf.** If x is red, delete it. If x is black, delete it and mark the NIL that replaces it doubly black: one black has been removed from that path and must be restored.
2. x **has one child** c**.** Replace x by c. If x was red, nothing else is needed (its child must be black, and the black heights are unchanged). If x was black and c red, recolour c black. If both were black, mark c doubly black.
3. x **has two children.** Replace x by its successor y (the minimum of its right subtree), then delete y as in case 1 or 2. (Using the predecessor, the maximum of the left subtree, gives an equally valid tree.)

The doubly black node x has an extra black weight. Let w be its sibling.

Case	Condition	Repair
1	w red	rotate w up; new w black
2	w black, both children black	recolour w red; push black up
3	w black, near red, far black	rotate near child up; go to case 4
4	w black, far child red	rotate w about $x.p$; done

The corresponding mnemonic is: "different colours swap siblings, a black family loses colour, a red near child rises, a red far child lets the parent rise". If the doubly black node reaches the root, remove one black layer and stop. A deletion performs at most **3** rotations, but case 2 may propagate $O(\log n)$ levels.

Operation	AVL tree	Red–black tree
insert rotations	≤ 2	≤ 2
delete rotations	$O(\log n)$	≤ 3
extra storage	height per node	1 bit per node
height	$\leq 1.44 \log_2 n$	$\leq 2 \log_2(n+1)$

3.2.4 B+ Trees

A B+ tree is a multi-way balanced search tree designed for disk-resident data: one node occupies one disk block, so a high branching factor reduces the number of disk accesses.

Definition (Order M)

A B+ tree of order M satisfies:
1. the root is either a leaf or has between 2 and M children;
2. every non-leaf node other than the root has between $\left\lceil \frac{M}{2} \right\rceil$ and M children;
3. all leaves have the same depth;
4. every leaf stores between $\left\lceil \frac{M}{2} \right\rceil$ and M keys.

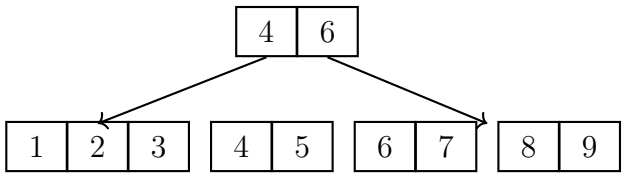
Thus a node with k children stores $k-1$ keys; a 2–3 tree is a B+ tree of order 3 and a 2–3–4 tree one of order 4. Internal keys are copies of keys that also appear in the leaves (property C of the classic multiple-choice question: leaves and non-leaf nodes share key values).

A B+ tree of order M with N keys has height $O(\log_M N)$; each search, insert or delete touches $O(\log_M N)$ nodes. Because M can be in the hundreds, the height is usually 2 or 3.

Insertion. Insert the key into the correct leaf. If the leaf now holds $M+1$ keys, split it into two leaves of $\left\lceil \frac{(M+1)}{2} \right\rceil$ and $\left\lfloor \frac{(M+1)}{2} \right\rfloor$ keys and push the smallest key of the new right leaf up into the parent. If the parent overflows, split it recursively; if the root splits, a new root is created and the tree grows by one level.

Deletion. Remove the key from its leaf. If the leaf still has at least $\left\lceil \frac{M}{2} \right\rceil$ keys, stop (and, if the deleted key was copied into an ancestor, replace that copy by the leaf's new smallest key). Otherwise:

1. **borrow** a key from an adjacent sibling that has more than $\left\lceil \frac{M}{2} \right\rceil$ keys, adjusting the separator in the parent; or
2. **merge** with an adjacent sibling, deleting the corresponding separator from the parent; if the parent then underflows, repeat recursively.



after inserting $0, 1, \ldots, 9$ into a 2–3 tree with splitting

Exercise: Building a 2–3 Tree

Problem. Insert $3, 1, 4, 5, 9, 2, 6, 8, 7, 0$ into an initially empty 2–3 tree with splitting. Which statement is false: (A) 7 and 8 are in the same node; (B) the parent of the node containing 5 has 3 children; (C) the first key in the root is 6; (D) there are 5 leaf nodes?

Solution. After inserting 6 the root splits and the root becomes the node $\langle 4, 6\rangle$. Inserting 8, then 7, splits the leaf $\langle 6, 7, 8, 9\rangle$ into $\langle 6, 7\rangle$ and $\langle 8, 9\rangle$, and the root becomes $\langle 4, 6, 8\rangle$. Inserting 0 splits the leaf $\langle 1, 2, 3\rangle$ into $\langle 0, 1\rangle$ and $\langle 2, 3\rangle$, so the root becomes $\langle 2, 4, 6, 8\rangle$ with 5 leaves. Hence 7 and 8 lie in different nodes, so (A) is false; statements (B), (C), (D) all hold. ■

Exercise: Maximum Keys in a 2–3 Tree

Problem. A 2–3 tree with 3 non-leaf nodes has at most 18 keys. True or false?

Solution. True. Three non-leaf nodes, each with at most 3 children, give at most $3 \times 3 = 9$ leaves if all are at the same depth; with 2 levels below the root the count is $3 \times 2 = 6$ leaves, each holding at most 3 keys, for at most 18 keys. ■

3.3 Inverted File Index

Motivation. A search engine must answer "which documents contain the word w?" for millions of documents without scanning them all. An **inverted file index** (inverted index) stores, for every distinct word, the list of documents in which it appears. The name comes from the fact that it inverts the natural "document → words" direction into "word → documents".

3.3.1 Definition

Inverted Index

For every term (word) t the index stores

$$t \longrightarrow \langle (d_1, p_1), (d_2, p_2), \dots \rangle,$$

where d_i is a document identifier and p_i is the position of t inside d_i. A **posting list** is the list of all entries for one term. The position information is optional; it is what makes phrase search ("data structure") possible, because two terms are adjacent only if their positions in the same document differ by 1.

The conceptual alternative is a **term–document matrix** M with $M[t][d] = 1$ if term t occurs in document d. The matrix is extremely sparse (most documents contain only a tiny fraction of all terms), so storing it densely wastes space; the inverted index stores only the 1-entries, one posting list per term. The number of documents containing a term is called its **document frequency**.

Term	Posting list	Document frequency
algorithm	$(1,4),\ (3,2),\ (7,9)$	3
tree	$(2,1),\ (7,3)$	2
index	$(3,8),\ (5,5),\ (9,1)$	3

3.3.2 Terms and Their Normalisation

Word Stemming and Stop Words

- **Word stemming** reduces a word to its stem before indexing: computing, computed, computer all map to comput. This increases recall (a query for compute finds documents containing computing) at the cost of a slightly less precise index.
- **Stop words** are extremely frequent words (the, a, of, is) that carry almost no discriminating information. They are not stored in the inverted index, which saves a large fraction of the posting lists. The trade-off is that a phrase query containing a stop word cannot be checked exactly.

3.3.3 Accessing Terms

Given a query term, the engine must locate its posting list. Two standard structures:

Structure	Lookup	Range search
balanced search tree	$O(\log n)$	$O(\log n + k)$ for k answers
hash table	$O(1)$ expected	expensive: hashing destroys order

Hashing is ideal for exact-term lookup but poor for range queries ("all terms between algo and alt") and for wildcard/prefix queries, because neighbouring terms are deliberately scattered. A search tree (or a trie) keeps the terms in sorted order and therefore supports range and prefix search.

3.3.4 Distributed Indexing

When the index does not fit on one machine, it is partitioned across nodes.

Term-Partitioned versus Document-Partitioned

- **Term-partitioned index:** each node stores the complete posting lists of a range of terms. A query is routed to the nodes owning the query terms and the results are merged. A multi-term query touches several nodes, so parallelism is limited by the slowest node, but each term's list is local.
- **Document-partitioned index:** each node stores the index of a subset of documents. Every node must process every query and return its local results, which are then merged; a single node holds complete information about its own documents.

Beware the classic true/false question: "in document-partitioned indexing each node stores all documents containing the terms in a certain range" is false — that sentence describes term-partitioning.

3.3.5 Threshold

Searching a large collection usually returns too many documents to show. A **threshold** rule ranks the retrieved documents by a relevance score and returns only the top x of them; the rest are discarded without being examined in detail. This is why the engine can react quickly even when a query matches millions of documents.

3.3.6 Evaluating a Search Engine

Performance versus Relevance

Two completely different aspects are measured:

- **Performance:** how fast the engine indexes documents, how fast it searches, and how large the index is (space). Response time and index size are the standard measures.
- **Relevance:** how good the answer set is. This is measured by precision and recall, not by running time.

The statement "it is important to measure the relevancy of the answer set when evaluating the performance of data retrieval" is therefore false: relevancy belongs to the quality of the answer, not to performance.

Precision and Recall

Let R_R be the number of relevant documents that are retrieved, I_R the number of irrelevant documents retrieved, and R_N the number of relevant documents that are not retrieved (missed). Then

$$\text{Precision } P = \frac{R_R}{R_R + I_R}, \qquad \text{Recall } R = \frac{R_R}{R_R + R_N}.$$

Precision is the fraction of the returned documents that are relevant; recall is the fraction of all relevant documents that were returned.

	relevant	**irrelevant**
retrieved	R_R	I_R
not retrieved	R_N	—

Precision	**Recall**	**Interpretation**
high	low	returned mostly relevant; many missed
low	high	relevant mostly returned; many extra
high	high	ideal

Which one matters more depends on the application. For **airport security** (explosive detection), missing a dangerous item is far worse than a false alarm, so recall is more important. For a web search where the user reads only the first page, precision matters more. The classic true/false question "Precision is more important than recall when evaluating explosive detection" is therefore false.

Exercise: Computing Recall

Problem. A database contains 28000 documents. For one query, 4000 relevant documents are retrieved, 8000 irrelevant documents are retrieved, and 8000 relevant documents are missed. What is the recall: (A) 14%; (B) 25%; (C) 33%; (D) 50%?

Solution. $R = \dfrac{R_R}{R_R + R_N} = \dfrac{4000}{4000 + 8000} = \dfrac{1}{3} \approx 33\%$. Answer (C). The total 28000 is a distractor. ■

Exercise: High Precision, Low Recall

Problem. If the precision is high but the recall is low, which description is correct: (A) most relevant documents are retrieved but too many irrelevant ones are returned; (B) most retrieved documents are relevant but many relevant documents are missed; (C) most relevant documents are retrieved but the benchmark set is too small; (D) most retrieved documents are relevant but the benchmark set is too small?

Solution. (B). High precision means $\dfrac{R_R}{(R_R + I_R)}$ is large, so few returned documents are irrelevant; low recall means $\dfrac{R_R}{(R_R + R_N)}$ is small, so many relevant documents were not returned. ■

Exercise: What Is Not Measured

Problem. Which of the following is not a concern when measuring a search engine: (A) how fast it indexes; (B) how fast it searches; (C) how friendly the interface is; (D) how relevant the answer set is?

Solution. (C). Indexing speed, search speed and relevance are the standard measures; interface friendliness is a usability issue, not a search-engine performance measure. ■

4 Priority Queues: Leftist, Skew, Binomial Heaps

4.1 Leftist Heaps and Skew Heaps

A binary heap supports insert and delete-min in $O(\log n)$, but merging two binary heaps costs $\Theta(n)$ because one of them has to be rebuilt. A **leftist heap** supports merge in $O(\log n)$ worst case by keeping the tree deliberately unbalanced to the left; a **skew heap** drops the stored length information and achieves $O(\log n)$ amortized merge with an even simpler rule.

4.1.1 Leftist Heaps

Null Path Length

For a node X, the **null path length** $Npl(X)$ is the length of the shortest path from X to a node with no child (equivalently, to a NIL pointer). By convention $Npl(\text{NIL}) = -1$ and

$$Npl(X) = \min\bigl\{Npl(\text{left}(X)),\ Npl(\text{right}(X))\bigr\} + 1.$$

A **leftist heap** is a min-heap-ordered binary tree in which, for every node X,

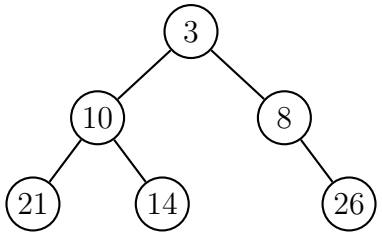
$$Npl(\text{left}(X)) \geq Npl(\text{right}(X)).$$

The **right path** of a tree is the path from the root that always follows right children. Because the left Npl dominates the right Npl at every node, the right path is the shortest path to a missing child, and it must be short:

Right-Path Theorem

If the right path of a leftist heap contains r nodes, then the heap contains at least $2^r - 1$ nodes. Consequently a leftist heap with n nodes has right path of length at most $\lfloor \log_2(n+1) \rfloor$, and every operation that touches only the right path costs $O(\log n)$.

Proof. Induct on r. For $r = 1$ the root has no right child, so $Npl(\text{right}) = -1$ and $Npl(\text{left}) \geq -1$; the tree has at least $2^1 - 1 = 1$ node. For $r > 1$, the right subtree has a right path of $r - 1$ nodes and hence, by induction, at least $2^{r-1} - 1$ nodes; the left subtree has Npl at least that of the right subtree, so it also contains at least $2^{r-1} - 1$ nodes. Adding the root, the tree has at least $2(2^{r-1} - 1) + 1 = 2^r - 1$ nodes. ∎



$$Npl(3) = Npl(10) = 1; \quad Npl(8) = Npl(21) = Npl(14) = Npl(26) = 0$$

4.1.2 Merging Two Leftist Heaps

Recursive idea. To merge H_1 and H_2, assume H_1's root has the smaller key (otherwise swap the roles). Then the root of the result is H_1's root; its left subtree stays, and its right subtree is recursively merged with H_2. Finally, if the left child has smaller Npl than the right child, swap the two children to restore the leftist property.

Algorithm 17 MERGE(H_1, H_2)

```
1: if H_1 = NIL then
2:     return H_2
3: end if
4: if H_2 = NIL then
5:     return H_1
6: end if
7: if H_1.key > H_2.key then
8:     swap H_1 and H_2
9: end if
10: H_1.right ← MERGE(H_1.right, H_2)
11: if Npl(H_1.left) < Npl(H_1.right) then
12:     swap H_1.left and H_1.right
13: end if
14: Npl(H_1) ← Npl(H_1.right) + 1
15: return H_1
```

Each recursive call moves down one node of a right path, so the recursion depth is at most the sum of the two right-path lengths, i.e. $O(\log n)$. All other work is $O(1)$ per level, hence the merge is $O(\log n)$.

Iterative version. Sort the nodes on the two right paths by key, link them into a single right path, then walk back up that path swapping children wherever the leftist property fails. The result is identical to the recursive merge; the iterative form only makes the two phases ("sort the right paths" and "fix the children") explicit.

Operation	Implementation	Time
INSERT(x)	merge the heap with a one-node heap containing x	$O(\log n)$
DELETEMIN	delete the root, then merge its two subtrees	$O(\log n)$
FINDMIN	read the root	$O(1)$
BUILDHEAP	merge pairs of one-node heaps, level by level	$O(n)$

BuildHeap for leftist heaps. Put all n one-node heaps in a queue; repeatedly dequeue two heaps, merge them and enqueue the result. After round k there are $\lceil n/2^k \rceil$ heaps, each containing 2^k nodes, and each merge costs $O(k)$. For $n = 2^K$ the total is

$$T(n) = \sum_{k=1}^{K} \frac{n}{2^k} \cdot O(k-1) = O(n),$$

because $\sum_{k \geq 1} k/2^k = 2$. So bottom-up construction is linear, like the binary-heap BUILDHEAP.

Exercise: Merging Leftist Heaps

Problem. Merge the two leftist heaps of the figure. Which statement is false: (A) 2 is the root and 11 is its right child; (B) 9 and 12 have the same depth; (C) 21 is the deepest node and its parent is 11; (D) $Npl(4) < Npl(2)$?
Solution. (D) is false. The merge links the right paths in increasing key order and swaps children on the way back; in the resulting tree $Npl(2) = Npl(4) = 2$, so the two null path lengths are equal. ■

4.1.3 Skew Heaps

A skew heap is a self-adjusting version of a leftist heap: it stores no Npl values and makes no attempt to maintain the leftist property. The merge is the same as above except that

every node on the merged right path swaps its children, and only the node with the largest key (the last node of the path) keeps its children in place.

Algorithm 18 SkewMerge(H_1, H_2)

```
1: if H1 = nil then
2:     return H2
3: end if
4: if H2 = nil then
5:     return H1
6: end if
7: if H1.key > H2.key then
8:     swap H1 and H2
9: end if
10: H1.right ← SkewMerge(H1.right, H2)
11: swap H1.left and H1.right          ▷ unconditional, except for the maximum node
12: return H1
```

Because the swap is unconditional, the right path can become arbitrarily long: a heap built by repeated insertion of increasing keys degenerates into a left chain whose right path is empty, and a subsequent merge can expose a long path. Nevertheless the amortized cost of a merge is $O(\log n)$.

Amortized Analysis of Skew Heaps

Call a node p **heavy** if its right subtree contains at least half of p's descendants (including p), and **light** otherwise. Any path from the root to a leaf contains at most $\log_2 n$ light nodes, because a light node's subtree is less than half the size of its parent's subtree. A merge traverses a right path; every heavy node on that path is swapped and becomes a left child, so it stops being heavy until it is visited again by another merge. Using the potential

$$\Phi = \text{number of heavy nodes},$$

the amortized number of heavy nodes visited by a merge is $O(\log n)$, and the light ones are at most $\log_2 n$ by definition. Hence

$$T_{\text{amortized}} = O(\log n).$$

Skew Heap Facts

- Insert = merge with a one-node heap; DeleteMin = merge the two subtrees of the root.
- Inserting the keys $1, 2, \ldots, 2^{k-1}$ in this order into an empty skew heap produces a complete binary tree.
- Inserting $1, 2, \ldots, 15$ produces a complete tree with 8 leaves, not 6; 11 is the right child of 7 and 6 is the left child of 2.

Exercise: Inserting into a Skew Heap

Problem. Insert the keys $1, 2, \ldots, 15$ in this order into an initially empty skew heap. Which statement is false: (A) the result is a complete binary tree; (B) there are 6 leaf nodes; (C) 6 is the left child of 2; (D) 11 is the right child of 7?
Solution. (B) is false. The resulting tree is complete with 8 leaves. Statements (A), (C) and (D) are all true of that tree. ■

4.2 Binomial Queues

A binomial queue (also called a binomial heap) is a forest of heap-ordered binomial trees. It is the priority-queue structure with the best amortized behaviour: merge is $O(\log n)$ worst case and insert is $O(1)$ amortized, which no single binary heap can achieve.

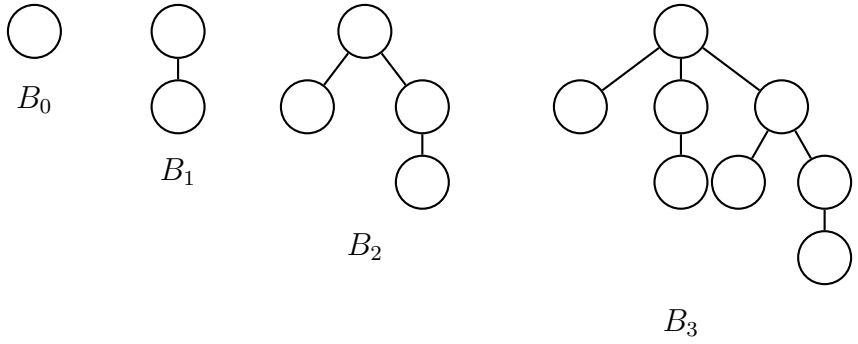
4.2.1 Binomial Trees

Definition and Basic Properties

A **binomial tree** B_k of order k is defined recursively:
- B_0 is a single node;
- B_k is formed by linking one B_{k-1} to the root of another B_{k-1}, making it the leftmost child.

Equivalently, B_k consists of a root whose children are $B_0, B_1, \ldots, B_{k-1}$. Therefore:

Property	**Value**
number of nodes	2^k
height	k
degree of the root	k
number of nodes at depth d	$\binom{k}{d}$



Binomial Queue

A **binomial queue** of n elements is a collection of binomial trees $B_{k_1}, B_{k_2}, \ldots$ with distinct orders $k_1 < k_2 < \cdots$, containing together n elements, each tree heap-ordered. Because the orders are distinct and the size of B_k is 2^k, the set of orders present is exactly the set of 1-bits in the binary representation of n; there is at most one tree of each order.

The forest is stored in an array indexed by order, each tree represented as left-child/next-sibling. Finding a free slot, merging two trees and carrying to the next order are exactly the operations of binary addition, which is the source of almost every complexity result below.

4.2.2 Operations

Operation	Idea	Time
FindMin	scan all roots	$O(\log n)$; $O(1)$ with a pointer
Merge	binary addition	$O(\log n)$
Insert	merge with one node	$O(\log n)$, $O(1)$ amortized
DeleteMin	remove root, merge its children	$O(\log n)$
BuildQueue	n insertions	$O(n)$ total

Merge. Walk the orders from 0 upward carrying a tree exactly as one adds binary numbers. At order k there are up to three trees (one from H_1, one from H_2, one carried). If there are 0 or 1, keep it; if there are 2, link the one with the larger root to the smaller and carry the result to order $k+1$; there can never be 3, because the carry is consumed before the next order. Two B_k's are linked in $O(1)$ by making the larger root a child of the smaller, so the whole merge is $O(\log n)$.

DeleteMin. Find (and remove) the tree B_k whose root is the global minimum. The remaining trees form a binomial queue H' of size $n - 2^k$. The children of the deleted root are $B_0, B_1, \ldots, B_{k-1}$, which together form a binomial queue H'' of size $2^k - 1$. Merge H' and H''; the cost is $O(\log n)$.

Insert. Merge a one-node B_0 into the queue. Starting from order 0, the new tree may collide with an existing tree, then the carried tree collides again, exactly as adding 1 to a binary counter.

4.2.3 Amortized Analysis

N insertions create a queue of N elements, in total $O(N)$ time, so insert costs $O(1)$ amortized. Two proofs:

Aggregate. Think of n as a binary counter. Inserting an element flips the lowest 0-bit to 1 and flips all lower 1-bits to 0; the number of flips equals the number of trailing ones plus one. Over n insertions there are $\sum_{i=1}^{n} O(\text{trailing ones of } i) = O(n)$ flips in total, because each bit flips at most $\lceil n/2^{j+1} \rceil$ times for a total of less than $2n$.

Potential. Let Φ be the number of trees in the forest (equivalently, the number of 1-bits of the size). An insertion that links t trees has actual cost $t+1$ and decreases the tree count by $t-1$ (it adds one tree, then each link removes one), so

$$\hat{c} = (t+1) + \Delta\Phi = (t+1) - (t-1) = 2 = O(1).$$

Exercise: Representing 42 Elements

Problem. Which set of binomial trees can represent a binomial queue of size 42: (A) $B_0B_1B_2B_3B_4B_5$; (B) $B_1B_3B_5$; (C) B_1B_5; (D) B_2B_4?
Solution. $42 = 101010_2$, so the queue must contain B_1, B_3, B_5 — answer (B). Indeed $2 + 8 + 32 = 42$. ■

Exercise: Constant-Time Operation

Problem. For a binomial queue, which operation takes constant time on average: (A) merging; (B) find-max; (C) delete-min; (D) insertion?
Solution. (D). Insert is $O(1)$ amortized (and hence $O(1)$ on average over a sequence of insertions). Merge and delete-min are $O(\log n)$; find-min is $O(\log n)$ without a stored pointer. ■

Exercise: Merging Two Queues

Problem. Merge the two binomial queues of the figure. Which statement must be false: (A) the result contains B_2 and B_4; (B) 13 and 15 are children of 4; (C) if 23 is a child of 2 then 12 is another child of 2; (D) if 4 is a child of 2 then 23 is another child of 2?

Solution. In a binomial tree the children of a root are subtrees of every smaller order, so a B_3-root has children that are roots of B_0, B_1, B_2. Statement (D) demands two particular children of the same order, which cannot both occur: a root has at most one child of each order. Hence (D) must be false. ■

5 Sorting and External Sorting

5.1 Quicksort

5.1.1 Deterministic Quicksort (Lomuto Partition)

Pick $A[r]$ as pivot; partition into $<$ pivot (left), $=$ pivot (middle), $>$ pivot (right).

Algorithm 19 Quicksort

```
1: function QUICKSORT(A, p, r)
2:     if p < r then
3:         q ← PARTITION(A, p, r)
4:         QUICKSORT(A, p, q − 1)
5:         QUICKSORT(A, q + 1, r)
6:     end if
7: end function
8: function PARTITION(A, p, r)
9:     pivot ← A[r]
10:    i ← p − 1
11:    for j = p to r − 1 do
12:        if A[j] ≤ pivot then
13:            i ← i + 1
14:            swap A[i] ↔ A[j]
15:        end if
16:    end for
17:    swap A[i + 1] ↔ A[r]
18:    return i + 1
19: end function
```

Partition Invariants

At the end of each j-iteration in Partition:

- $A[p \dots i] \le pivot$,
- $A[i+1 \dots j] > pivot$,
- $A[j+1 \dots r-1]$ are unprocessed,
- $A[r] = pivot$.

Maintain with swaps; runtime is $\Theta(r - p + 1)$ on subarray size n.

Worked example: Partition $[2, 8, 7, 1, 3, 5, 6, 4]$ with pivot $A[r] = 4$ (Lomuto; bold marks $A[i]$ boundary):

j	$A[j] \le 4$?	i **after step**	**array**
start	—	$p - 1$	$[2, 8, 7, 1, 3, 5, 6, \mathbf{4}]$
p ($A[j] = 2$)	yes	p	$[\mathbf{2}, 8, 7, 1, 3, 5, 6, 4]$
$p + 1$ (8)	no	p	$[2, 8, 7, 1, 3, 5, 6, 4]$
$p + 2$ (7)	no	p	$[2, 8, 7, 1, 3, 5, 6, 4]$
$p + 3$ (1)	yes	$p + 1$	$[2, \mathbf{1}, 7, 8, 3, 5, 6, 4]$
$p + 4$ (3)	yes	$p + 2$	$[2, 1, \mathbf{3}, 8, 7, 5, 6, 4]$
$p + 5, p + 6$ (5, 6)	no	$p + 2$	unchanged
final swap	—	$q = p + 3$	$[2, 1, 3, \mathbf{4}, 7, 5, 6, 8]$

Pivot lands at its final sorted position; quicksort then recurses on $[2, 1, 3]$ and $[7, 5, 6, 8]$ independently.

5.1.2 Quicksort Analysis

Deterministic Quicksort Worst-Case

If the input is **already sorted** and pivot is always the **last** (or first) element:

$$T(n) = T(n-1) + \Theta(n) \quad \Longrightarrow \quad T(n) = \Theta(n^2).$$

Same as insertion sort! This is why we randomize.

Best case: perfectly balanced splits give $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n)$.

"Balanced Enough" Splits: $\frac{1}{4}$–$\frac{3}{4}$

Suppose every split were a **worst balanced case**, sizes $\frac{n}{4}$ and $\frac{3n}{4}$:

$$T(n) = T(n/4) + T(3n/4) + cn.$$

The recursion tree has a short left spine (depth $\log_4 n$) and a long right spine (depth $\log_{4/3} n$); every level does at most cn work, so even here

$$T(n) \le cn \cdot \log_{4/3} n = O(n \log n).$$

With a random pivot, Pr[pivot rank lies in middle half $[\frac{n}{4}, \frac{3n}{4}]] = \frac{1}{2}$ — so on average every other split is good, which is already enough for $\Theta(n \log n)$ expected depth.

Randomized Quicksort Expected Time

Pick pivot **uniformly at random** from $A[p..r]$ (swap it to position r before Partition). Let X = total number of **comparisons** made. Use indicator variables $X_{ij} = 1$ if elements of ranks i and j $(i < j)$ are ever compared.
Key insight: ranks i and j are compared iff **the first pivot chosen from within $\{i, i+1, \ldots, j\}$ is either i or j**. That happens with probability $\frac{2}{j-i+1}$.

$$\mathbb{E}[X] = \sum_{1 \le i < j \le n} \frac{2}{j-i+1} = \sum_{k=2}^{n} \frac{2}{k} \cdot (n-k+1) = 2(n+1)H_n - 4n.$$

So $\mathbb{E}[X] = \Theta(n \log n)$, and therefore expected runtime is $\Theta(n \log n)$.

Alternative View: Pivots Form a Random BST

Imagine inserting elements into a BST in the order in which they become pivots (the first pivot is the root, left-subarray pivots are its left subtree, ...). Two elements are compared exactly when one is an ancestor of the other — the same indicator argument, reinterpreted. The expected depth of the node ranked i is

$$H_i + H_{n-i+1} - 2 \le 2H_n = O(\log n),$$

so the expected total comparisons are $\sum_i O(\log n) = O(n \log n)$. Equivalently, arrange the pair-comparison probabilities in an upper-triangular matrix: every row is a truncated harmonic series of $O(\log n)$ "large" terms $(\frac{2}{k})$, and all n rows together sum to $O(n \log n)$.

Quicksort Properties

- **Stable:** no (Lomuto and Hoare partitions swap non-adjacent elements).
- **In-place:** yes (only $O(\log n)$ recursion stack; no auxiliary array).
- **Practical speed:** usually faster than Mergesort or Heapsort despite the same $\Theta(n \log n)$ bound, thanks to better cache locality and low constants.

Engineering Quicksort (What Real Libraries Do)

- **Insertion-sort cutoff:** for subarrays of size $\lesssim 16$, switch to insertion sort, which beats recursive quicksort on tiny nearly-sorted ranges.
- **Median-of-three pivot:** choose the median of first/middle/last elements — cheaply defeats already-sorted and reverse inputs.
- **Introsort:** switch to Heapsort after recursion depth exceeds $2\log_2 n$, retaining Quicksort's average speed with a guaranteed $O(n \log n)$ worst case (used by C++ `std::sort`).
- **3-way partition (Dutch national flag):** group equal keys together; without it, arrays with many duplicate keys degenerate toward $\Theta(n^2)$.

5.1.3 Randomized Selection (Quickselect)

Find the k-th smallest element of an unsorted array (median is the $k = \left\lceil \frac{n}{2} \right\rceil$ case). Reuse Partition: after partitioning on a random pivot, the pivot is at its final rank $k = m - l + 1$ inside the current segment; only **one** recursive call is needed.

Algorithm 20 RandomizedSelect(A, l, r, k) — return the k-th smallest of $A[l..r]$

```
1: if l = r then
2:     return A[l]
3: end if
4: pick pivot uniformly at random, swap it to r
5: m ← Partition(A, l, r)
6: pos ← m − l + 1                                  ▷ rank of pivot within A[l..r]
7: if k = pos then
8:     return A[m]
9: else if k < pos then
10:    return RandomizedSelect(A, l, m − 1, k)
11: else
12:    return RandomizedSelect(A, m + 1, r, k − pos)        ▷ relative rank
13: end if
```

Worked example: find the 4-th smallest of $[7, 2, 9, 1, 4, 8, 3]$ (sorted: $1, 2, 3, \mathbf{4}, 7, 8, 9$).

Segment	k	pivot	result
$[7, 2, 9, 1, 4, 8, 3]$	4	3	partition $\to [2, 1, 3 \mid 7, 4, 8, 9]$, pivot rank $3 < 4$, go right
$[7, 4, 8, 9]$	1	9	rank $4 > 1$, go left
$[7, 4, 8]$	1	8	rank $3 > 1$, go left
$[7, 4]$	1	4	partition $\to [4 \mid 7]$, rank $1 = k$, **return** 4

Expected Time $\Theta(n)$

Call a pivot **good** when its rank lies in the middle half $[\frac{n}{4}, \frac{3n}{4}]$; this happens with probability $\frac{1}{2}$. After a good pivot the surviving subproblem has size $\leq \frac{3n}{4}$; bad pivots only add linear partitioning cost. Stages between good pivots are geometric with expected count 2, so the expected total work is

$$2cn + 2c\frac{3}{4}n + 2c\left(\frac{3}{4}\right)^2 n + \cdots = 2cn\sum_{i\geq 0}\left(\frac{3}{4}\right)^i = 8cn = \Theta(n).$$

Worst case remains $\Theta(n^2)$ (probability tiny); the deterministic **median-of-medians** algorithm achieves worst-case $O(n)$ by splitting into groups of 5, whose pivot is guaranteed to have at least $\frac{3n}{10}$ elements on each side: $T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n) = O(n)$.

Bonus: k-th of Two Sorted Arrays in $\Theta(\log k)$

Given sorted arrays A, B and rank k, compare $A[\left\lfloor\frac{k}{2}\right\rfloor]$ and $B[\left\lfloor\frac{k}{2}\right\rfloor]$ and discard the smaller half (none of its elements can be the k-th), then recurse with rank $k - \left\lfloor\frac{k}{2}\right\rfloor$. Every step halves k; binary-search variants achieve $O(\log\min(m, n))$.

5.1.4 Hoare's Partition (Alternative)

Two pointers moving toward each other from opposite ends; swap inversions. Does slightly fewer swaps than Lomuto. Pivot is not guaranteed to end at index q; recur on $A[p..q]$ and $A[q+1..r]$, **skipping** q.

5.2 Heapsort

5.2.1 Priority Queue Motivation

A **priority queue** stores n elements, each with a priority (key), and supports `Insert` and `Extract-Max` (min version: negate priorities). Concrete uses: OS process scheduling (the `top` table — new/preempted processes are inserted; when the CPU is free, the lowest-PR process is extracted), event-driven simulation, Dijkstra's shortest paths, and streaming top-k.

Implementation	**Insert**	**Extract-Max**	**Notes**
Unsorted array/list	$O(1)$	$O(n)$	extract must scan everything
Sorted array/list	$O(n)$	$O(1)$	insert requires a shift
Balanced BST	$O(\log n)$	$O(\log n)$	correct bound but large pointer/balance overhead
Binary heap	$O(\log n)$	$O(\log n)$	compact array, no pointers, small constants

5.2.2 Heap Data Structure

A (binary) **max-heap** is a complete binary tree satisfying:

$$A[\text{Parent}(i)] \geq A[i] \quad \forall i.$$

(Min-heap: $\leq$ instead.)

Array Representation of a Heap

Given 1-indexed array $A[1..n]$:

$$\begin{aligned} \text{Parent}(i) &= \lfloor i/2 \rfloor, \\ \text{Left}(i) &= 2i, \\ \text{Right}(i) &= 2i+1. \end{aligned}$$

A complete binary tree on n nodes has height $\lfloor \lg n \rfloor$.

Height derivation. A complete tree of height h has all levels $0, \ldots, h-1$ full ($2^h - 1$ nodes) and at least one node on level h; hence

$$2^0 + 2^1 + \cdots + 2^{h-1} = 2^h - 1 < n \quad \Longrightarrow \quad n \leq 2^0 + 2^1 + \cdots + 2^h = 2^{h+1} - 1,$$

so $2^h \leq n < 2^{h+1}$ and $h = \lfloor \lg n \rfloor$. This $\Theta(\log n)$ height is why every heap operation is fast.

2025–26 Fall, Problem 2(a) (Min-heap insertions, 3 points)

Problem. Insert $4, 3, 8, 2, 5, 6, 7, 9$ in this order into an initially empty **min**-heap, drawing the tree after each insertion.
Solution. Appending then bubbling up (array form shown; identical to the tree):

Insert	Array	Action	Bubble-up path
4	$[4]$	root	—
3	$[3, 4]$	swap with parent 4	3 beats 4
8	$[3, 4, 8]$	parent $4 < 8$	none
2	$[2, 3, 8, 4]$	swap with 4, then with 3	$2 < 4$, $2 < 3$
5	$[2, 3, 8, 4, 5]$	parent $3 < 5$	none
6	$[2, 3, 6, 4, 5, 8]$	swap with 8	$6 < 8$, then $3 < 6$
7	$[2, 3, 6, 4, 5, 8, 7]$	parent $6 < 7$	none
9	$[2, 3, 6, 4, 5, 8, 7, 9]$	parent $4 < 9$	none

Each insertion is $O(\log n)$ (one root-to-leaf path). Final tree: 2 over $\{3, 6\}$, 3 over $\{4, 5\}$, 6 over $\{8, 7\}$, 4 over $\{9\}$. ■

2025–26 Spring, Problem 4(a) (Min-heap insertions, 8 points)

Problem. Insert $5, 8, 1, 4, 7, 3, 2, 6$ in this order into an initially empty min-heap, drawing the tree after each insertion.
Solution.

Insert	Array	Action	Bubble-up path
5	$[5]$	root	—
8	$[5, 8]$	parent $5 < 8$	none
1	$[1, 8, 5]$	swap with 5	$1 < 5$
4	$[1, 4, 5, 8]$	swap with 8	$4 < 8$, then $1 < 4$
7	$[1, 4, 5, 8, 7]$	parent $4 < 7$	none
3	$[1, 4, 3, 8, 7, 5]$	swap with 5	$3 < 5$, then $1 < 3$
2	$[1, 4, 2, 8, 7, 5, 3]$	swap with 3	$2 < 3$, then $1 < 2$
6	$[1, 4, 2, 6, 7, 5, 3, 8]$	swap with 8	$6 < 8$, then $4 < 6$

Final tree: 1 over $\{4, 2\}$; 4 over $\{6, 7\}$; 2 over $\{5, 3\}$; 6 over $\{8\}$. ■

5.2.3 Max-Heapify (Bubble-Down)

Assume children of node i are heaps, but $A[i]$ may be too small. Fix by swapping $A[i]$ with the **larger** child, recursing down.

Algorithm 21 Max-Heapify

```
1: function MaxHeapify(A, i, heap_size)
2:     L ← 2i;    R ← 2i + 1
3:     largest ← i
4:     if L ≤ heap_size and A[L] > A[largest] then
5:         largest ← L
6:     end if
7:     if R ≤ heap_size and A[R] > A[largest] then
8:         largest ← R
9:     end if
10:    if largest ≠ i then
11:        swap A[i] ↔ A[largest]
12:        MaxHeapify(A, largest, heap_size)
13:    end if
14: end function
```

Time: $O(\log n)$ per call (proportional to tree height).

5.2.4 Build Max-Heap

Naive approach: run Max-Heapify top-down from the **last non-leaf** ($\left\lfloor \frac{n}{2} \right\rfloor$) *down to* root 1. Leaves are trivially heaps of size 1.

Algorithm 22 Build Max-Heap

```
1: function BuildMaxHeap(A, n)
2:     for i = ⌊n/2⌋ downto 1 do
3:         MaxHeapify(A, i, n)
4:     end for
5: end function
```

Build-Max-Heap Cost is $O(n)$, Not $O(n \log n)$

Height-h nodes are $\leq n/2^{h+1}$; each MaxHeapify on a height-h node costs $O(h)$:

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \frac{n}{2^{h+1}} \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = O(2n) = O(n).$$

(Building from the leaves up is much cheaper than building from the root down.)

Per-level bookkeeping (take $n = 2^{h+1} - 1$, a perfectly full tree):

Level (root = 0)	**# nodes**	**heapify cost per node**	**total for level**
0	1	h	h
1	2	$h-1$	$2(h-1)$
2	4	$h-2$	$4(h-2)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	2^i	$h-i$	$2^i(h-i)$
h	2^h	0	0

Summing with $j = h - i$:

$$\sum_{i=0}^{h-1} 2^i(h-i) = 2^h \sum_{j=1}^{h} \frac{j}{2^j} = 2^h \left(2 - \frac{h+2}{2^h} \right) = 2^{h+1} - h - 2 < n,$$

using $\sum_{j=1}^{h} j/2^j = 2 - (h+2)/2^h$. Nearly all the work is at the bottom levels; the root level costs only h.

2025–26 Fall, Problem 2(b) (Merging equal-sized heaps, 3 points)

Problem. Given 2^{m-k} min-heaps $H_1, \ldots, H_{2^{m-k}}$, each with $2^k - 1$ elements and stored with parent/child pointers, plus 2^{m-k-1} new numbers in nodes $v_1, \ldots, v_{2^{m-k-1}}$, merge everything into 2^{m-k-1} min-heaps of size $2^{k+1} - 1$ in $O(k\, 2^{m-k-1})$ time.
Solution. For each i, make v_i the parent of the roots of H_{2i-1} and H_{2i}, which yields

a binary tree of size $1 + 2(2^k - 1) = 2^{k+1} - 1$.

1. Compare $x = \text{value}(v_i)$ with the two children roots; if x is no larger, the subtree is already a min-heap.
2. Otherwise swap x with the smaller of the two child roots and continue downward exactly as MinHeapify: at each level compare with the two children and swap into the smaller one.
3. Stop when x is $\leq$ both children or x has reached a leaf.

Correctness. The only violation of the heap order is at the node holding x; all other subtrees are untouched valid heaps, so one bubble-down fixes everything (the standard Heapify precondition).

Running time. Each level costs $O(1)$ and there are $O(k)$ levels, so $O(k)$ per merge; over 2^{m-k-1} merges, $O(k\,2^{m-k-1})$. ■

2025–26 Fall, Problem 2(c) (Linear-time build-heap from scratch, 4 points)

Problem. Let $n = 2^m - 1$, $m \geq 2$. Using (b), build a min-heap of n numbers in $O(n)$ time, proving the bound from scratch.

Solution. Start with the first 2^{m-1} numbers as 2^{m-1} heaps of size $2^1 - 1 = 1$. For $k = 1, 2, \ldots, m-1$: suppose 2^{m-k} heaps of size $2^k - 1$ hold the first $2^{m-1} + 2^{m-2} + \cdots + 2^{m-k}$ numbers; take the next 2^{m-k-1} numbers as the nodes $v_1, \ldots$ and apply (b) to obtain 2^{m-k-1} heaps of size $2^{k+1} - 1$. After the last iteration ($k = m - 1$) one heap holding all n numbers remains.

Time. The total is

$$\sum_{k=1}^{m-1} k\,2^{m-k-1} = 2^m \sum_{k=1}^{m-1} \frac{k}{2^{k+1}} \;\leq\; 2^m = n + 1 = O(n),$$

because for $|x| < 1$, $S(x) = \sum_{k\geq 1} x^k = \dfrac{x}{1-x}$ gives $S'(x) = \sum_{k\geq 1} k x^{k-1} = \dfrac{1}{(1-x)^2}$, so $\sum_{k\geq 1} \dfrac{k}{2^{k+1}} = \dfrac{1}{4} S'\left(\dfrac{1}{2}\right) = 1$. Hence $O(n)$ — the same $O(n)$ as the array-based Build-Max-Heap, but here the tree is pointer-based and we never need random access. ■

5.2.5 Heapsort Algorithm

(1) Build max-heap on $A[1..n]$. (2) Repeatedly: swap max (root $A[1]$) with last element, shrink heap, MaxHeapify root.

Algorithm 23 Heapsort

```
1: function HEAPSORT(A, n)
2:     BUILDMAXHEAP(A, n)
3:     for i = n downto 2 do
4:         swap A[1] ↔ A[i]
5:         heap_size ← i − 1
6:         MAXHEAPIFY(A, 1, heap_size)
7:     end for
8: end function
```

Heapsort Analysis

- Build: $O(n)$.

- $n-1$ swaps each followed by MaxHeapify: $(n-1)\,O(\log n) = O(n \log n)$.
- **Total:** $O(n \log n)$ (best = worst = average, unlike Quicksort).
- **Stable?** No.
- **In-place?** Yes (only $O(1)$ extra storage, no recursion stack if MaxHeapify is written iteratively).

5.2.6 Priority Queue Operations (Applications of Heaps)

Max-priority queue supports: `Maximum`, `ExtractMax`, `IncreaseKey`, `Insert`.

Algorithm 24 Priority Queue Operations

```
1: function ExtractMax(A, heap_size)
2:     max ← A[1]
3:     A[1] ← A[heap_size];    heap_size ← heap_size − 1
4:     MaxHeapify(A, 1, heap_size)
5:     return max
6: end function
7: function IncreaseKey(A, i, key)
8:     if key < A[i] then error
9:     end if
10:    A[i] ← key
11:    while i > 1 and A[Parent(i)] < A[i] do                ▷ bubble up
12:        swap A[i] ↔ A[Parent(i)]
13:        i ← Parent(i)
14:    end while
15: end function
16: function Insert(A, key, heap_size)
17:    heap_size ← heap_size + 1
18:    A[heap_size] ← −∞
19:    IncreaseKey(A, heap_size, key)
20: end function
```

Each operation runs in $O(\log n)$ time.

Extract-Max trace on $[42, 29, 18, 14, 7, 6]$:

initial	save $m = 42$, move 6 to root	$[\mathbf{6}, 29, 18, 14, 7]$
bubble down	$6 < 29$, swap with larger child	$[\mathbf{29}, 6, 18, 14, 7]$
bubble down	$6 < \max(14, 7) = 14$, swap with 14	$[29, \mathbf{14}, 18, 6, 7]$

Returned 42; heap is now $[29, 14, 18, 6, 7]$.

Insert trace (bubble up): insert 20 into $[29, 14, 18, 6, 7]$:

append	$[29, 14, 18, 6, 7, \mathbf{20}]$	parent of 20 is 18
swap up	$20 > 18$	$[29, 14, \mathbf{20}, 6, 7, 18]$
stop	parent is $29 > 20$	done

2024–25 Spring, Problem 4 (Heaps, 10 points)

Problem. Maintain m sets $S_1, \ldots, S_m$ supporting

- FindMin: return $\min(S_1 \cup \cdots \cup S_m)$, or null if empty;
- Add(x, i): insert x into S_i.

Each $n_i = |S_i| \le N$. Achieve FindMin in $O(1)$ and Add in $O(\log m + \log n_i)$.

Solution (heap of heap-minima + position map). Keep one min-heap H (as array $B[1..m]$) holding the pair (a_i, i) where $a_i = \min S_i$, ordered by a_i. Store each set as a pair (A_i, p_i): A_i is an array of length N holding the elements of S_i as a heap, and $p_i \in [1, m]$ is the position in H where the pair (a_i, i) currently sits.

- FindMin: if H is empty return null, else return the key stored at $B[1]$ — $O(1)$.
- Add(x, i): let $y = A_i[1]$ (the current minimum of S_i). Insert x into the heap A_i in $O(\log n_i)$. If $x < y$ the minimum changed: overwrite $B[p_i] \leftarrow (x, i)$ and **bubble up** while x is smaller than its parent. During each swap, if a pair (a_j, j) moves to position $B[k]$, update $p_j \leftarrow k$. There are $O(\log m)$ levels.

Total: $O(\log n_i) + O(\log m)$ per Add. The subtlety the marker looks for: the position map must be repaired on every swap in H, otherwise Add would have to search for (a_i, i) and the bound would break. ■

2025–26 Spring, Problem 4(b) (Largest $\lceil \log_2 n \rceil$ elements in $O(n \log \log n)$, 12 points)

Problem. $A[1..n]$ is unsorted and distinct. Return its largest $\lceil \log_2 n \rceil$ elements in $O(n \log \log n)$ time, using any algorithm taught in class.

Solution (block sort + heap of block maxima). Let $L = \log_2 n$.

1. Split A into $\dfrac{n}{L}$ blocks of L consecutive elements and sort each block increasing in $O(L \log L) = O(\log n \log \log n)$; total
$$\frac{n}{L} \cdot O(L \log L) = O(n \log \log n).$$
2. Build a max-heap H containing the last (largest) element of each block, together with its block index; each block keeps a pointer to its current position. Heap construction costs $O\left(\dfrac{n}{L}\right)$ insertions of $O(\log \left(\dfrac{n}{L}\right)) = O(\log n) \Rightarrow O(n)$.
3. Repeat L times: extract the maximum v from H (it is the next largest overall), output v, move that block's pointer one step left and insert the new block maximum. Each step is $O(\log \left(\dfrac{n}{L}\right)) = O(\log n)$, so this stage costs $O(L \log n) = O(\log^2 n)$.

The first stage dominates: $O(n \log \log n)$ total. **Why the pointers matter:** the heap must return the largest remaining value of a block, so after consuming one element we refill from the same block — the standard "merge k sorted lists" trick with $k = \dfrac{n}{L}$. ■

5.2.7 Applications

Merging k Sorted Arrays Combine k sorted arrays of length n into one sorted sequence of length kn.

Solution	Time	Idea
Sort everything	$O(kn \log(kn))$	ignore existing order
Divide & conquer	$O(kn \log k)$	pairwise 2-way merges, $\log_2 k$ rounds of kn work
Min-heap	$O(kn \log k)$	keep one current head per array in a heap of size k

Heap method: insert the first element of each array (tagged with its source array) into a min-heap; repeatedly extract-min and insert the next element from the same source array. Each of the kn elements costs one insert + one extract, each $O(\log k)$. Uses only $O(k)$ extra space, and it works for online/streaming inputs and huge files (external merge sort).

Delete and Update-Priority Store `(key, priority, payload)` tuples, where `key` uniquely identifies the record.

- `Delete(key)`: locate its heap index k, swap $A[k]$ with $A[n]$, shrink, then **bubble up or bubble down** from k (either direction may be needed) — $O(\log n)$ once located.
- `UpdatePriority(key, p)`: set the new priority and bubble in the appropriate direction.

Locating an element by key requires a separate **position map** (hash table: key $\to$ heap index), updated on every swap. It preserves the $O(\log n)$ bounds but adds real overhead, which is why C++/Python standard heaps do not support it directly (`std::set` or lazy deletion are common workarounds).

The Skyline Problem Given buildings (x_L, x_R, y), output the skyline key points (positions where the maximum height changes).

- **D&C solution:** recursively get the skylines of two halves; merge them like mergesort while tracking current heights h_1, h_2 of each side. At each next boundary $x = \min(x_1, x_2)$, advance that side and append $(x, \max(h_1, h_2))$ only when it differs from the last emitted height. An $O(n)$ merge gives $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = O(n \log n)$.
- **Heap (sweep-line) solution:** sort all vertical boundaries $\left(x, y, \text{id}, \frac{L}{R}\right)$; sweep left to right, inserting a building at its left edge and deleting it at its right edge from a max-heap of live heights. The heap top is the current skyline height; emit a key point whenever it changes. $O(n \log n)$.

5.3 Linear-Time Sorting

All comparison-based sorts have *worst-case* $\Omega(n \log n)$ lower bound (decision-tree height). The following beat the bound by **not comparing keys directly**.

When Can We Beat $\Omega(n \log n)$?

We need extra information about the keys: their values lie in a **small integer range**, or they have structure that allows *digit-wise* processing.

5.3.1 The Decision-Tree Model and the Sorting Lower Bound

A **comparison-based** algorithm makes decisions only from pairwise comparisons ("$a_i \leq a_j$?"); its behavior depends solely on the relative order, never on actual values. Merge, heap and quick sort are comparison-based; counting/radix/bucket sort are not.

Represent any execution as a **decision tree**: each internal node is one comparison with two outcomes (left/right branch); each leaf is one possible output. The **worst-case number of comparisons is the height** of the tree, and a binary tree with N leaves has height at least $\lceil \log_2 N \rceil$.

Warm-up: Lower Bound for Binary Search

Searching a sorted array of size n has $n+1$ distinct outputs — one per element, plus "not found." Hence the decision tree needs $\geq n+1$ leaves and height $\geq \log_2(n+1) = \Omega(\log n)$. Binary search matches this bound, so it is asymptotically optimal.

Exercise: The Defective Coin Balance

Among n coins, at most one is too heavy or too light (unknown which); a three-way balance reports left-heavy / balanced / right-heavy.

- **Lower bound:** there are $2n+1$ outcomes (n heavy, n light, all genuine), so the ternary decision tree has $\geq 2n+1$ leaves: height $\geq \log_3(2n+1)$ weighings.
- **Algorithm:** split into three equal groups C_1, C_2, C_3. Two weighings (C_1 vs C_2, then C_1 vs C_3) distinguish the 7 outcomes $C_1H, C_1L, C_2H, C_2L, C_3H, C_3L$, "none"; recurse into the defective group:

$$T(n) = T(n/3) + 2 = 2 \log_3 n = O(\log n),$$

matching the lower bound up to constants.

Theorem: Sorting Requires $\Omega(n \log n)$ Comparisons

A sorting algorithm must be able to output any of the $n!$ input permutations, so its decision tree has $\geq n!$ leaves and height

$$h \geq \log_2(n!) = \sum_{i=1}^{n} \log_2 i \geq \sum_{i=\lceil n/2 \rceil}^{n} \log_2 i \geq \frac{n}{2} \log_2 \frac{n}{2} = \Omega(n \log n)$$

(equivalently use Stirling: $n! \geq \left(\frac{n}{e}\right)^n$, so $\log_2 n! = \Theta(n \log n)$). Therefore **every** comparison-based sort needs $\Omega(n \log n)$ comparisons in the worst case — Merge Sort and Heapsort are thus asymptotically optimal among comparison sorts. To do better we must leave the comparison model.

Algorithm vs problem lower bounds. "Insertion sort is $\Omega(n^2)$" is an algorithm bound (a particular algorithm's cost); "sorting is $\Omega(n \log n)$" is a problem bound (no comparison-based algorithm can beat it, proved independently of how clever the algorithm

is). Problem lower bounds are the rarer and more valuable kind — they tell us when to stop improving and point toward new models (non-comparison methods, randomization, average-case assumptions).

2024–25 Spring, Problem 1 (Decision tree, 5 points)

Problem. The following recursive algorithm returns the minimum of $A[1..n]$ of distinct numbers; the top-level call is $\textsc{Smallest}(A, 1, n)$.

```
1: function SMALLEST(A, p, r)
2:     if p = r then
3:         return A[p]
4:     end if
5:     q ← ⌊(p + r)/2⌋
6:     x ← SMALLEST(A, p, q);    y ← SMALLEST(A, q + 1, r)
7:     return min{x, y}
8: end function
```

Draw the decision tree of this algorithm for four distinct inputs a_1, a_2, a_3, a_4.

Solution. The recursion splits $\{a_1, a_2, a_3, a_4\}$ into $\{a_1, a_2\}$ and $\{a_3, a_4\}$; each half is resolved with **one** comparison, and the two winners are compared once more. Every root–leaf path therefore contains exactly 3 comparisons:

Level	Comparison made	Branching
1	a_1 vs. a_2	winner is a_1 or a_2
2	a_3 vs. a_4	winner is a_3 or a_4
3	winner of left vs. winner of right	4 combinations: a_1:a_3, a_1:a_4, a_2:a_3, a_2:a_4

The tree has 4 leaves (one per candidate minimum) and height 3, so the worst case is $3 = n - 1$ comparisons. This matches the information-theoretic bound: to certify a minimum one must "eliminate" the other $n - 1$ elements, each elimination costing one comparison. ■

2025–26 Fall, Problem 1(b) (Lower bound for the k smallest, 4 points)

Problem. $A[1..n]$ holds n distinct integers and $1 \le k \le \dfrac{n}{2}$. Prove that outputting the k smallest numbers in increasing order requires $\Omega(k \log n)$ comparisons in the worst case.

Solution (decision tree / counting outputs). A valid output is a sequence of k distinct values that are the k smallest. Choose the set: $\binom{n}{k}$ ways; choose their increasing order: $k!$ ways. So the number of possible outputs is

$$\binom{n}{k} k! = \frac{n!}{(n-k)!} = n(n-1)\cdots(n-k+1) \;\ge\; (n-k+1)^k.$$

Any binary decision tree with N leaves has height $\ge \log_2 N$, hence

$$h \ge \log_2 \left((n-k+1)^k\right) = k \log_2 (n-k+1).$$

Since $k \le \dfrac{n}{2}$ we have $n-k+1 > \dfrac{n}{2}$, so $h \ge k(\log_2 n - 1) = \Omega(k \log n)$. Why the order matters: dropping "in increasing order" would remove the $k!$ factor and destroy the bound — the multiset alone can be certified with far fewer comparisons. ■

5.3.2 Counting Sort

Assume each of n input keys is an integer in $\{0, 1, \ldots, k\}$.

Algorithm 25 Counting Sort

```
1: function CountingSort(A, B, n, k)
2:     for i = 0 to k do
3:         C[i] ← 0
4:     end for
5:     for j = 1 to n do
6:         C[A[j]] ← C[A[j]] + 1                          ▷ count frequencies
7:     end for
8:     for i = 1 to k do
9:         C[i] ← C[i] + C[i − 1]                         ▷ prefix sums = "how many ≤ i"
10:    end for
11:    for j = n downto 1 do
12:        B[C[A[j]]] ← A[j]
13:        C[A[j]] ← C[A[j]] − 1
14:    end for
15: end function
```

Counting Sort Properties

- Runtime: $\Theta(n + k)$. If $k = O(n)$ then $\Theta(n)$ overall.
- **Stable:** yes (fill B *right-to-left* so equal keys preserve input order). This is critical for being used as a subroutine in Radix Sort.
- **In-place:** no (uses arrays C of size $k + 1$ and output B of size n).

Worked example: $A = [2, 5, 3, 0, 2, 3, 0, 3]$, $k = 5$.

i (key value)	0	1	2	3	4	5
frequencies C	2	0	2	3	0	1
prefix sums C	2	2	4	7	7	8

Placing right-to-left (equal keys keep their order): $B = [0, 0, 2, 2, 3, 3, 3, 5]$.

5.3.3 Radix Sort

Sort d-digit keys by applying a **stable** sort digit-by-digit, least significant digit first (LSD). Each digit sort is Counting Sort on that digit.

Algorithm 26 Radix Sort

```
1: function RadixSort(A, d, n)
2:     for i = 1 to d do
3:         Use a stable sort to sort A on digit i (the i-th least significant)
4:     end for
5: end function
```

Radix Sort Correctness & Complexity

- Why LSD works: after pass i, numbers are sorted correctly w.r.t. their low i digits; next pass (higher digit) is stable so ties keep the lower-order ordering.
- Each digit in $\{0, \ldots, k - 1\}$ sorted by Counting Sort in $\Theta(n + k)$.

- Total: $\Theta(d(n+k))$. If d is constant and $k = O(n)$, this is $\Theta(n)$.
- **Stable:** yes (because inner sort is stable).

Radix Sort Gotchas

- Must process digits **from LSD to MSD** (not the other way around).
- Inner sort **must be stable**; otherwise equal-digit elements lose their previously-computed order.

Correctness by Induction on Digit Position

Claim: after pass i, keys are sorted by their low-order i digits. Base $i = 0$ vacuous. Assume sorted by low $i-1$ digits; after stably sorting on digit i:

- two keys differing in digit i are correctly ordered by that more significant digit;
- two keys equal in digit i keep their pre-pass order by **stability**, which is correct ordering by the low $i-1$ digits (induction hypothesis).

Both cases give correctness by low i digits; after d passes the keys are fully sorted.

Worked example (3 LSD passes):

input	$[329, 457, 657, 839, 436, 720, 355]$
pass 1 (units)	$[720, 355, 436, 457, 657, 329, 839]$
pass 2 (tens)	$[720, 329, 436, 839, 355, 457, 657]$
pass 3 (hundreds)	$[329, 355, 436, 457, 657, 720, 839]$

Bit-Parameter Analysis (w-bit Integers)

Each "digit" spans $\log_2 k$ bits, so the number of passes is $d = \left\lceil \frac{w}{\log k} \right\rceil$ and total time is

$$\Theta\left(\left\lceil \frac{w}{\log k} \right\rceil (n+k) \right).$$

The $(n+k)$ factor is minimized at $k = n$ (equal split between pass count and bucket overhead), giving $\Theta(\left\lceil \frac{w}{\log n} \right\rceil n)$. In the standard word-RAM regime $w = O(\log n)$ (pointers/indices fit in a machine word; all machines from 8-bit to 64-bit satisfy this for realistic RAM sizes), radix sort runs in $\Theta(n)$. It also extends to fixed-length strings and to floating-point keys (their IEEE-754 bit order is monotone after a sign flip).

2025–26 Fall, Problem 1(c) (Radix sort passes, 4 points)

Problem. Run radix sort (LSD first) on the five 4-digit numbers below and give the array after each pass:

$$9341, \quad 6275, \quad 4194, \quad 7819, \quad 3532.$$

Solution. Each pass is a stable counting sort on one digit.

Pass	Digit sorted	Order of that digit	Array afterwards
1 (units)	$1, 5, 4, 9, 2$	$1, 2, 4, 5, 9$	[9341, 3532, 4194, 6275, 7819]
2 (tens)	$4, 3, 9, 7, 1$	$1, 3, 4, 7, 9$	[7819, 3532, 9341, 6275, 4194]
3 (hundreds)	$8, 5, 3, 2, 1$	$1, 2, 3, 5, 8$	[4194, 6275, 9341, 3532, 7819]
4 (thousands)	$4, 6, 9, 3, 7$	$3, 4, 6, 7, 9$	[3532, 4194, 6275, 7819, 9341]

The fourth pass yields the sorted order. Check the stability requirement: in pass 2, 9341 and 4194 both have tens digit 4 and 9; their relative order inherited from pass 1 must be preserved — a non-stable inner sort would silently break correctness. Total time $\Theta(d(n+k))$ with $d = 4$, $k = 10$. ■

5.3.4 Bucket Sort

Assume input is drawn **uniformly at random** from $[0, 1)$. Distribute into n equal-sized buckets, sort each bucket (e.g. insertion sort), concatenate.

Algorithm 27 Bucket Sort

```
1: function BucketSort(A, n)
2:     Create B[0..n − 1] (each an empty list)
3:     for i = 1 to n do
4:         insert A[i] into B[⌊n · A[i]⌋]
5:     end for
6:     for i = 0 to n − 1 do
7:         sort list B[i] (e.g. insertion sort)
8:     end for
9:     Concatenate B[0], B[1], . . . , B[n − 1] in order
10: end function
```

Bucket Sort Expected Time

With uniform input, each bucket has $O(1)$ elements in expectation; sorting each by insertion sort is $O(1)$ per bucket.

$$\mathbb{E}[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} O(1) = \Theta(n).$$

Worst case (all elements fall in one bucket): $\Theta(n^2)$ if inner sort is insertion sort.

Why the Expected Cost Is Linear: $\mathbb{E}[X_0^2] = O(1)$

Let X_i be the number of elements in bucket i and $Y_j = \mathbf{1}\{A[j] \text{ lands in bucket } 0\}$, so $X_0 = \sum_{j=1}^{n} Y_j$. Insertion sort on bucket i costs $\Theta(X_i^2)$; by identical distributions,

$$\mathbb{E}\left[\sum_{i=0}^{n-1} X_i^2\right] = n\,\mathbb{E}[X_0^2].$$

Expand the square and use independence of distinct Y_j ($\Pr[Y_j = 1] = \dfrac{1}{n}$):

$$\mathbb{E}[X_0^2] = \sum_j \mathbb{E}[Y_j^2] + 2\sum_{j<\ell} \mathbb{E}[Y_j Y_\ell] = n \cdot \frac{1}{n} + 2\binom{n}{2}\frac{1}{n^2} = 1 + 1 - \frac{1}{n} = 2 - \frac{1}{n} = O(1).$$

So bucket sorting costs $O(n)$ expected time total (plus the $O(n)$ distribution/concatenation passes). If inputs are not in $[0, 1)$, normalize first; with non-uniform distributions a custom bucket partition is needed.

Sorting Algorithm Cheat Sheet

Algorithm	**Worst**	**Avg.**	**Space**	**Comp.**	**Stab.**	**Cache**	**Par.**
Insertion	n^2	n^2	1	yes	yes	good	no
Mergesort	$n \log n$	$n \log n$	n	yes	yes	good	good
Quicksort	n^2	$n \log n$	$\log n$	yes	no	good	good
Heapsort	$n \log n$	$n \log n$	1	yes	no	bad	no
Counting	$n + k$	$n + k$	$n + k$	no	yes	bad	no
Radix	$\dfrac{w}{\log n} n$	same	n	no	yes	bad	good
Bucket	n^2	n (exp.)	n	no	yes	bad	good

2025–26 Fall, Problem 1(d) (Choosing the sorting algorithm, 3 points)

Problem. Among selection/insertion/merge/heap/quick/counting/radix sort, which gives the smallest worst-case running time, and what is it?

(i) n integers in $[1, n]$;

(ii) n floating-point numbers with $O(\sqrt{\log n})$ inversions;

(iii) n arbitrary floating-point numbers.

Solution.

Case	Structure exploited	Best algorithm	Worst case
(i)	keys are integers in a size-n range	counting sort	$O(n)$
(ii)	nearly sorted (few inversions)	insertion sort	$O(n\sqrt{\log n})$
(iii)	comparison model only (no small range)	merge sort or heapsort	$O(n \log n)$

Reasoning: (i) counting sort runs in $\Theta(n + k)$ with $k = n$. (ii) insertion sort costs $\Theta(n + I)$ where I is the inversion count, and no comparison sort can beat $\Omega(n + I)$; quicksort/mergesort ignore the structure and stay $\Theta(n \log n)$. (iii) floating-point values are not restricted to a small integer range, so the $\Omega(n \log n)$ comparison lower bound applies and must be met (quicksort is only $O(n \log n)$ average). ■

5.4 External Sorting

Sorting a file that does not fit in main memory requires a different model. Records are stored on disks or tapes and are read and written in large sequential blocks; the cost of an algorithm is dominated by the number of passes over the data, not by comparisons.

5.4.1 The Model

Input. N records, each with a key. Internal memory can hold and sort at most M records at a time. There are T external storage units (tapes or disk files), each of which can be read or written sequentially.

A **run** is a sorted sequence of records stored on one tape. In **pass 1** we read M records at a time, sort them internally and write them out as runs; this produces $\left\lceil \frac{N}{M} \right\rceil$ initial runs. Each subsequent **pass** performs a **k-way merge**: it takes k runs and merges them into one longer run. A k-way merge of $k \cdot c$ runs produces c runs, each k times longer.

Quantity	Value
initial runs	$\left\lceil \frac{N}{M} \right\rceil$
runs after one k-way pass	divided by k
number of passes	$1 + \left\lceil \log_k \frac{N}{M} \right\rceil$
number of tapes for k-way merge	$2k$

The extra 1 counts pass 1 (run generation). To merge k runs we need k input tapes (one run each) plus k output tapes (so that output can be written while the next runs are being read), hence $2k$ tapes. This is why a large k is attractive — it reduces the number of passes logarithmically — but also why it is not simply "as large as possible": with many simultaneous tape units the probability of a hardware read error grows, and a single error forces a re-run.

Exercise: Counting Passes

Problem. We have 100,000,000 records of 256 bytes each, and internal memory is 128 MB. If simple 2-way merges are used, how many passes are needed: (A) 10; (B) 9; (C) 8; (D) 7?

Solution. $M = 128$ MB $= 2^{27}$ bytes, which holds $2^{27}/2^8 = 2^{19}$ records. $N = 10^8$. Then

$$\text{passes} = 1 + \left\lceil \log_2 \frac{10^8}{2^{19}} \right\rceil = 1 + \lceil \log_2 (1.9 \times 10^2) \rceil = 1 + 8 = 9.$$

Answer (B). ■

5.4.2 Replacement Selection: Longer Initial Runs

The initial runs need not be of length M. **Replacement selection** uses a min-heap of size M:

1. read M records into the heap;
2. repeatedly extract the minimum and write it to the current run, then read the next input record;
3. if the new record is larger than the record just written, insert it into the heap and continue the same run;

4. if it is smaller, it belongs to the next run: keep it aside ("frozen") rather than inserting it into the heap;
5. when the heap is empty, the current run ends, the frozen records form the new heap, and a new run begins.

If the input is random, the expected run length is about $2M$, so replacement selection halves the number of initial runs. If the input is already sorted, every record can join the current run and a single run of length N is produced.

Exercise: Replacement Selection

Problem. The input keys are $(25, 74, 56, 34, 21, 11, 29, 80, 38, 53)$ and memory holds 3 records. What is the minimum number of initial runs produced by replacement selection?
Solution. Load $25, 74, 56$; output 25, load 34; output 34, load 21 (frozen, since $21 < 34$); output 56, load 11 (frozen); output 74; the heap has only frozen records, so run 1 is $(25, 34, 56, 74)$ and ends. The frozen $(21, 11, 29)$ become the new heap; output 11, 21, 29, then read $80, 38, 53$, output $38, 53, 80$. Run 2 is $(11, 21, 29, 38, 53, 80)$. Answer: 2 runs. ■

Exercise: Longer Runs with a Heap of Size 4

Problem. Apply replacement selection with a priority queue of size 4 to $(9, 75, 17, 12, 88, 91, 25, 22, 35, 41, 58, 96, 15)$. Which is the second output run: (A) 22 35 88 91; (B) 22 35 41 58 88 91 96; (C) 22 35 41 58 96; (D) 22 35 41 58?
Solution. Run 1 is $(9, 12, 17, 25, 75, 88, 91)$; run 2 is $(22, 35, 41, 58, 96)$; run 3 is (15). Answer (C). ■

5.4.3 Polyphase Merge: Fewer Tapes

Keeping $2k$ tapes is wasteful because half of them are output tapes that are idle most of the time. **Polyphase merging** uses only $k + 1$ tapes: at any moment one tape is the output tape and the other k hold input runs, and the roles rotate from pass to pass.

For $k = 2$ and three tapes, the distribution of runs must follow the **Fibonacci numbers**, because in a pass each output run consumes one run from each of the two input tapes; if a tape runs out early, the merge cannot continue efficiently. Thus F_N runs should be split as F_{N-1} and F_{N-2}.

More generally, the k-th order Fibonacci numbers are

$$F_N^k = F_{N-1}^k + F_{N-2}^k + \cdots + F_{N-k}^k, \qquad F_N^k = 0 \ (0 \le N \le k-2), \quad F_{k-1}^k = 1.$$

With 34 runs and 2-way merging, split them as $(21, 13)$ on tapes T_1, T_2; the merges then proceed as follows.

T_1	T_2	T_3	**Explanation**
21	13	—	initial distribution
8	—	13	merge the first 13 runs of T_1 with T_2 into T_3
—	8	5	merge the first 8 runs of T_3 with T_1 into T_2
5	3	—	merge T_2's 5 runs with T_3's 5 runs into T_1
2	—	3	continue rotating
—	2	1	continue rotating
1	1	—	continue rotating
—	—	1	done

Exercise: Fibonacci Distribution

Problem. To merge 55 runs using 3 tapes for a 2-way merge, is the original distribution $(34, 21)$ better than $(27, 28)$?
Solution. True. $55 = F_{10}$, and Fibonacci splits $(F_9, F_8) = (34, 21)$ are exactly the distributions for which no tape empties early. A balanced split $(27, 28)$ wastes work because one tape runs out before the other, so extra passes are needed. ■

5.4.4 Optimising the Merge Order: Huffman Merging

When the initial runs have different lengths, the order in which they are merged affects the total amount of data moved. Merging runs of lengths a and b costs $a+b$ (every record is read and rewritten), so the total cost is the weighted external path length of the merge tree. By the same argument as Huffman coding, merging the two shortest runs first is optimal.

Exercise: Minimum Merge Time

Problem. Five runs have lengths $2, 8, 9, 5, 3$. Which merging order minimises the total merge time?
(A) merge $2 + 3$, then with 5, then with 8, then with 9;
(B) merge $2 + 3$, then with 5; merge $8 + 9$; then merge the two results;
(C) merge $2 + 3$, merge $5 + 8$, then merge, then with 9;
(D) merge $2 + 3$, merge $5 + 8$, then with 9, then merge.
Solution. (B), the Huffman strategy. The costs are: $2+3 = 5$; $5+5 = 10$; $8+9 = 17$; $10+17 = 27$; total $= 5+10+17+27 = 59$. In (A) the total is $5+10+18+27 = 60$; in (C) and (D) it is larger as well. ■

5.4.5 Parallel Merging

To overlap reading, merging and writing, each input run gets **2 input buffers** and the output gets **2 output buffers**, so a k-way merge uses $2k$ input buffers and 2 output buffers. While one buffer of a run is being consumed by the merge, the other is being refilled from tape; while one output buffer is being written, the other is being filled. A 3-way merge therefore needs 6 input buffers and 2 output buffers.

Exercise: Parallel Operations

Problem. Memory holds 12 numbers. Merge Run 1 $= (1, 3, 5, 7, 8, 9, 10, 12)$ and Run 2 $= (2, 4, 6, 15, 20, 25, 30, 32)$ with 2-way merge, 4 input buffers and 2 output buffers. Which group of operations is not done in parallel: (A) write $1, 2$; merge $3, 4$; read $6, 15$; (B) write $3, 4$; merge $5, 6$; read $8, 9$; (C) write $5, 6$; merge $7, 8$; read $20, 25$; (D) write $7, 8$; merge $9, 15$; read $10, 12$?
Solution. (D). With two buffers per run, the reads must stay two positions ahead of the merge and the writes two positions behind. In (D) the merge has reached 9 and 15, so the corresponding input buffers are exhausted and $10, 12$ cannot yet have been read — the read described is out of step with the merge. ■

6 Design Paradigms: Greedy, Huffman, Backtracking and Dynamic Programming

6.1 Greedy Algorithms

A **greedy algorithm** always makes the choice that looks best at the moment and adds it to the current partial solution. It builds a solution to an optimization problem through a sequence of choices that are:

- **Feasible:** satisfying the given constraints.
- **Locally optimal:** make the choice that looks best right now, without considering the future.
- **Irrevocable:** once a choice is made it is never undone (no backtracking).

When Does Greedy Work?

- A greedy algorithm is correct when the problem has the **greedy-choice property** (a locally optimal choice can be part of some global optimum) together with **optimal substructure** (an optimal solution contains optimal solutions to subproblems).
- When they work, greedy algorithms are usually the simplest and most efficient algorithms available. A sorting step often dominates the cost, giving $\Theta(n \log n)$.
- When they fail, they may still give fast **approximations**.

Greedy Correctness Must Be Proven!

"Picking the locally best option" is not automatically optimal. The correctness (optimality) of a greedy algorithm is usually non-obvious. Two standard proof techniques:

1. **Exchange argument:** transform any optimal solution into the greedy solution one choice at a time, without worsening it (Interval Scheduling, Fractional Knapsack, Huffman).
2. **Greedy stays ahead / match a lower bound:** show the greedy solution reaches a universal lower bound (Interval Partitioning vs. depth).

6.1.1 Warm-up: Change-Making

Input: amount n and coin denominations $d_1 > d_2 > \cdots > d_m$. Output: the minimum number of coins summing to n.

Greedy rule: give as many coins as possible of the largest denomination, then the second largest, and so on.

Example (Hong Kong coins): $n = 19$, denominations $10, 5, 2, 1$.

$$19 = 10 + 5 + 2 + 2 \quad \Longrightarrow \quad 4 \text{ coins, and this is optimal.}$$

Greedy Change-Making Is Not Always Optimal

British coinage before 1971 used denominations $1, 3, 6, 12, 24, 30, 60$. For $n = 48$:

- Greedy: $48 = 30 + 12 + 6$ (3 coins).
- Optimal: $48 = 24 + 24$ (2 coins).

The greedy choice (take 30) "steals" capacity that two better coins could have used. For arbitrary denominations, change-making requires **dynamic programming**.

6.1.2 Warm-up: Max Sum of Non-Adjacent Elements

Given n positive numbers $A[1], \ldots, A[n]$, pick a subset of **pairwise non-adjacent** elements with maximum total sum. Example: $A = (1, 8, 6, 3, 6) \Rightarrow$ pick $A[2], A[5]$ with sum $8 + 6 = 14$.

Greedy attempt: repeatedly take the largest remaining element, then delete it and its two neighbors.

Algorithm 28 Greedy Non-Adjacent Selection (Not Optimal!)

```
S ← ∅
while some elements remain in A do
    pick index i with the largest remaining A[i]
    S ← S ∪ {A[i]}
    mark A[i − 1], A[i], A[i + 1] as deleted
end while
return S
```

Counterexample: $(1, 4, 6, 4)$

- Greedy takes the maximum 6, which forces deleting both neighbors 4 and 4, then takes 1: sum $= 6 + 1 = 7$.
- Optimal: take the two 4's (positions 2 and 4): sum $= 8 > 7$.

The optimal solution is given by DP: $\mathrm{OPT}(i) = \max\big(\mathrm{OPT}(i-1),\ \mathrm{OPT}(i-2) + A[i]\big)$.

6.1.3 Interval Scheduling

Job j starts at s_j and finishes at f_j. Two jobs are **compatible** if their intervals do not overlap. Goal: select a **maximum-size** subset of mutually compatible jobs.

Three Tempting Greedy Rules (All Fail)

Counterexamples

1. **Earliest start time.** One long job $e = [0, 10)$ starting first can block four short jobs $[1, 2), [2, 3), [3, 4), [4, 5)$: greedy chooses $\{e\}$ (size 1) instead of $\{a, b, c, d\}$ (size 4).
2. **Shortest interval.** Let $a = [0, 4)$, $b = [4, 8)$, and a very short $c = [3.5, 4.5)$ overlapping both. Shortest-first chooses $\{c\}$ (size 1) instead of $\{a, b\}$ (size 2).
3. **Fewest conflicts.** Count each job's overlapping jobs c_j and schedule in ascending c_j. A job conflicting with only a few others can still sit in the middle of a long compatible chain and, once chosen, force several of them out — the slide example forces $\{a, f, d\}$ (size 3) instead of $\{a, b, c, d\}$ (size 4). The number of conflicts is a global statistic and does not measure local opportunity cost.

The Correct Rule: Earliest Finish Time **Greedy algorithm:** consider jobs in increasing order of finish time; take a job whenever it is compatible with the jobs already chosen. Intuition: finishing early **leaves the maximum remaining time** for all later jobs.

Algorithm 29 Interval Scheduling (Earliest Finish Time)

```
Sort jobs so that f_1 ≤ f_2 ≤ ··· ≤ f_n
A ← ∅;   last ← 0                          ▷ finish time of the last selected job
for j = 1 to n do
    if s_j ≥ last then                     ▷ compatible: ends are inclusive, so ≥ is fine
        A ← A ∪ {j};   last ← f_j
    end if
end for
return A
```

Running time: $\Theta(n \log n)$, dominated by sorting; the scan itself is $\Theta(n)$.

Worked trace. Jobs sorted by finish time:

$$j_1 = [0, 2),\ j_2 = [1, 4),\ j_3 = [3, 5),\ j_4 = [4, 6),\ j_5 = [5, 7).$$

- Select j_1 (earliest finish), set $last = 2$.
- Skip j_2: $s_2 = 1 < 2$, overlaps j_1.
- Select j_3: $s_3 = 3 \geq 2$, set $last = 5$.
- Skip j_4: $s_4 = 4 < 5$.
- Select j_5: $s_5 = 5 \geq 5$.

Greedy returns $\{j_1, j_3, j_5\}$, size 3 — the maximum possible.

Correctness Proof (Exchange Argument)

Theorem: Earliest-Finish-Time Greedy Is Optimal

Let $G = (i_1, i_2, \ldots, i_k)$ be the jobs selected by greedy and $O = (j_1, j_2, \ldots, j_m)$ be any optimal schedule, both in selection order.

1. Suppose $G \neq O$. Find the largest r such that $i_1 = j_1, \ldots, i_r = j_r$ (the common prefix; $r = 0$ if they differ immediately).
2. By the greedy rule, i_{r+1} is the compatible job with the **earliest finish**, so $f_{i_{r+1}} \leq f_{j_{r+1}}$.
3. Build O^* from O by replacing j_{r+1} with i_{r+1}. Because i_{r+1} finishes no later than j_{r+1}, it is still compatible with $j_{r+2}, \ldots, j_m$; hence O^* is a feasible schedule with the same size as O, so O^* is also optimal.
4. O^* now shares the first $r + 1$ jobs with G. Repeat the replacement; the common prefix grows each time, so eventually O is transformed into G with its size (value) unchanged.

Therefore $|G| = |O|$, i.e. greedy is optimal. Equivalently, the greedy schedule "stays ahead": the r-th greedy job finishes no later than the r-th job of any schedule, so greedy can always fit at least as many jobs.

6.1.4 Exercise: Interval Covering

Given n intervals that together cover $[0, 1]$; interval x has start $x.s$ and finish $x.f$. Find the **minimum** number of intervals whose union covers $[0, 1]$. (Assume all endpoints are distinct.)

Greedy idea: maintain the rightmost covered point x. Among all intervals covering x ($I.s \leq x < I.f$), pick the one extending **furthest to the right**; advance x to its finish and repeat until $x \geq 1$.

Algorithm 30 Greedy Interval Covering

```
x ← 0;   G ← ∅
while x < 1 do
    pick interval I with I.s ≤ x that maximizes I.f
    G ← G ∪ {I};   x ← I.f
end while
return G
```

Worked trace. Intervals $A = [0, 0.3]$, $B = [0.2, 0.6]$, $C = [0.5, 0.8]$, $D = [0.75, 1]$:

- $x = 0$: only A covers $0 \Rightarrow$ take A, $x = 0.3$.
- $x = 0.3$: B covers 0.3 and extends furthest $\Rightarrow$ take B, $x = 0.6$.
- $x = 0.6$: C covers 0.6 (D starts at 0.75) $\Rightarrow$ take C, $x = 0.8$.
- $x = 0.8$: take D, $x = 1$. Done with 4 intervals.

Optimality Sketch & Implementation

Let $O = (o_1, \ldots, o_m)$ be optimal and $G = (g_1, \ldots, g_k)$ greedy, ordered by start time. Let i be the first index with $o_i \neq g_i$. The greedy choice extends at least as far: $g_i.f \geq o_i.f$. Replacing o_i by g_i keeps the cover valid; iterating converts O into G, so $k \leq m$ and greedy is optimal. Sorting plus a single sweep (among intervals already started, track the maximum finish) gives $O(n \log n)$.

6.1.5 Fractional Knapsack

Input: n items, item i has weight w_i and value v_i; knapsack capacity W. Choose fractions $0 \leq x_i \leq 1$ maximizing $\sum_i x_i v_i$ subject to $\sum_i x_i w_i \leq W$.

- **0/1 knapsack:** $x_i \in \{0, 1\}$ (take the whole item or nothing) — greedy fails; needs DP.
- **Fractional knapsack:** $x_i \in [0, 1]$ (items are divisible, e.g. liquids, grain) — greedy is optimal.

Greedy rule: sort items by **value per unit weight** $\frac{v_i}{w_i}$ decreasing; take as much as possible of each item in that order. Running time $\Theta(n \log n)$.

Algorithm 31 Fractional Knapsack (Greedy)

```
Sort items so that v_1/w_1 ≥ v_2/w_2 ≥ ··· ≥ v_n/w_n
w ← W                                   ▷ remaining capacity
for i = 1 to n do
    if w_i ≤ w then
        x_i ← 1;   w ← w − w_i
    else
        x_i ← w/w_i;   w ← 0;   return
    end if
end for
```

Worked example. $W = 50$; items $(w_i, v_i) = (10, 60), (20, 100), (30, 120)$, with ratios $6, 5, 4$.

1. Take all of item 1: used weight 10, value 60.
2. Take all of item 2: used weight 30, value 160.
3. Fill the remaining 20 units with a fraction $\frac{20}{30} = \frac{2}{3}$ of item 3: added value $\frac{2}{3} \cdot 120 = 80$.

Optimal total: $60 + 100 + 80 = \mathbf{240}$, weight exactly 50.

Why the Same Greedy Fails for 0/1 Knapsack

On the example above, "take by ratio while the whole item fits" grabs items 1 and 2 (weight 30, value 160) and then item 3 does not fit — leaving 20 units of capacity wasted. The 0/1 optimum takes items 2 and 3: weight 50, value 220 > 160. Being forced to leave capacity unused is exactly what divisibility (the fractional version) avoids.

Correctness Proof (Exchange Argument)

Assume $\sum_i w_i \geq W$ (otherwise take everything; trivially optimal), so both greedy $G = (x_1, \ldots, x_k, 0, \ldots, 0)$ (where $x_i = 1$ for $i < k$ and $0 \leq x_k \leq 1$) and any optimum $O = (y_1, \ldots, y_n)$ fill the knapsack exactly.

1. Let i be the first index where $x_i \neq y_i$. By the greedy ordering and the prefix agreement, greedy takes at least as much of the best remaining item: $x_i \geq y_i$.
2. Increase y_i to x_i and compensate by removing total weight $(x_i - y_i)w_i$ from items $i+1, \ldots, n$ (enough such weight exists because both solutions pack exactly W).
3. Value cannot decrease: every item $j > i$ has ratio $\frac{v_j}{w_j} \leq \frac{v_i}{w_i}$, so swapping weight onto item i replaces "cheaper" weight with "dearer" weight.
4. Since O was optimal, its value also cannot increase, so it stays exactly the same: the modified O is still optimal and now agrees with G up to index $i+1$.

Repeating converts O into G with unchanged value $\Rightarrow$ greedy is optimal.

6.1.6 Interval Partitioning

Lecture j occupies $[s_j, f_j)$. Goal: schedule all lectures using the **minimum number of classrooms** so that no two lectures overlap in the same room.

Greedy algorithm: sort lectures by increasing **start** time; assign each lecture to any classroom whose last lecture has finished; if none exists, open a new classroom.

Algorithm 32 Interval Partitioning (Greedy by Start Time)

```
Sort lectures so that s_1 ≤ s_2 ≤ ··· ≤ s_n
H ← empty min-heap of classroom finish times
for j = 1 to n do
    if H ≠ ∅ and min(H) ≤ s_j then
        ExtractMin(H)                          ▷ reuse that classroom
    end if
    Insert(H, f_j)                             ▷ new room if none was reused
end for
return |H|                                     ▷ number of classrooms used
```

Worked trace. Lectures (sorted by start): $A = [1, 4)$, $B = [2, 6)$, $F = [3, 3.5)$, $C = [4, 5)$, $D = [5, 7)$, $E = [6, 8)$.

Lecture	Decision	Heap of finish times	Overlapping rooms
$A[1, 4)$	open room 1	$\{4\}$	1
$B[2, 6)$	$4 > 2$, open room 2	$\{4, 6\}$	2
$F[3, 3.5)$	$4 > 3$, open room 3	$\{3.5, 4, 6\}$	3
$C[4, 5)$	$3.5 \leq 4$, reuse	$\{4, 5, 6\}$	3
$D[5, 7)$	$4 \leq 5$, reuse	$\{5, 6, 7\}$	3
$E[6, 8)$	$5 \leq 6$, reuse	$\{6, 7, 8\}$	3

Three rooms are used; at time 3 lectures A, B, F overlap simultaneously, so 3 is unavoidable.

Depth Lower Bound and Correctness

The **depth** of a set of intervals is the maximum number of intervals covering any single point in time.

- **Lower bound:** every schedule needs at least depth classrooms — at that instant all those lectures are running and each needs its own room.
- **Greedy matches it:** suppose greedy opens its d-th classroom for lecture j. At that moment all $d-1$ existing rooms have finish time $> s_j$; by the start-time ordering their lectures started at or before s_j, so together with j there are d lectures covering time s_j. Hence depth $\geq d$.

Greedy uses exactly (no more than) the depth $\Rightarrow$ optimal.

Running Time

- Naively scanning all rooms for compatibility: $O(n^2)$.
- Key observation: if j is incompatible with the room whose last lecture finishes **earliest**, it is incompatible with every room. So one min-heap keyed by room finish times suffices: each lecture does one `Min` check ($O(1)$) and at most one `ExtractMin`/`Insert` ($O(\log n)$ each).
- Total: $\Theta(n \log n)$ including the sort.

Sorting by Finish Time with Arbitrary Room Assignment Fails

Intervals $(1,3), (2,4), (4,5), (3,6)$, processed by increasing finish time:

1. $(1,3) \rightarrow$ room 1 (finishes 3).
2. $(2,4)$: no room free $\rightarrow$ room 2 (finishes 4).
3. $(4,5)$: both rooms are free; **arbitrarily** choose room 1 (finish 3) $\rightarrow$ room 1 now finishes 5.
4. $(3,6)$: neither room is free $(5, 4 > 3) \rightarrow$ room 3.

This uses 3 rooms although depth is 2 (optimal: room 1 gets $(1,3), (3,6)$; room 2 gets $(2,4), (4,5)$). **Fix (tie-breaking):** if several rooms are available, reuse the one with the **latest** finish time (best fit). Implement with a BST of rooms ordered by finish time: take the predecessor with finish $\leq s_j$ and update it to f_j; total $O(n \log n)$.

6.2 Huffman Coding

Huffman coding is another greedy algorithm: it builds an optimal **prefix-free binary code** for a file whose character frequencies are known.

6.2.1 Motivation: Variable-Length Encoding

There are over 10^6 characters across all languages: a fixed-length code would need ≥ 21 bits per character, while English needs only 7 (ASCII). Internet traffic is predominantly English, so **variable-length** codes — short codewords for frequent characters, long codewords for rare ones — save many bits in expectation.

6.2.2 Prefix-Free Codes

A code is a **prefix-free code** ("prefix code") if no codeword is a prefix of any other codeword.

Prefix-Free ⇒ Unique Decodability

Theorem. Every message encoded by a prefix-free code is uniquely decodable: reading left to right, the bits can match only one codeword (it cannot be a prefix of a longer one), so peel it off and continue.

Examples.

- $C_1 = \{a = 00, b = 01, c = 10, d = 11\}$ (fixed-length): $010011 = 01\,|\,00\,|\,11 \rightarrow$ "bad".
- $C_2 = \{a = 0, b = 110, c = 10, d = 111\}$ (prefix-free): $1100111 = 110\,|\,0\,|\,111 \rightarrow$ "bad".
- $C_3 = \{a = 1, b = 110, c = 10, d = 111\}$ (not prefix-free, since 1 is a prefix of 110 and 111): 1101111 can be parsed as $110\,|\,1\,|\,\overline{111}$ = "bad" or $1\,|\,10\,|\,1\,|\,111$ = "acad" — ambiguous!

Why Prefix Codes in Practice?

UTF-8 marks continuation bytes with the prefix `10`. If bytes are corrupted or dropped, the decoder self-synchronizes: skip ahead to the next byte not beginning with `10` rather than losing the rest of the file.

6.2.3 The Encoding Problem

Given characters with known frequencies, find a prefix code minimizing the total encoded length (compression).

Example file: 100,000 characters over $\{a, b, c, d, e, f\}$ with frequencies in thousands:

	a	b	c	d	e	f	Total (bits)
Frequency ($\times$1000)	45	13	12	16	9	5	100,000 chars
Fixed-length code	000	001	010	011	100	101	$3 \cdot 100{,}000 = 300{,}000$
Variable-length code	0	101	100	111	1101	1100	224,000

The variable code costs

$(45{\cdot}1{+}13{\cdot}3{+}12{\cdot}3{+}16{\cdot}3{+}9{\cdot}4{+}5{\cdot}4){\cdot}1000 = (45{+}39{+}36{+}48{+}36{+}20){\cdot}1000 = 224{,}000$ bits,

a 25% reduction.

6.2.4 Tree Correspondence and Weighted External Path Length

Prefix Codes ↔ Binary Trees

Represent a prefix code by a binary tree: label the left edge 0 and the right edge 1; the codeword of a character is the bit string on the root-to-leaf path.

- Every **leaf** stores one character; an internal node has two children (a "full" tree — no leaf codeword can be a prefix of another).
- Leaf depth $d(a_i)$ equals codeword length.
- Set leaf weight to the character frequency $f(a_i)$. The **weighted external path length**

$$B(T) = \sum_{i=1}^{n} f(a_i)\, d(a_i)$$

is exactly the total encoded length. Finding the minimum-cost prefix code = finding a minimum-weighted-external-path-length binary tree!

In the example, the fixed-length tree (depth 3 for all six leaves) costs (45+13+12+16+9+5)·3 = 300 (thousand bits); the variable tree costs 45·1+(12+13+16)·3+(9+5)·4 = 224.

6.2.5 Huffman's Greedy Algorithm

Greedy idea (a "merge"):

1. Pick the two characters x, y with the **smallest** weights.
2. Join them as sibling leaves under a new internal node z with weight $f(z) = f(x)+f(y)$.
3. Remove x, y from the alphabet and insert z.
4. Repeat until one node (the root) remains.

Algorithm 33 Huffman

```
1:  function HUFFMAN(A)
2:      build a min-priority queue Q on A, keyed by weight
3:      for i = 1 to n − 1 do
4:          allocate a new node z
5:          x ← EXTRACTMIN(Q)
6:          y ← EXTRACTMIN(Q)
7:          z.left ← x;    z.right ← y
8:          z.weight ← x.weight + y.weight
9:          INSERT(Q, z)
10:     end for
11:     return EXTRACTMIN(Q)                    ▷ root of the Huffman tree
12: end function
```

Running time: $O(n \log n)$ with a binary min-heap ($n-1$ merge rounds, each with two extract-mins and one insert, each $O(\log n)$).

6.2.6 Worked Example

Initial alphabet (weights): f:5, e:9, c:12, b:13, d:16, a:45.

Round	Merge	Priority queue afterwards
0	—	$\{5f,\ 9e,\ 12c,\ 13b,\ 16d,\ 45a\}$
1	$5f + 9e = 14$	$\{12c,\ 13b,\ 14,\ 16d,\ 45a\}$
2	$12c + 13b = 25$	$\{14,\ 16d,\ 25,\ 45a\}$
3	$14 + 16d = 30$	$\{25,\ 30,\ 45a\}$
4	$25 + 30 = 55$	$\{45a,\ 55\}$
5	$45a + 55 = 100$	$\{100\}$ (root)

Reading root-to-leaf paths (left = 0, right = 1) gives:

Character	a	c	b	f	e	d
Codeword	0	100	101	1100	1101	111
Length	1	3	3	4	4	3

Two Ways to Compute the Total Cost

- By leaf depths: $B(T) = \sum_i f_i d_i = 45{\cdot}1 + (12 + 13 + 16){\cdot}3 + (9 + 5){\cdot}4 = 224.$
- By merge weights: $B(T) = 14 + 25 + 30 + 55 + 100 = 224.$

The sums agree because a leaf of weight f contributes f to the merge weight of every internal node on its path to the root — exactly one contribution per level of depth. This identity is the key to both the proof and the optimal-merge application.

6.2.7 Correctness Proof

Lemma 1: An Optimal Tree Is Full

Every internal node in an optimal prefix-code tree has exactly two children. If an internal node had only one child, contract that node (replace it by its child): no leaf gets deeper, and some leaves get shallower, strictly decreasing $B(T)$ — contradiction.

Lemma 2: Swapping Leaves

Let T' be obtained from T by swapping two leaves x and b. If $f(x) \le f(b)$ and $d(x) \le d(b)$ (move the smaller-frequency leaf down and the larger one up), then

$$B(T') = B(T) + \big(f(x) - f(b)\big)\big(d(b) - d(x)\big) \le B(T),$$

since the first factor is ≤ 0 and the second is ≥ 0. So placing frequent characters nearer the root never increases the cost.

Lemma 3: The Two Smallest Weights Can Be Siblings at Maximum Depth

Let x, y be the two least frequent characters. Take any optimal tree T (full by Lemma 1); it contains two sibling leaves b, c at the deepest level. Assuming $f(x) \le f(b)$ and $f(y) \le f(c)$ (relabel if needed), swap $x \leftrightarrow b$ and $y \leftrightarrow c$. By Lemma 2 the cost does not increase; since T was optimal, the resulting tree is also optimal and now has x, y as sibling leaves at the deepest level.

Lemma 4: Contracting Two Sibling Leaves

If x, y are sibling leaves in T, contract them into one leaf z with $f(z) = f(x) + f(y)$

to obtain T', then

$$\begin{aligned} B(T) &= B(T') - f(z)d(z) + f(x)(d(z)+1) + f(y)(d(z)+1) \\ &= B(T') + f(x) + f(y). \end{aligned}$$

Theorem: Huffman Produces an Optimal Tree (Induction on n)

Base ($n = 2$): the only full tree has two leaves at depth 1; it is optimal.
Inductive step: assume Huffman is optimal for alphabets of size $n-1$. For size n, let H be the Huffman tree and x, y its first merge (the two smallest weights, siblings in H). Contract them to z with $f(z) = f(x) + f(y)$: the contracted tree H' is exactly the tree Huffman builds on the $n-1$-character alphabet $A' = A - \{x, y\} \cup \{z\}$, so by the induction hypothesis H' is optimal for A', and by Lemma 4

$$B(H) = B(H') + f(x) + f(y).$$

By Lemma 3 there is an optimal tree T for A with x, y as siblings; contract them to get T' for A'. Then

$$B(H) = B(H') + f(x) + f(y) \leq B(T') + f(x) + f(y) = B(T).$$

Since T is optimal, H is optimal too. ■

Huffman Caveats

- Huffman is optimal **among prefix codes**, not among all possible codes (arithmetic coding can exploit fractional code lengths).
- Each merge must join the two current smallest weights — joining the smallest with an arbitrary node is not optimal.
- Ties in weight may be broken arbitrarily: different trees can result, all with the same minimum cost.
- Edge case $n = 1$: the single character gets the empty codeword (conventionally one bit may be used).

6.2.8 Application: Optimal Two-Way Merge

Given n sorted lists $L_1, \ldots, L_n$, merge them into one sorted list, merging only two lists at a time; merging lists of sizes p and q costs $p + q$ comparisons and produces a list of size $p + q$. Find the merge pattern minimizing total comparisons.

A merge pattern is a binary tree: leaves are the initial lists (weight = list size), each internal node is a merge, and the total cost equals the **weighted external path length** — so Huffman (always merge the two smallest lists) is optimal.

Example: three lists of sizes $30, 20, 10$:

Pattern	**Computation**	**Total comparisons**
L_1, L_2 first, then L_3	$30 + 20 = 50$, then $50 + 10 = 60$	$50 + 60 = 110$
L_1, L_3 first, then L_2	$30 + 10 = 40$, then $40 + 20 = 60$	$40 + 60 = 100$
L_2, L_3 **first, then** L_1 **(Huffman)**	$20 + 10 = 30$, then $30 + 30 = 60$	$\mathbf{30 + 60 = 90}$

Algorithm 34 Optimal Binary Merge Tree

```
build a min-heap from the n initial list sizes
while heap size ≥ 2 do
    a ← ExtractMin;  b ← ExtractMin
    create an internal node of size a + b with children a, b
    Insert(a + b)
end while
```

Time $O(n \log n)$.

Worked example: initial sizes $2, 3, 5, 7, 9$:

Round	Merge	Heap afterwards
1	$2 + 3 = 5$	$\{5, 5, 7, 9\}$
2	$5 + 5 = 10$	$\{7, 9, 10\}$
3	$7 + 9 = 16$	$\{10, 16\}$
4	$10 + 16 = 26$	$\{26\}$

Total cost (sum of merge sizes): $5+10+16+26 = \mathbf{57}$, which equals the weighted external path length of the merge tree:

$$2 \cdot 3 + 3 \cdot 3 + 5 \cdot 2 + 7 \cdot 2 + 9 \cdot 2 = 6 + 9 + 10 + 14 + 18 = 57.$$

Greedy Algorithms at a Glance

Problem	Greedy choice	Complexity	Correctness idea
Interval scheduling	earliest finish time	$\Theta(n \log n)$	exchange argument
Interval covering	furthest reach covering frontier	$O(n \log n)$	exchange argument
Fractional knapsack	largest value/weight	$\Theta(n \log n)$	exchange argument
Interval partitioning	earliest start, reuse a room	$\Theta(n \log n)$	match depth lower bound
Huffman coding	merge two smallest weights	$O(n \log n)$	exchange + induction
Optimal merge	merge two smallest lists	$O(n \log n)$	same as Huffman

6.3 Backtracking

Idea. Backtracking explores the state space tree of a problem by depth-first search. At every node we test whether the partial solution can still be completed; if it cannot, we prune the whole subtree and backtrack to the nearest ancestor that still has unexplored choices. It is systematic exhaustive search with early rejection, and it is the natural method for constraint-satisfaction problems.

Generic Framework

1. The solution is built incrementally, one component at a time.
2. At each step, try every candidate value for the next component.
3. Reject a candidate immediately if it violates a constraint with the components already chosen (feasibility pruning).
4. If no candidate works, undo the last choice and return to the previous level (backtrack).
5. When all components are chosen, output the solution.

The nodes of the state space tree are partial solutions; leaves are complete candidates. Pruning removes nodes whose subtree contains no solution, which is what makes backtracking much faster than brute force in practice.

6.3.1 The Eight Queens Problem

Place 8 queens on an 8×8 chessboard so that no two queens attack each other: no two share a row, a column or a diagonal.

Representation. Because each column must contain exactly one queen, a solution is a vector $(x_1, \ldots, x_8)$ where x_i is the row of the queen in column i. The constraints become

$$x_i \neq x_j, \qquad |x_i - x_j| \neq |i - j| \quad \text{for all } i \neq j.$$

The state space tree has 8^8 leaves if searched naively; the constraint check reduces the explored nodes to a few thousand.

One solution: $(4, 7, 3, 8, 2, 5, 1, 6)$

			Q				
					Q		
		Q					
							Q
	Q						
				Q			
Q							
						Q	

The eight-queens problem has 92 distinct solutions in total (counting rotations and reflections as different; 12 if they are considered the same). The first solution found by backtracking in lexicographic order is $(1, 5, 8, 6, 3, 7, 2, 4)$.

Algorithm 35 QUEENS(k, n)

```
1: if k > n then
2:     output (x_1, ..., x_n)
3: end if
4: for j = 1 to n do
5:     if no earlier queen i < k has x_i = j or |x_i − j| = k − i then
6:         x_k ← j
7:         QUEENS(k + 1, n)
8:     end if
9: end for
```

6.3.2 The Turnpike Reconstruction Problem

A turnpike has n toll booths at positions $x_1 < x_2 < \cdots < x_n$ on a line, with $x_1 = 0$. A driver records the distance between every pair of booths, giving a multiset D of $n\dfrac{(n-1)}{2}$ distances. The **turnpike reconstruction problem** asks us to recover the positions x_i from D alone.

Algorithm. Maintain a set X of already determined positions and the multiset D of distances not yet accounted for. Initially $X = \{0\}$ and $\max(D)$ is the rightmost position x_n.

1. Let $d = \max(D)$.
2. Try to place a point at $x'_n = d$ (measured from 0); this is feasible only if all distances $|x'_n - x|$ for $x \in X$ are present in D. If so, remove them from D, add x'_n to X, and recurse.
3. Otherwise try $x'_1 = x_n - d$ (measured from the right end) with the same test.
4. If neither placement is feasible, or the recursion fails, restore D and X and backtrack.

Each feasible placement removes $|X|$ distances, so the work per node is $O(n)$ and the search is typically very small. The problem has a unique solution up to reflection when the distances are generic, e.g. $D = \{1, 2, 2, 2, 3, 3, 3, 4, 5, 5, 5, 6, 7, 8, 10\}$ reconstructs $X = \{0, 2, 4, 7, 10\}$.

Why Backtracking Beats Brute Force

The maximum distance d of D must be realised by the outermost pair, so the largest distance is known before any choice is made. Backtracking exploits this by fixing the extremes first; a naive search that guesses all positions would have exponentially many more candidates.

6.3.3 Game Trees and Minimax

Two-player games (tic-tac-toe, chess) are searched with a **game tree**: nodes are positions, edges are moves, and leaves are terminal positions evaluated by a goodness function $f(P)$. For tic-tac-toe a standard choice is

$$f(P) = W_{\text{computer}} - W_{\text{human}},$$

where W counts the number of potential winning lines still open to each side. For example, if both sides have 3 open winning lines, the goodness of the position is $3 - 3 = 0$.

Minimax

The computer (MAX) chooses the move that maximises the backed-up value; the human (MIN) chooses the move that minimises it. Values are propagated from the leaves to the root, alternating MAX and MIN levels. The value at the root is the best result the computer can guarantee against optimal play. The cost is exponential in the depth: a complete tic-tac-toe tree has about $9! = 362{,}880$ leaves, which is still

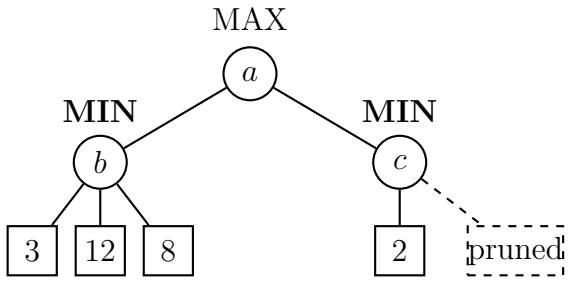
feasible, but chess is not.

6.3.4 Alpha–Beta Pruning

Alpha–beta pruning computes the same minimax value as a full search but skips branches that cannot possibly influence the result.

Alpha and Beta

- α is the best value that MAX has already found on the current path; it is initialised to $-\infty$.
- β is the best value that MIN has already found on the current path; it is initialised to $+\infty$.
- MAX nodes raise α; MIN nodes lower β.
- **Prune** as soon as $\alpha \geq \beta$: the current player already has a better alternative higher up, so this subtree will never be chosen.



$b = 3$, so $\alpha = 3$ at a; at c, $\beta = 2$: since $\alpha \geq \beta$, prune

In the figure, the left subtree makes $b = 3$, so MAX at a sets $\alpha = 3$. When the MIN node c examines its first leaf 2 it sets $\beta = 2$; since $\alpha = 3 \geq \beta = 2$ and MIN can only decrease c's value, MAX would never choose c, so the remaining leaves of c are pruned. With optimal move ordering, alpha–beta pruning examines $O(b^{d/2})$ nodes instead of $O(b^d)$ for branching factor b and depth d — effectively doubling the search depth.

Exercise: Goodness of a Position

Problem. In tic-tac-toe, $f(P) = W_{\text{computer}} - W_{\text{human}}$, where W counts potential wins. If the computer has 3 potential winning lines and the human has 3, what is $f(P)$: (A) -1; (B) 0; (C) 4; (D) 5?
Solution. (B), since $3 - 3 = 0$. ■

Exercise: First Pruned Node

Problem. With alpha–beta pruning on the game tree of the figure, which node is pruned first?
Solution. The remaining children of the MIN node c are pruned. Evaluating b gives 3, so $\alpha = 3$ at the root; the first leaf of c gives $\beta = 2$. Since $\alpha \geq \beta$ at c, MIN's remaining moves cannot raise c above 2 and MAX would never choose c, so the rest of c's subtree is pruned. ■

6.4 Dynamic Programming

Dynamic programming (DP) designs an algorithm for an optimization problem by combining the answers of smaller subproblems. It applies when the problem has

- **optimal substructure**: an optimal solution is built from optimal solutions of subproblems;
- **overlapping subproblems**: a plain recursion solves the same subproblem exponentially many times, so the answer must be stored and reused.

Every DP solution in this chapter is obtained by the same five steps:

1. Define the **subproblem** (what the table entry means, and which indices/parameters identify it).
2. Write the **recurrence**: the value of an entry as a function of strictly smaller entries.
3. Fix the **base cases** and the **order** of evaluation (so that all entries used are already computed).
4. Prove the recurrence by a **cut-and-paste** argument (exchange/optimality).
5. Store the **decision** that achieved the optimum in a second table, to reconstruct an optimal solution.

Top-Down (Memoization) versus Bottom-Up

Memoization keeps the recursion but caches each answer, so each of the (few) distinct subproblems is solved once. Bottom-up DP evaluates the subproblems in increasing size order with nested loops, reusing a table. Bottom-up is usually preferred: it has no recursion overhead, allows space optimization (e.g. $\Theta(1)$ instead of $\Theta(n)$ for Fibonacci), and is safer. Top-down wins when the number of reachable subproblems is much smaller than the table size (a sparse problem space), e.g. coin change with huge denominations.

6.4.1 Warm-up: Stair Climbing and Fibonacci

Stair Climbing

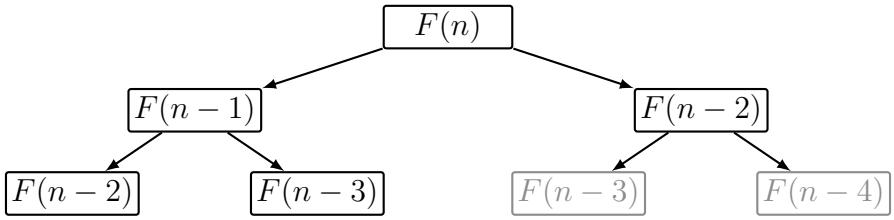
Problem. One step climbs either 1 or 2 stairs. How many different ways are there to climb n stairs?
Solution. Let $F(n)$ be the number of ways to reach stair n. The last step starts at stair $n-1$ (a 1-step) or at stair $n-2$ (a 2-step), and these two cases are disjoint and exhaustive:

$$F(1) = 1, \qquad F(2) = 2, \qquad F(n) = F(n-1) + F(n-2) \quad (n \geq 3).$$

Hence $F(1), F(2), \ldots$ are the Fibonacci numbers. ■

Why the Naive Recursion Is Exponential

A direct recursive implementation solves the same subproblem many times:



duplicated subproblems (grey) are solved again and again ···

The call count satisfies $T(n) = T(n-1)+T(n-2)+1$, hence $T(n) = \Theta\big(F(n)\big) = \Theta(\varphi^n)$ with $\varphi = \dfrac{(1+\sqrt{5})}{2} \approx 1.618$. There are only n distinct subproblems; storing their answers reduces the time to $\Theta(n)$.

Algorithm 36 Memoized (top-down) and bottom-up Fibonacci

```
allocate F[1..n] = [1, 2, 0, 0, ...]
function TD-F(n)
    if F[n] ≠ 0 then
        return F[n]
    end if
    F[n] ← TD-F(n − 1) + TD-F(n − 2)
    return F[n]
end function
function BU-F(n)
    Fp ← 1;   F ← 2
    for i = 3 to n do
        temp ← Fp;   Fp ← F;   F ← Fp + temp
    end for
    return F
end function
```

The bottom-up version needs only two variables, because $F(n)$ depends on the last two values: time $\Theta(n)$, space $\Theta(1)$.

6.4.2 The Rod Cutting Problem

Problem. Given a rod of length n and prices p_i for $i = 1, \ldots, n$, where p_i is the price of a rod of length i, cut the rod into pieces (integer lengths) to maximize total revenue.

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

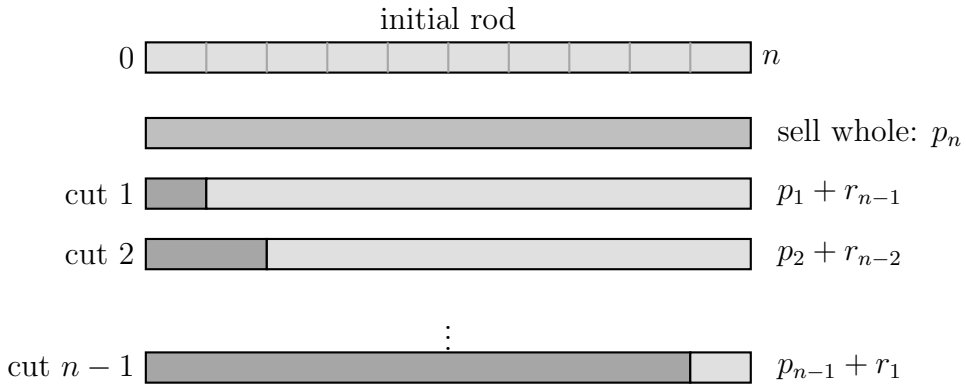
There are 2^{n-1} ways of cutting a rod of size n (a cut or no cut between each pair of consecutive unit lengths), so exhaustive search is hopeless.

Optimal Substructure

Let r_n be the maximum revenue for a rod of length n. Look at the first piece: it has some length $i \in \{1, \ldots, n\}$, earns p_i, and the remainder of length $n - i$ must be cut optimally, i.e. it earns r_{n-i}. Therefore

$$r_n = \max_{1 \le i \le n} \big(p_i + r_{n-i}\big), \qquad r_0 = 0,$$

where $i = n$ is the option "sell the rod in one piece" $(p_n + r_0)$.



Algorithm 37 ROD-CUTTING(p, n)

```
allocate r[0..n], s[1..n]
r[0] ← 0
for j = 1 to n do
    q ← −∞
    for i = 1 to j do
        if q < p[i] + r[j − i] then
            q ← p[i] + r[j − i];   s[j] ← i
        end if
    end for
    r[j] ← q
end for
return r[n]
```

Running time $\Theta(n^2)$ (there are $\binom{n}{2}$ pairs (i, j)), space $\Theta(n)$.

Worked Example: the Full r and s Tables

With the prices above, the optimal revenues and the optimal first-piece lengths are

i	0	1	2	3	4	5	6	7	8	9	10
$p[i]$	0	1	5	8	9	10	17	17	20	24	30
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	–	1	2	3	2	2	6	1	2	3	10

Each entry is the maximum of $p_i + r_{j-i}$ over $i = 1, \ldots, j$:

j	**candidates** $p_i + r_{j-i}$	r_j **= max (argmax)**
1	$p_1 + r_0 = 1$	1 (i=1)
2	$1 + r_1 = 2$, $5 + r_0 = 5$	5 (i=2)
3	$1 + r_2 = 6$, $5 + r_1 = 6$, $8 + r_0 = 8$	8 (i=3)
4	$1 + 8 = 9$, $5 + 5 = 10$, $8 + 1 = 9$, 9	10 (i=2)
5	$1 + 10 = 11$, $5 + 8 = 13$, $8 + 5 = 13$, $9 + 1 = 10$, 10	13 (i=2)
6	$1 + 13 = 14$, $5 + 10 = 15$, $8 + 8 = 16$, $9 + 5 = 14$, $10 + 1 = 11$, 17	17 (i=6)
7	$1 + 17 = 18$, $5 + 13 = 18$, $8 + 10 = 18$, $9 + 8 = 17$, $10 + 5 = 15$, $17 + 1 = 18$, 17	18 (i=1)
8	$1 + 18 = 19$, $5 + 17 = 22$, $8 + 13 = 21$, $9 + 10 = 19$, $10 + 8 = 18$, $17 + 5 = 22$, $17 + 1 = 18$, 20	22 (i=2)
9	$1 + 22 = 23$, $5 + 18 = 23$, $8 + 17 = 25$, $9 + 13 = 22$, $10 + 10 = 20$, $17 + 8 = 25$, $17 + 5 = 22$, $20 + 1 = 21$, 24	25 (i=3)
10	$1 + 25 = 26$, $5 + 22 = 27$, $8 + 18 = 26$, $9 + 17 = 26$, $10 + 13 = 23$, $17 + 10 = 27$, $17 + 8 = 25$, $20 + 5 = 25$, $24 + 1 = 25$, 30	30 (i=10)

Reconstructing a Solution

Table r gives only the optimal value. Storing the index $s[j]$ where the maximum occurred lets us recover the pieces:

1: $j \leftarrow n$
2: **while** $j > 0$ **do**
3: print $s[j]$; $j \leftarrow j - s[j]$
4: **end while**

For $n = 9$: $s[9] = 3$ gives a piece of length 3 and leaves $j = 9 - 3 = 6$; then $s[6] = 6$ gives a piece of length 6 and leaves $j = 0$. The optimal cut is $\{3, 6\}$ with revenue $25 = 8 + 17$. For $n = 7$: $s[7] = 1$, then $s[6] = 6$, giving $\{1, 6\}$ with revenue $18 = 1 + 17$.

6.4.3 Exercise: Maximum Sum of Non-Adjacent Elements

Maximum Sum of Non-Adjacent Elements

Problem. Given positive numbers $a_1, \ldots, a_n$, choose a subset S maximizing $\sum_{a_i \in S} a_i$ under the constraint that a_i and $a_{i\pm1}$ cannot both be chosen.

Greedy fails. For $A = (7, 8, 6, 3)$ greedy takes the largest, 8, deletes its neighbours $7, 6$, then takes 3: value 11. The optimum is $\{7, 6\}$ with value 13.

Solution. Let $A_i = (a_1, \ldots, a_i)$, let S_i be an optimal subset of A_i, let W_i be its sum, and let $b_i = \mathsf{true}$ iff $a_i \in S_i$. For the last element there are two cases:

- $a_i \in S_i$: then $a_{i-1} \notin S_i$, and the rest must be optimal for A_{i-2}, so $W_i = W_{i-2} + a_i$;
- $a_i \notin S_i$: then S_i is optimal for A_{i-1}, so $W_i = W_{i-1}$.

Hence

$$W_i = \max\{W_{i-2} + a_i,\ W_{i-1}\}, \qquad W_1 = a_1, \quad W_2 = \max\{a_1, a_2\}.$$

Algorithm 38 Maximum sum of non-adjacent elements

```
W[1] ← a_1;   b[1] ← true
if a_2 > a_1 then
    W[2] ← a_2;   b[2] ← true
else
    W[2] ← a_1;   b[2] ← false
end if
for i = 3 to n do
    if W[i − 2] + a_i > W[i − 1] then
        W[i] ← W[i − 2] + a_i;   b[i] ← true
    else
        W[i] ← W[i − 1];   b[i] ← false
    end if
end for
```

Time $\Theta(n)$, space $\Theta(n)$ (only $W[i-1], W[i-2]$ are needed for the value).

Worked Example

For $A = (1, 8, 6, 3, 7)$:

i	1	2	3	4	5
a_i	1	8	6	3	7
$W[i]$	1	8	8	11	15
$b[i]$	true	true	false	true	true

Check: $W_3 = \max(W_1 + 6, W_2) = \max(7, 8) = 8$; $W_4 = \max(W_2 + 3, W_3) = \max(11, 8) = 11$; $W_5 = \max(W_3 + 7, W_4) = \max(15, 11) = 15$.
Reconstruction: start at $i = n$; if b_i is true, output a_i and set $i \leftarrow i - 2$; otherwise set $i \leftarrow i - 1$.

$b_5 = \mathsf{true} \Rightarrow$ print 7, $i = 3$; $\quad b_3 = \mathsf{false} \Rightarrow i = 2$; $\quad b_2 = \mathsf{true} \Rightarrow$ print 8, $i = 0$.

Optimal subset $\{7, 8\}$, sum 15. ■

6.4.4 Weighted Interval Scheduling

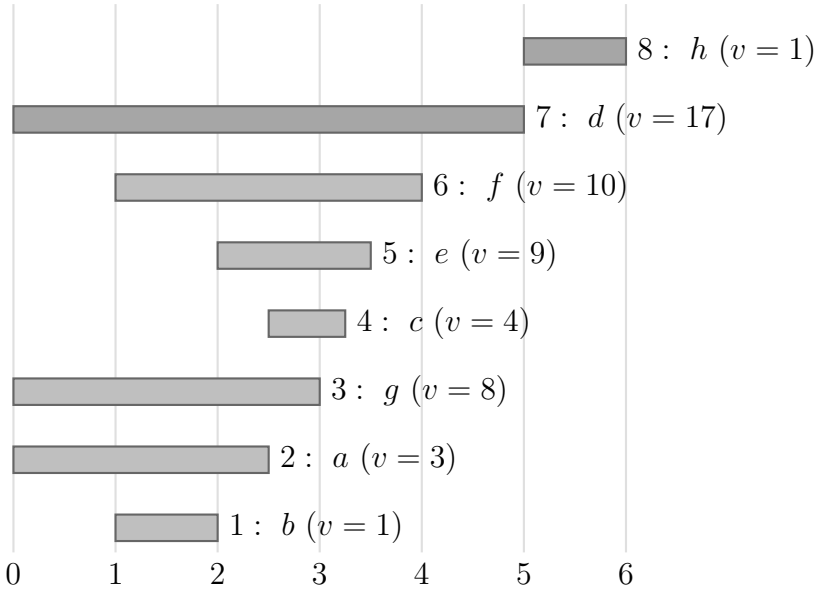
Problem. Job j starts at s_j, finishes at f_j and has weight $v_j > 0$. Two jobs are compatible if they do not overlap (touching is allowed: $f_i \le s_j$). Goal: a maximum-weight subset of mutually compatible jobs.

- If all weights are 1, the greedy algorithm (take jobs in increasing finish time, keep those compatible with the current set) is optimal.
- With arbitrary weights greedy fails badly: a single long job b of weight 999 and a short job a of weight 1 that overlaps b — greedy by finish time may take both, or may take the wrong one, but it cannot know which combination is best.

Notation. Sort the jobs so that $f_1 \le f_2 \le \cdots \le f_n$. Define

$$p(j) = \max\{i < j : \ f_i \le s_j\}$$

(the largest index of a job compatible with j; all jobs i' with $p(j) < i' < j$ are incompatible with j). Since the jobs are sorted by finish time, $p(j)$ is found by a binary search for s_j in $\Theta(\log n)$ time.



jobs sorted by finish time (optimum $\{d, h\}$ in darker grey)

The Recurrence

Let $V(j)$ be the value of an optimal solution using only jobs $1, \dots, j$. Either

- job j is not selected: then $V(j) = V(j-1)$; or
- job j is selected: then all jobs $p(j)+1, \dots, j-1$ are excluded and the rest must be optimal for jobs $1, \dots, p(j)$, so $V(j) = v_j + V(p(j))$.

$$V(j) = \max\big\{v_j + V(p(j)),\ V(j-1)\big\}, \qquad V(0) = 0.$$

The subproblems are again prefixes of the sorted list, so the table is 1-dimensional and filled left to right.

Algorithm 39 WEIGHTED-INTERVAL-SCHEDULING($\{s_j, f_j, v_j\}$)

```
sort all jobs by finish time; compute p(j) for all j by binary search
V[0] ← 0
for j = 1 to n do
    if v_j + V[p(j)] > V[j − 1] then
        V[j] ← v_j + V[p(j)];   keep[j] ← 1
    else
        V[j] ← V[j − 1];   keep[j] ← 0
    end if
end for
j ← n;   while j > 0: if keep[j] = 1 then print j; j ← p(j) else j ← j − 1
```

Running time $\Theta(n \log n)$ (sorting plus one binary search per job, or $\Theta(n)$ after sorting with a two-pointer scan), space $\Theta(n)$.

Worked Example

The eight jobs of the picture, sorted by finish time, give

j	1	2	3	4	5	6	7	8
job	b	a	g	c	e	f	d	h
$[s_j, f_j]$	$[1, 2]$	$[0, 2.5]$	$[0, 3]$	$[2.5, 3.25]$	$[2, 3.5]$	$[1, 4]$	$[0, 5]$	$[5, 6]$
v_j	1	3	8	4	9	10	17	1
$p(j)$	0	0	0	2	1	0	0	7
$V(j)$	1	3	8	8	10	10	17	18
$keep[j]$	1	1	1	0	1	0	1	1

Sample computations: $j = 5$: $\max\{9 + V(1), V(4)\} = \max\{10, 8\} = 10$; $j = 7$: $\max\{17 + V(0), V(6)\} = \max\{17, 10\} = 17$; $j = 8$: $\max\{1 + V(7), V(7)\} = \max\{18, 17\} = 18$.
Reconstruction: $j = 8$, $keep[8] = 1$: print h, $j \leftarrow p(8) = 7$; $keep[7] = 1$: print d, $j \leftarrow p(7) = 0$. Optimal set $\{h, d\}$ with value 18. ■

6.4.5 Exercise: Highway Billboards

Highway Billboards

Problem. Sites $x_1 \le \cdots \le x_n$ on a highway, revenue $r_i > 0$ if a billboard is placed at x_i, and any two billboards must be at least 5 miles apart. Maximize the revenue.
Question 1: a greedy algorithm. Place a billboard at the site with maximum revenue, delete that site and all sites within 5 miles of it, and repeat.
Question 2: is it optimal? No, for the same reason as in the maximum-sum exercise: with sites spaced 4 miles apart the problem is the non-adjacent maximum-sum problem, for which greedy is suboptimal ($7, 8, 6, \overline{3}$: greedy $8 + 3 = 11$, optimum $7 + 6 = 13$).
DP. Sort the sites; let $p(i) = \max\{j < i : \ x_i - x_j > 5\}$ be the last site compatible with x_i, and let $R(i)$ be the optimal revenue using the first i sites. Then

$$R(i) = \max\big\{R(i-1),\ r_i + R(p(i))\big\}, \qquad R(0) = 0,$$

computed for $i = 1, \ldots, n$; the answer is $R(n)$ and the reconstruction is the same as for weighted interval scheduling. Time $\Theta(n \log n)$. ■

6.4.6 Kadane's Algorithm Derived from DP

Maximum Subarray Revisited

For an array $A[1..n]$, let $V[i]$ be the maximum sum of a subarray that ends at $A[i]$ (a "local" optimum). Then either the subarray is just $A[i]$, or it extends an optimal subarray ending at $A[i-1]$:

$$V[1] = A[1], \qquad V[i] = \max\big(A[i],\ A[i] + V[i-1]\big),$$

and the global answer is $\max_i V[i]$ (the optimum subarray ends somewhere).

Full Trace

For $A = (-2,\ 1,\ -3,\ 4,\ -1,\ 2,\ 1,\ -5,\ 4)$:

i	1	2	3	4	5	6	7	8	9
$A[i]$	-2	1	-3	4	-1	2	1	-5	4
$V[i]$	-2	1	-2	4	3	5	6	1	5
$V_{\max}$	-2	1	1	4	4	5	6	6	6

For instance $V[5] = \max(-1, -1 + 4) = 3$ and $V[6] = \max(2, 2 + 3) = 5$; at $i = 8$ the running value drops to 1 but $V_{\max} = 6$ is kept. The optimum subarray is $A[4..7] = (4, -1, 2, 1)$ with sum 6.

Space Optimization and Recovering the Indices

$V[i]$ depends only on $V[i-1]$, so the array is unnecessary: keep a scalar V together with $V_{\max}$. To also return the location, remember i whenever $V_{\max}$ is updated (this is the end), and set $temp \leftarrow i$ whenever V is restarted at $A[i]$ (because $V[i-1] < 0$); the start is the last $temp$ that produced a new maximum. Time $\Theta(n)$, space $\Theta(1)$.

Algorithm 40 KADANE(A, n) with start/end positions

```
1: V ← 0;  Vmax ← −∞;  start ← 1;  end ← 1;  temp ← 1
2: for i = 1 to n do
3:     V ← V + A[i]
4:     if V < A[i] then
5:         V ← A[i];  temp ← i
6:     end if
7:     if V > Vmax then
8:         Vmax ← V;  start ← temp;  end ← i
9:     end if
10: end for
11: return (Vmax, start, end)
```

6.4.7 Minimum Number of Coins

Coin Change

Input: an amount n and denominations $d_1, \ldots, d_k$ (unlimited supply of each). **Output:** the minimum number of coins summing to n.
Greedy is not optimal for arbitrary denominations (e.g. $\{1, 3, 4\}$ and $n = 6$: greedy $4 + 1 + 1$, optimum $3 + 3$).
DP. Let $C[p]$ be the minimum number of coins for amount p. If the last coin used has denomination d_i, the rest must be optimal for $p - d_i$:

$$C[0] = 0, \qquad C[p] = 1 + \min_{1 \le i \le k,\ d_i \le p} \{C[p - d_i]\}.$$

Store in $S[p]$ the denomination of that last coin, then print the solution by repeatedly printing $S[p]$ and setting $p \leftarrow p - S[p]$.

Algorithm 41 DP-Coin(n)

```
1: C[0] ← 0
2: for p = 1 to n do
3:     min ← ∞
4:     for i = 1 to k do
5:         if p ≥ d_i and 1 + C[p − d_i] < min then
6:             min ← C[p − d_i] + 1;   coin ← d_i
7:         end if
8:     end for
9:     C[p] ← min;   S[p] ← coin
10: end for
11: while n > 0: print S[n];   n ← n − S[n]
```

Time $\Theta(nk)$, space $\Theta(n)$.

When the Denominations Are Huge

If $n = 19999$ and the denominations are $1000, 5000, 9999, 10000, 50000$, the bottom-up table has 20000 entries although only a few amounts are reachable. Then top-down DP with memoization is better: keep hash tables C, S and call MakeChange(p), which returns the cached $C[p]$ if present, and otherwise tries all $d_i \le p$ recursively. The cost becomes $\Theta\big(k \cdot \#\{\text{reachable amounts}\}\big)$. First scale all values down by their GCD to shrink n.

6.4.8 The 0/1 Knapsack Problem

Input: items $1, \dots, n$ with weights w_i and values $v_i > 0$, and a knapsack of capacity W. **Goal:** choose $x_i \in \{0, 1\}$ with $\sum x_i w_i \le W$ maximizing $\sum x_i v_i$. Greedy (by value, by weight, or by value/weight) is not optimal.

A Wrong One-Dimensional Definition

Letting $V[w]$ be the best value for capacity w and writing $V[w] = \max_i (v_i + V[w - w_i])$ (as for rod cutting) is **wrong**: nothing prevents the same item from being chosen twice, so the "solution" may be illegal. The state must also say which items are still available.

Correct Two-Dimensional Recurrence

Let $V[i, w]$ be the maximum value obtainable with capacity w using only items $1, \dots, i$. For item i there are two cases:

$$V[i, w] = \max\big\{V[i-1, w],\ \ v_i + V[i-1,\ w - w_i]\big\} \quad (w_i \le w),$$

and $V[i, w] = V[i-1, w]$ if $w_i > w$; base cases $V[0, w] = V[i, 0] = 0$. The answer is $V[n, W]$. The table is filled row by row (i increasing) and, inside a row, w increasing, because entry (i, w) needs row $i - 1$ only.

Algorithm 42 0/1-KNAPSACK(n, W, v, w)

```
allocate V[0..n, 0..W] and keep[0..n, 0..W], all 0
for i = 1 to n do
    for w = 1 to W do
        if w_i ≤ w and v_i + V[i−1, w−w_i] > V[i−1, w] then
            V[i, w] ← v_i + V[i−1, w−w_i];   keep[i, w] ← 1
        else
            V[i, w] ← V[i−1, w];   keep[i, w] ← 0
        end if
    end for
end for
K ← W;   for i = n downto 1: if keep[i, K] = 1 then print i; K ← K − w_i
```

Time and space $\Theta(nW)$; the space can be reduced to $\Theta(n + W)$ (keep one row plus the *keep* bits, or two rows and recompute).

Worked Example (the Full Table)

Items: $v = (10, 40, 30, 50)$, $w = (5, 4, 6, 3)$, $W = 10$.

$V[i,w]$	w=0	1	2	3	4	5	6	7	8	9	10
$i = 0$	0	0	0	0	0	0	0	0	0	0	0
$i = 1$	0	0	0	0	0	10	10	10	10	10	10
$i = 2$	0	0	0	0	40	40	40	40	40	50	50
$i = 3$	0	0	0	0	40	40	40	40	40	50	70
$i = 4$	0	0	0	50	50	50	50	90	90	90	90

Sample entries: $V[2,4] = \max(0,\ 40+V[1,0]) = 40$; $V[2,9] = \max(10,\ 40+V[1,5]) = \max(10, 50) = 50$; $V[4,3] = \max(0,\ 50 + V[3,0]) = 50$; $V[4,7] = \max(50,\ 50 + V[3,4]) = \max(50, 90) = 90$.
Reconstruction: $(4, 10)$: $keep = 1$ (since $90 > 70$), take item 4, $K = 7$; $(3, 7)$: $keep = 0$ (since $70 < 90$... compare $V[2,7] = 40$ with $30 + V[2,1] = 30$, so $keep = 0$), skip item 3; $(2, 7)$: $keep = 1$ ($40 + V[1,3] = 40 > 0$), take item 2, $K = 3$; $(1, 3)$: $keep = 0$. Items $\{2, 4\}$, value 90, weight $7 \le 10$. ■

Exercise: Two Knapsacks

Problem. Two knapsacks, both of capacity W. Give an $O(nW^2)$ DP for the maximum total value carried.
Solution. Let $V[i, w_1, w_2]$ be the best value using items $1, \ldots, i$ with capacities w_1, w_2 left. For item i there are three cases (skip it, put it in knapsack 1, put it in knapsack 2):

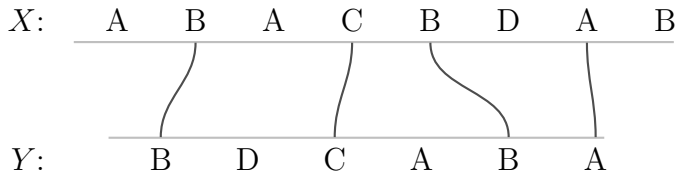
$$V[i, w_1, w_2] = \max\big\{V[i-1, w_1, w_2],\ v_i + V[i-1, w_1 - w_i, w_2],\ v_i + V[i-1, w_1, w_2 - w_i]\big\},$$

with the terms omitted when the corresponding capacity is insufficient, and base cases $V[0, w_1, w_2] = 0$. Fill with i increasing, then w_1 increasing, then w_2 increasing; the answer is $V[n, W, W]$. Time and space $\Theta(nW^2)$. ■

6.4.9 Longest Common Subsequence

Problem. Given $X = (x_1, \ldots, x_m)$ and $Y = (y_1, \ldots, y_n)$, a common subsequence is $Z = (z_1, \ldots, z_k)$ with $z_p = x_{i_p} = y_{j_p}$ for $i_1 < \cdots < i_k$ and $j_1 < \cdots < j_k$. Find a longest one (LCS).

Equivalently, match characters of X with equal characters of Y so that **no two matched pairs cross**:



$Z = (B, C, B, A)$ is an LCS of length 4

Recurrence

Let $c[i, j]$ be the length of an LCS of $X[1..i]$ and $Y[1..j]$.

- If $x_i = y_j$, then some optimal solution matches x_i with y_j (exchange argument: replace whatever x_i/y_j were matched to), so $c[i, j] = c[i-1, j-1] + 1$.
- If $x_i \neq y_j$, then at least one of them is unmatched, so $c[i, j] = \max\{c[i-1, j],\ c[i, j-1]\}$.

$$c[i,j] = \begin{cases} 0, & i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1, & i, j > 0,\ x_i = y_j, \\ \max\{c[i-1, j],\ c[i, j-1]\}, & i, j > 0,\ x_i \neq y_j. \end{cases}$$

Worked Example: the c Table

$X = A\,B\,A\,C\,B\,D\,A\,B$, $Y = B\,D\,C\,A\,B\,A$:

$c[i,j]$	j=0	1	2	3	4	5	6
$i = 0$	0	0	0	0	0	0	0
$i = 1$ (A)	0	0	0	0	1	1	1
$i = 2$ (B)	0	1	1	1	1	2	2
$i = 3$ (A)	0	1	1	1	2	2	3
$i = 4$ (C)	0	1	1	2	2	2	3
$i = 5$ (B)	0	1	1	2	2	3	3
$i = 6$ (D)	0	1	2	2	2	3	3
$i = 7$ (A)	0	1	2	2	3	3	4
$i = 8$ (B)	0	1	2	2	3	4	4

For example $c[2,1]$: $x_2 = y_1 = B$, so $c[2,1] = c[1,0]+1 = 1$; $c[2,6]$: $B \neq A$, so $c[2,6] = \max(c[1,6], c[2,5]) = \max(1,2) = 2$; $c[8,6]$: $B \neq A$, $c[8,6] = \max(c[7,6], c[8,5]) = \max(4,4) = 4$. The LCS length is $c[m,n] = 4$, and the entry $c[7,6] = c[6,5]+1 = 3+1$ reveals the match $x_7 = y_6 = A$.

Algorithm 43 LCS-Length and Print-LCS

```
allocate c[0..m, 0..n], b[0..m, 0..n], all 0
for i = 1 to m do
    for j = 1 to n do
        if x_i = y_j then
            c[i,j] ← c[i−1, j−1] + 1;   b[i,j] ←diag
        else if c[i−1, j] ≥ c[i, j−1] then
            c[i,j] ← c[i−1, j];   b[i,j] ←up
        else
            c[i,j] ← c[i, j−1];   b[i,j] ←left
        end if
    end for
end for
procedure Print-LCS(b, i, j)
    if i = 0 or j = 0 then
        return
    end if
    if b[i,j] =diag then
        Print-LCS(b, i−1, j−1); print x_i
    else if b[i,j] =up then
        Print-LCS(b, i−1, j)
    else
        Print-LCS(b, i, j−1)
    end if
end procedure
```

Time $\Theta(mn)$, space $\Theta(mn)$; only $\Theta(\min(m,n))$ space is needed if the optimal length alone is required (keep two rows, or one row plus one diagonal variable).

Longest Common Substring

A substring must be contiguous. Define $d[i,j]$ as the length of the longest common substring of $X[1..i]$, $Y[1..j]$ that ends at x_i and y_j. Then

$$d[i,j] = d[i-1,j-1]+1 \ \text{ if } x_i = y_j, \qquad d[i,j] = 0 \ \text{ otherwise,}$$

and the answer is $\max_{i,j} d[i,j]$ — the problem is changed (it is no longer a prefix problem), which is allowed because every common substring ends at some pair (i,j). Time $\Theta(mn)$. Example: X = DEADBEEF, Y = EATBEEF $\Rightarrow$ answer BEEF. The brute-force algorithm takes $O(n^4)$.

Exercise: Edit Distance

Definition. $edit(s,t)$ is the minimum number of insertions, deletions and substitutions that turn s into t. For s = abode and t = blog:

$$\text{abode} \xrightarrow{\text{delete } a} \text{bode} \xrightarrow{\text{insert } l} \text{blode} \xrightarrow{\text{delete } d} \text{bloe} \xrightarrow{e\to g} \text{blog},$$

so $edit(s,t) \le 4$; three operations cannot suffice because the lengths differ by one and the strings share only the letters b, o.

DP. Let $edit[i,j]$ be the distance between $s[1..i]$ and $t[1..j]$. Then

$$edit[0,j] = j, \qquad edit[i,0] = i,$$

$$edit[i,j] = \min \begin{cases} 1 + edit[i,j-1] & \text{(insert)},\\ 1 + edit[i-1,j] & \text{(delete)},\\ \mathbf{I}[s_i \neq t_j] + edit[i-1,j-1] & \text{(substitute/keep)}, \end{cases}$$

where $\mathbf{I}[\phi] = 1$ if ϕ is true and 0 otherwise. Time and space $\Theta(mn)$; $\Theta(\min(m,n))$

space if only the distance is wanted. ■

6.4.10 Exercise: Choosing m Numbers with a Sliding-Window Constraint

At Least One in Every k Consecutive Positions

Problem. Given $A[1..n]$, choose exactly m numbers maximizing their sum, subject to: every k consecutive positions contain at least one chosen position ($k, m \le n$).
DP. Let $dp[i, j]$ be the maximum sum of i chosen numbers from $A[1..j]$ with $A[j]$ chosen. The previous chosen position is some $t \in [\max(0, j-k),\ j-1]$, hence

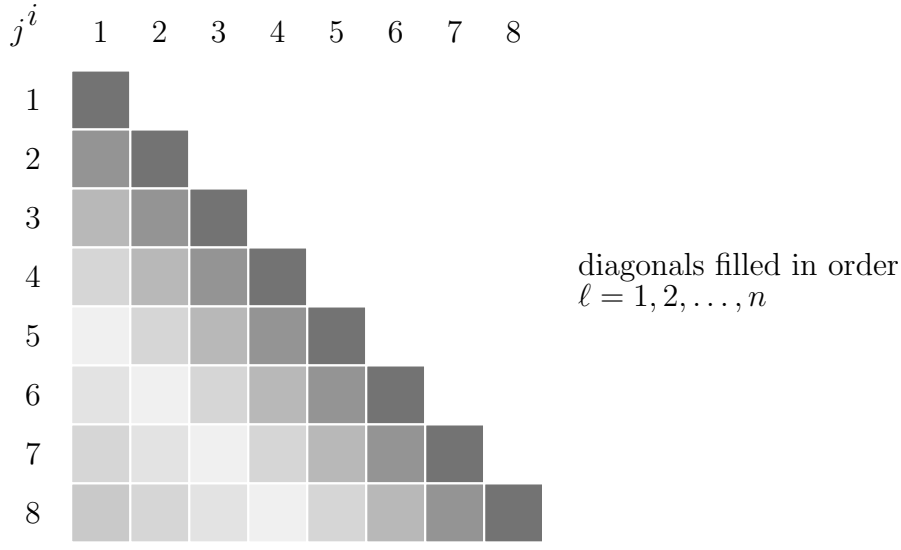
$$dp[i, j] = A[j] + \max_{\max(0,j-k) \le t \le j-1} dp[i-1, t],$$

with $dp[0, 0] = 0$ and all other entries $-\infty$. The answer is $\max_{n-k+1 \le t \le n} dp[m, t]$ (the last chosen position must still "cover" the final window).
Complexity. Directly the maximum is scanned for each (i, j), giving $O(nmk)$. But when going from layer $i-1$ to layer i the maximum is over a sliding window of size k, which a monotonic deque maintains in $O(1)$ amortized time per entry: $O(nm)$. ■

6.4.11 DP over Intervals: Longest Palindromic Substring

Interval DP

The subproblems are now contiguous intervals $[i, j]$, not prefixes. An entry (i, j) depends on intervals strictly shorter than $j - i + 1$ (typically $[i+1, j-1]$, $[i+1, j]$, $[i, j-1]$). Therefore the table is filled **by increasing length**: first the main diagonal (length 1, base cases), then the diagonal of length 2, and so on; the final answer sits at the top-right corner $(1, n)$.



Longest Palindromic Substring

Problem. A palindrome reads the same forward and backward (radar, level, racecar, madam). Given $X = x_1 \dots x_n$, find the longest palindromic substring (contiguous). Example: X = ACCABA has palindromic substrings CC, ACCA, ABA, and the longest is ACCA. Brute force takes $O(n^3)$.

DP. Let $p[i, j] = \mathsf{true}$ iff $X[i..j]$ is a palindrome.
- base cases: $p[i, i] = \mathsf{true}$ for all i; $p[i, i+1] = \mathsf{true}$ iff $x_i = x_{i+1}$;
- recurrence: $p[i, j] = \mathsf{true}$ iff $x_i = x_j$ and $p[i+1, j-1] = \mathsf{true}$.

The answer is the largest length $\ell = j - i + 1$ with $p[i, j] = \mathsf{true}$.

Worked Example: the Completed Table for $X =$ BABBCCCB

Rows are i, columns are j; only $j \geq i$ entries exist.

$p[i,j]$	j=1	2	3	4	5	6	7	8
$i=1$	T	F	T	F	F	F	F	F
$i=2$		T	F	F	F	F	F	F
$i=3$			T	T	F	F	F	F
$i=4$				T	F	F	F	T
$i=5$					T	T	T	F
$i=6$						T	T	F
$i=7$							T	F
$i=8$								T

Reading off the table: $p[1,3] = \mathsf{T}$ because $x_1 = x_3 = B$ and $p[2,2] = \mathsf{T}$ (substring BAB); $p[4,8] = \mathsf{T}$ because $x_4 = x_8 = B$ and $p[5,7] = \mathsf{T}$ (substring BCCCB, length 5, the optimum). Note $p[1,8] = \mathsf{F}$ even though $x_1 = x_8$, because $p[2,7] = \mathsf{F}$.

Algorithm 44 LONGEST-PALINDROMIC-SUBSTRING(X, n)

```
1: max ← 1
2: for i = 1 to n − 1 do
3:     p[i, i] ← true
4:     if x_i = x_{i+1} then
5:         p[i, i + 1] ← true;   max ← 2
6:     else
7:         p[i, i + 1] ← false
8:     end if
9: end for
10: p[n, n] ← true
11: for ℓ = 3 to n do
12:     for i = 1 to n − ℓ + 1 do
13:         j ← i + ℓ − 1
14:         if x_i = x_j and p[i + 1, j − 1] then
15:             p[i, j] ← true;   max ← ℓ
16:         else
17:             p[i, j] ← false
18:         end if
19:     end for
20: end for
21: return max
```

Time $O(n^2)$ (there are $n^2/2$ intervals, each in $O(1)$), space $O(n^2)$, reducible to $O(n)$ by keeping only two diagonals.

6.4.12 Longest Palindromic Subsequence

A Subsequence Need Not Be Contiguous

Let $dp[i, j]$ be the length of the longest palindromic subsequence of $X[i..j]$. Base cases: $dp[i, i] = 1$; $dp[i, i+1] = 2$ if $x_i = x_{i+1}$ and 1 otherwise. For $\ell \geq 3$:

$$dp[i, j] = \begin{cases} dp[i+1, j-1] + 2, & x_i = x_j, \\ \max\{dp[i+1, j],\ dp[i, j-1]\}, & x_i \neq x_j. \end{cases}$$

Store in $sol[i, j] \in \{1, 2, 3\}$ which case was used (1: take both ends, 2: drop x_i, 3: drop x_j).

Algorithm 45 LONGEST-PALINDROMIC-SUBSEQUENCE and PRINT-SOL

```
1: for i = 1 to n − 1 do
2:     dp[i, i] ← 1
3:     if x_i = x_{i+1} then
4:         dp[i, i+1] ← 2;   sol[i, i+1] ← 1
5:     else
6:         dp[i, i+1] ← 1;   sol[i, i+1] ← 2
7:     end if
8: end for
9: dp[n, n] ← 1
10: for ℓ = 3 to n do
11:     for i = 1 to n − ℓ + 1 do
12:         j ← i + ℓ − 1
13:         if x_i = x_j then
14:             dp[i, j] ← dp[i+1, j−1] + 2;   sol[i, j] ← 1
15:         else if dp[i+1, j] > dp[i, j−1] then
16:             dp[i, j] ← dp[i+1, j];   sol[i, j] ← 2
17:         else
18:             dp[i, j] ← dp[i, j−1];   sol[i, j] ← 3
19:         end if
20:     end for
21: end for
22: procedure PRINT-SOL(i, j)
23:     if i > j then
24:         return
25:     end if
26:     if i = j then
27:         print x_i; return
28:     end if
29:     if sol[i, j] = 1 then
30:         print x_i; PRINT-SOL(i+1, j−1); print x_j
31:     else if sol[i, j] = 2 then
32:         PRINT-SOL(i+1, j)
33:     else
34:         PRINT-SOL(i, j−1)
35:     end if
36: end procedure
```

Time and space $O(n^2)$.

Can We Just Use LCS with the Reversed String?

- **Substring version: no.** X = abcdba, rev(X) = abdcba: the longest palindromic substring of X is a single letter (length 1), while the longest common substring of X and rev(X) is ab (or ba), length 2 — the common substring need not be a

palindrome.
- **Subsequence version: yes, but only the length.** $X = \text{abcab}$, $\text{rev}(X) = \text{bacba}$: the longest palindromic subsequence has length 3 (aba, bcb), and the LCS of X and $\text{rev}(X)$ also has length 3 (aba, bcb, acb, ...). The values of the LCS need not be palindromes, but the lengths agree (the proof is not immediate).

6.4.13 Matrix-Chain Multiplication

Cost model. The product of $A_{p\times q}$ and $B_{q\times r}$ is a $p \times r$ matrix C with $c_{ij} = \sum_{k=1}^{q} a_{ik} b_{kj}$; computing it takes $p\,q\,r$ scalar multiplications.

$$\begin{array}{ccccc} A & & B & & C \\ 5\times 4 & \times & 4\times 8 & = & 5\times 8 \end{array}$$

$5 \cdot 4 \cdot 8 = 160$ operations

Problem. Given a chain $A_1 A_2 \cdots A_n$ where A_i has dimensions $p_{i-1} \times p_i$, fully parenthesize the product so that the number of scalar multiplications is minimum. Different parenthesizations give very different costs.

Three Matrices: $A_{10\times 100}$, $B_{100\times 5}$, $C_{5\times 50}$

Parenthesization	Cost of the two products	Total
$A(BC)$	BC: $100 \cdot 5 \cdot 50 = 25{,}000$; then $A \cdot (BC)$: $10 \cdot 100 \cdot 50 = 50{,}000$	75,000
$(AB)C$	AB: $10 \cdot 100 \cdot 5 = 5{,}000$; then $(AB) \cdot C$: $10 \cdot 5 \cdot 50 = 2{,}500$	**7,500**

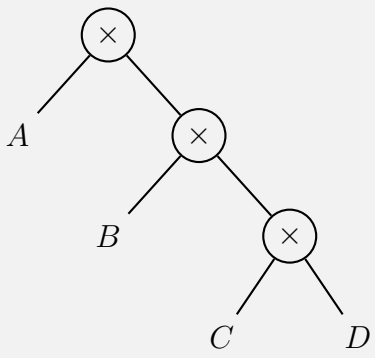
The same three matrices cost ten times more when the parenthesis is put on the wrong side.

Four Matrices: $A_{5\times 5}$, $B_{5\times 4}$, $C_{4\times 8}$, $D_{8\times 2}$

There are $C_3 = \frac{1}{4}\binom{6}{3} = 5$ parenthesizations (C_{n-1} is the $(n-1)$-st Catalan number):

#	Parenthesization and cost of each product	Total
1	$(A(B(CD)))$: $CD = 4 \cdot 8 \cdot 2 = 64$, $B(CD) = 5 \cdot 4 \cdot 2 = 40$, $A(\cdot) = 5 \cdot 5 \cdot 2 = 50$	**154**
2	$(A((BC)D))$: $BC = 160$, $(BC)D = 5 \cdot 8 \cdot 2 = 80$, $A(\cdot) = 50$	290
3	$((AB)(CD))$: $AB = 100$, $CD = 64$, $(\cdot)(\cdot) = 5 \cdot 4 \cdot 2 = 40$	204
4	$((A(BC))D)$: $BC = 160$, $A(BC) = 5 \cdot 5 \cdot 8 = 200$, $\cdot D = 5 \cdot 8 \cdot 2 = 80$	440
5	$(((AB)C)D)$: $AB = 100$, $(AB)C = 5 \cdot 4 \cdot 8 = 160$, $\cdot D = 80$	340

The optimum is $(A(B(CD)))$ with 154 multiplications; a flat binary tree picture of it is



internal nodes are multiplications; leaves are the matrices

Optimal Structure and Recurrence

Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i\cdots j} = A_i \cdots A_j$ (which has dimensions $p_{i-1} \times p_j$). An optimal parenthesization splits $A_{i\cdots j} = A_{i\cdots k} \cdot A_{k+1\cdots j}$ at some k with $i \le k < j$, and both factors must themselves be computed optimally (otherwise a cheaper factor would lower the total). Hence

$$m[i, j] = \begin{cases} 0, & i = j, \\ \min\limits_{i \le k < j} \big\{ m[i, k] + m[k+1, j] + p_{i-1} p_k p_j \big\}, & i < j, \end{cases}$$

where $p_{i-1} p_k p_j$ is the cost of the final multiplication of a $p_{i-1} \times p_k$ matrix by a $p_k \times p_j$ matrix.

Algorithm 46 MATRIX-CHAIN-ORDER(p, n)

```
for i = 1 to n do
    m[i, i] ← 0
end for
for ℓ = 2 to n do
    for i = 1 to n − ℓ + 1 do
        j ← i + ℓ − 1;   m[i, j] ← ∞
        for k = i to j − 1 do
            q ← m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
            if q < m[i, j] then
                m[i, j] ← q;   s[i, j] ← k
            end if
        end for
    end for
end for
return m[ ], s[ ]
```

Time $O(n^3)$ (there are $O(n^2)$ intervals, each trying $O(n)$ splits), space $O(n^2)$. The best known algorithm runs in $O(n \log n)$ (Hu and Shing).

Worked Example: $p = (5, 4, 6, 2, 7, 4)$

So $A_1 : 5 \times 4$, $A_2 : 4 \times 6$, $A_3 : 6 \times 2$, $A_4 : 2 \times 7$, $A_5 : 7 \times 4$.

$m[i,j]$	j=1	2	3	4	5
$i=1$	0	120	88	158	184
$i=2$	–	0	48	104	136
$i=3$	–	–	0	84	104
$i=4$	–	–	–	0	56
$i=5$	–	–	–	–	0

$s[i,j]$	j=1	2	3	4	5
$i=1$	–	1	1	3	3
$i=2$	–	–	2	3	3
$i=3$	–	–	–	3	3
$i=4$	–	–	–	–	4
$i=5$	–	–	–	–	–

Length-2 chains: $m[1,2] = 0+0+5\cdot 4\cdot 6 = 120$; $m[2,3] = 4\cdot 6\cdot 2 = 48$; $m[3,4] = 6\cdot 2\cdot 7 = 84$; $m[4,5] = 2\cdot 7\cdot 4 = 56$.
Length-3 chains: $m[1,3] = \min\{m[1,1]+m[2,3]+5\cdot 4\cdot 2,\ m[1,2]+m[3,3]+5\cdot 6\cdot 2\} = \min\{88,\ 180\} = 88$ $(k=1)$; $m[2,4] = \min\{0+84+4\cdot 6\cdot 7,\ 48+0+4\cdot 2\cdot 7\} = \min\{252, 104\} = 104$ $(k=3)$; $m[3,5] = \min\{0+56+6\cdot 2\cdot 4,\ 84+0+6\cdot 7\cdot 4\} = \min\{104, 252\} = 104$ $(k=3)$.
Length-4 chains: $m[1,4] = \min\{0+104+140,\ 120+84+210,\ 88+0+70\} = \min\{244, 414, 158\} = 158$ $(k=3)$; $m[2,5] = \min\{0+104+96,\ 48+56+32,\ 104+0+112\} = \min\{200, 136, 216\} = 136$ $(k=3)$.
Length-5: $m[1,5] = \min\{0+136+80,\ 120+104+120,\ 88+56+40,\ 158+0+140\} = \min\{216, 344, 184, 298\} = 184$ $(k=3)$.
Reconstruction: $s[1,5]=3$, so the last multiplication is $(A_1A_2A_3)(A_4A_5)$; $s[1,3]=1$ gives $(A_1)(A_2A_3)$; $s[2,3]=2$ gives (A_2A_3); $s[4,5]=4$ gives (A_4A_5). Hence

$$\big((A_1(A_2A_3))(A_4A_5)\big)$$

is optimal with cost 184. ■

6.4.14 Optimal Binary Search Trees

A search in a BST with n keys takes $\Theta(\log n)$ in the worst case, but if we know the probabilities $f(a_i)$ of searching each key, an unbalanced tree can minimize the expected search cost.

The Optimal BST Problem

Given keys $a_1 < a_2 < \cdots < a_n$ with positive weights $f(a_1), \ldots, f(a_n)$, find a BST T on these keys minimizing

$$B(T) = \sum_{i=1}^{n} f(a_i)\big(d(a_i)+1\big),$$

where $d(a_i)$ is the depth of a_i (so $d+1$ counts the comparisons, or the number of nodes on the root-to-a_i path). If f gives search probabilities, a minimum-B tree is the minimum expected-cost tree.

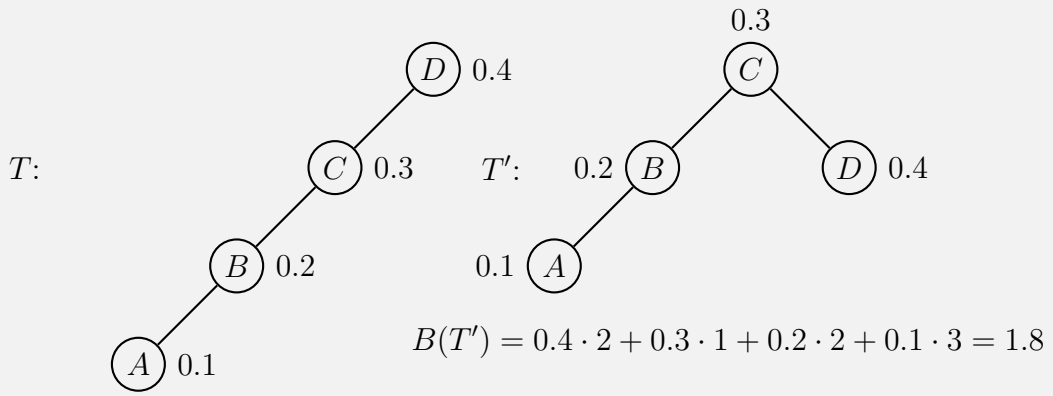
Two Differences from Huffman Coding

- The tree must be a **BST**: in-order traversal gives the sorted keys. A Huffman tree imposes no order on its leaves.
- Keys appear as **both internal and leaf nodes**; in Huffman coding characters appear only at leaves.

Consequently Huffman's greedy algorithm cannot be used.

The "Heaviest Key as Root" Greedy Fails

Keys $A < B < C < D$ with weights $f(A) = 0.1$, $f(B) = 0.2$, $f(C) = 0.3$, $f(D) = 0.4$.



$$B(T') = 0.4 \cdot 2 + 0.3 \cdot 1 + 0.2 \cdot 2 + 0.1 \cdot 3 = 1.8$$

$$B(T) = 0.4 \cdot 1 + 0.3 \cdot 2 + 0.2 \cdot 3 + 0.1 \cdot 4 = 2.0$$

The greedy tree T has cost 2.0, but T' has cost 1.8: the heaviest key need not be the root.

Recurrence

Let $e[i, j]$ be the minimum cost of any BST on $a_i, \ldots, a_j$, and let $w[i, j] = f(a_i) + \cdots + f(a_j)$ (all $w[i, j]$ are precomputed in $O(n^2)$ time). Suppose the root of the optimal tree $T_{i,j}$ is a_k; then its left subtree must be optimal for $[i, k-1]$ and its right subtree optimal for $[k+1, j]$ (cut-and-paste). Every node of both subtrees is one level deeper inside $T_{i,j}$, so

$$B(T_{i,j}) = e[i, k-1] + e[k+1, j] + \underbrace{w[i, k-1] + f(a_k) + w[k+1, j]}_{w[i,j]}.$$

Trying every possible root,

$$e[i, j] = \min_{i \le k \le j} \big\{ e[i, k-1] + e[k+1, j] + w[i, j] \big\},$$

$$e[i, j] = 0 \text{ for } i > j, \qquad e[i, i] = f(a_i).$$

Order: increasing interval length $j - i + 1$; the answer is $e[1, n]$.

Algorithm 47 OPTIMAL-BST(f, n)

```
allocate e[1..n, 1..n], w[1..n, 1..n], root[1..n, 1..n], all 0
for i = 1 to n do
    w[i, i] ← f(a_i)
    for j = i + 1 to n do
        w[i, j] ← w[i, j − 1] + f(a_j)
    end for
end for
for ℓ = 1 to n do
    for i = 1 to n − ℓ + 1 do
        j ← i + ℓ − 1;   e[i, j] ← ∞
        for k = i to j do
            t ← e[i, k − 1] + e[k + 1, j] + w[i, j]
            if t < e[i, j] then
                e[i, j] ← t;   root[i, j] ← k
            end if
        end for
    end for
end for
return CONSTRUCT-BST(root, 1, n)
```

```
procedure CONSTRUCT-BST(root, i, j)
    if i > j then
        return nil
    end if
    create node z;   z.key ← a[root[i, j]]
    z.left ← CONSTRUCT-BST(root, i, root[i, j] − 1)
    z.right ← CONSTRUCT-BST(root, root[i, j] + 1, j);   return z
end procedure
```

Time $O(n^3)$ for the table (splits $\times$ intervals) and $O(n)$ for the construction; space $O(n^2)$. The Knuth–Yao speedup reduces the table to $O(n^2)$.

Worked Example: Weights $f = (5, 20, 30, 50, 10, 20, 10)$ on $A, \ldots, G$

$e[i,j]$	j=1	2	3	4	5	6	7
$i = 1$ (A)	5	30	85	185	210	260	290
$i = 2$ (B)	–	20	70	170	190	240	270
$i = 3$ (C)	–	–	30	110	130	180	210
$i = 4$ (D)	–	–	–	50	70	120	150
$i = 5$ (E)	–	–	–	–	10	40	60
$i = 6$ (F)	–	–	–	–	–	20	40
$i = 7$ (G)	–	–	–	–	–	–	10

$root[i,j]$	j=1	2	3	4	5	6	7
$i = 1$	1	2	3	3	4	4	4
$i = 2$	–	2	3	3	4	4	4
$i = 3$	–	–	3	4	4	4	4
$i = 4$	–	–	–	4	4	4	4
$i = 5$	–	–	–	–	5	6	6
$i = 6$	–	–	–	–	–	6	6
$i = 7$	–	–	–	–	–	–	7

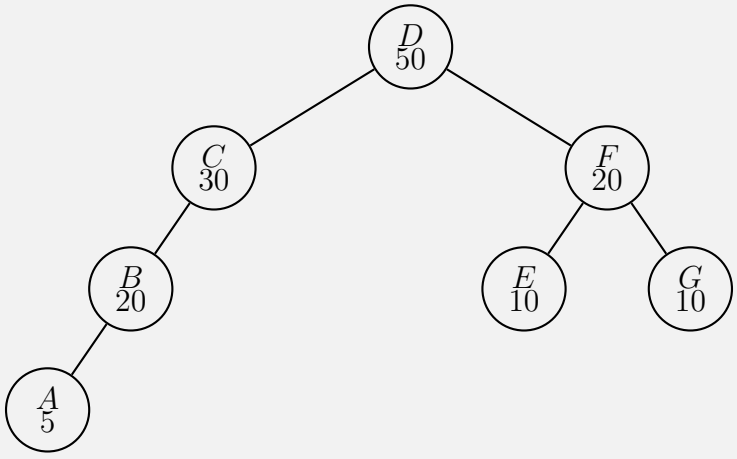
Selected computations ($w[i,j]$ is the sum of the weights on $[i,j]$):

$$e[1,2] = \min\{0+20+25,\ 5+0+25\} = \min\{45,30\} = 30 \quad (k=2),$$

$$e[1,4] = \min\{0+170+105,\ 5+110+105,\ 30+50+105,\ 85+0+105\}$$
$$= \min\{275,220,185,190\} = 185 \quad (k=3),$$

$$e[1,7] = \min\{415,\ 360,\ 325,\ \mathbf{290},\ 370,\ 365,\ 405\} = 290 \quad (k=4).$$

The optimal tree: root $root[1,7] = 4$ (D); left part $[1,3]$ has root $root[1,3] = 3$ (C) with left part $[1,2]$ rooted at B (left child A); right part $[5,7]$ has root $root[5,7] = 6$ (F) with left child E and right child G:



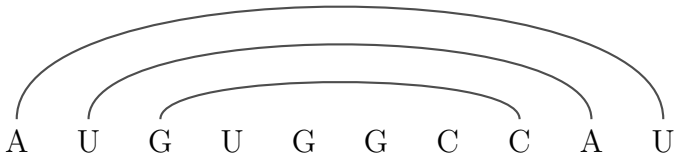
Cost check: $50{\cdot}1+30{\cdot}2+20{\cdot}2+20{\cdot}3+10{\cdot}3+10{\cdot}3+5{\cdot}4 = 50+60+40+60+30+30+20 = 290$, matching $e[1,7]$. ■

6.4.15 RNA Secondary Structure

Problem. An RNA molecule is a sequence $B = b_1 b_2 \cdots b_n$ over $\{A, U, C, G\}$. A secondary structure is a set S of pairs (b_i, b_j) such that

(1) Watson–Crick: S is a matching and every pair is A–U, U–A, C–G or G–C;
(2) No sharp turns: if $(b_i, b_j) \in S$ then $i < j - 4$ (at least 4 bases between the two ends);
(3) Non-crossing: there are no $(b_i, b_j), (b_k, b_l) \in S$ with $i < k < j < l$.

Goal: maximize $|S|$ (the number of base pairs), which is assumed proportional to the (negative) free energy.



$S = \{(1,10),(2,9),(3,8)\}$, valid: nested pairs, no sharp turns, $|S| = 3$

Recurrence

Let $M[i,j]$ be the maximum number of pairs in a secondary structure of $b_i \cdots b_j$.
- If $i \geq j - 4$: $M[i,j] = 0$ by the no-sharp-turn condition.
- Otherwise either b_j is unmatched, giving $M[i, j-1]$; or b_j pairs with some b_k,

$i \le k \le j-5$, and the non-crossing condition decouples the rest into the independent intervals $[i, k-1]$ and $[k+1, j-1]$:

$$M[i,j] = \max\Big\{ M[i,j-1],\ \max_{\substack{i \le k \le j-5 \\ b_k, b_j \text{ complementary}}} \big(1 + M[i,k-1] + M[k+1,j-1]\big) \Big\}.$$

The answer is $M[1,n]$, filled by increasing length.

Algorithm 48 RNA(b,n) and Construct-RNA

```
1: allocate M[1..n, 1..n], s[1..n, 1..n], all 0
2: for ℓ = 1 to n do
3:     for i = 1 to n − ℓ + 1 do
4:         j ← i + ℓ − 1;   M[i, j] ← M[i, j − 1]
5:         for k = i to j − 5 do
6:             if b_k and b_j are complementary then
7:                 t ← 1 + M[i, k − 1] + M[k + 1, j − 1]
8:                 if t > M[i, j] then
9:                     M[i, j] ← t;   s[i, j] ← k
10:                end if
11:            end if
12:        end for
13:    end for
14: end for
15: procedure Construct-RNA(i, j)
16:    if i ≥ j − 4 then
17:        return
18:    end if
19:    if s[i, j] = 0 then
20:        Construct-RNA(i, j − 1)
21:    end if
22:    print "s[i, j]–j";   Construct-RNA(i, s[i, j] − 1);   Construct-RNA(s[i, j] +
   1, j − 1)
23: end procedure
```

Time $O(n^3)$ ($O(n^2)$ intervals, $O(n)$ partner choices each), space $O(n^2)$. With the total number of bases fixed, a secondary structure is found by tracing s from $(1,n)$ downwards.

7 Graph Search and Fundamentals

7.1 Graph Algorithms

Graphs model relationships, so they are ubiquitous. Rather than solving one application, we design algorithms for abstract graphs and then reuse them everywhere.

Application	Nodes	Edges
Transportation	street intersections	highways
Communication	computers	fiber-optic cables
World Wide Web	web pages	hyperlinks (directed)
Social network	people	relationships
Food web	species	predator–prey
Software systems	functions	function calls
Scheduling	tasks	precedence constraints
Circuits	gates	wires

Two canonical graph problems motivate this chapter.

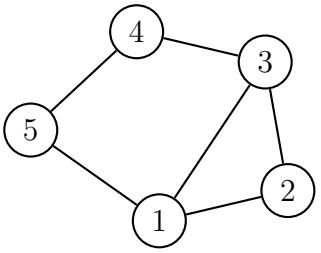
- **Travelling Salesman Problem (TSP).** Given a graph whose edges carry lengths, find a cycle of minimum total weight visiting every vertex. No polynomial-time algorithm is known; the problem is equivalent to $P = NP$.
- **Topological ordering.** Given a directed graph, order the vertices so that for every edge $u \to v$, u comes before v — or report that the graph contains a cycle.

7.1.1 Graph Basics

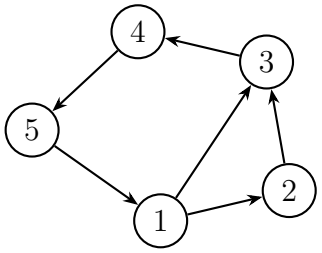
Definition

A graph is a pair $G = (V, E)$ where V is a set of nodes (vertices) and E is a set of edges between pairs of nodes. We abuse notation and also write V and E for the counts $|V|$ and $|E|$, and often set $n = |V|$, $m = |E|$.

- **Undirected graph:** edges have no direction. An edge is an unordered pair $\{u, v\}$.
- **Directed graph:** edges have directions; an edge is an ordered pair (u, v). If an edge exists in both directions we represent it with two opposite directed edges.
- **Simple graph:** at most one edge between a pair of nodes (at most two, in opposite directions, for a directed graph). No self-loops, no duplicate edges.



undirected graph



directed graph

Degrees. In an undirected graph, $\deg(v)$ is the number of edges touching v. In a directed graph, $\deg_{out}(v)$ counts edges leaving v and $\deg_{in}(v)$ counts edges entering v.

Handshaking identity

For every undirected graph, $\sum_{v \in V} \deg(v) = 2|E|$: each edge contributes exactly 1 to the degree of each of its two endpoints. For every directed graph,

$$\sum_{v \in V} \deg_{out}(v) = \sum_{v \in V} \deg_{in}(v) = |E|,$$

because each edge has exactly one tail and one head.

Maximum number of edges. Each node can connect to at most $V - 1$ other nodes, so a directed simple graph has at most $V(V-1)$ edges. Replacing each pair of opposite directed edges with a single undirected edge gives at most $V\dfrac{(V-1)}{2}$ edges in an undirected simple graph. Thus

$$0 \le |E| \le \frac{|V|(|V|-1)}{2} \text{ (undirected)}, \qquad 0 \le |E| \le |V|(|V|-1) \text{ (directed)}.$$

Degree parity exercises

Problem. (a) Show that if every node of a graph has degree 3, then $|V|$ is even. (b) Show that if every node has odd degree, then $|V|$ is even.
Solution. (a) By handshaking, $3|V| = 2|E|$. The right side is even, hence $3|V|$ is even, hence $|V|$ is even.
(b) Write $d(v_i) = 2k_i + 1$ with $k_i \in \mathbb{Z}$. Then

$$\sum_{i=1}^{|V|} d(v_i) = \sum_{i=1}^{|V|} (2k_i + 1) = |V| + 2\sum_{i=1}^{|V|} k_i = 2|E|,$$

so $|V| = 2|E| - 2\sum_i k_i$ is a difference of two even numbers and therefore even. ■

The party handshake

Problem. At a party, show that the number of guests who shake hands an odd number of times must be even.
Solution. Build $G = (V, E)$ with one node per guest and one edge per handshake. The claim becomes: in every graph, the number of odd-degree nodes is even. Split $V = V_{odd} \cup V_{even}$. The sum of the even degrees is even, and $\sum_v \deg(v) = 2|E|$ is even, so the sum of the odd degrees is even. Writing each odd degree as $2k_i + 1$,

$$\sum_{v_i \in V_{odd}} \deg(v_i) = |V_{odd}| + 2\sum_{v_i \in V_{odd}} k_i \text{ is even } \implies |V_{odd}| \text{ is even.}$$

■

Two degree puzzles

Problem 1. In a group of 8 people some pairs shake hands. Is it possible that everyone shakes hands with a different number of people?
Solution. Each person's handshake count lies in $\{0, 1, \ldots, 7\}$, seven values for eight people, so two must coincide — unless the counts are exactly $0, 1, \ldots, 7$. But the person with 7 handshakes shook hands with everybody, including the person with 0, a contradiction. Hence at least two people have the same count, so it is not possible. ■

Problem 2. In a simple connected graph on 6 vertices, five vertices have degrees $1, 2, 3, 4, 5$. What is the degree of the sixth vertex?
Solution 1. Name the vertices $v_1, \ldots, v_5$ with $\deg(v_i) = i$ and let v_6 be the unknown one. Since $\deg(v_5) = 5$, vertex v_5 is adjacent to all others, so the only neighbour of v_1 is v_5. Since $\deg(v_4) = 4$, v_4 is adjacent to every vertex except v_1; hence the two neighbours of v_2 are v_4 and v_5. Thus v_6 is adjacent to v_4 and v_5, and is not adjacent to v_1 or v_2; the same is true of v_3. Since $\deg(v_3) = 3$, v_3 must be adjacent to v_6. Therefore v_6 is adjacent exactly to v_4, v_5, v_3, so $\deg(v_6) = 3$.
Solution 2. The sum of degrees is even, so the missing degree is odd: 1, 3 or 5. As above, v_6 is adjacent to v_4 and v_5 but not to v_1 or v_2, which rules out 5 (needs adjacency to v_1, v_2) and 1 (needs adjacency only to one vertex). Hence $\deg(v_6) = 3$. ■

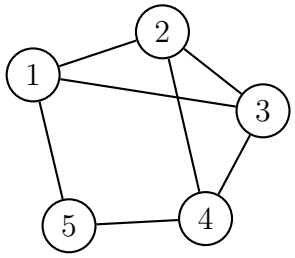
7.1.2 Graph Representation

Operation	Adjacency list	Adjacency matrix
Storage	$\Theta(\|V\| + \|E\|)$	$\Theta(\|V\|^2)$
List all neighbours of u	$\Theta(\deg(u))$	$\Theta(\|V\|)$
Test whether $(u, v) \in E$	$\Theta(\deg(u))$	$O(1)$

An **adjacency list** is a node-indexed array of lists: node u stores exactly its neighbours. An **adjacency matrix** is a $|V| \times |V|$ table with $A[u][v] = 1$ iff $(u, v) \in E$. One can convert between the two representations in $\Theta(|V|^2)$ time.

Which representation?

Adjacency lists are used far more often because most real graphs are sparse ($|E| \ll |V|^2$); they also let BFS/DFS enumerate neighbours in time proportional to the degree. Matrices win when the graph is dense or when $O(1)$ edge queries are essential. Both BFS and DFS run in $\Theta(|V| + |E|)$ with adjacency lists, but $\Theta(|V|^2)$ with a matrix.

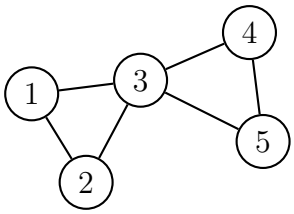


adjacency list:
1: 2, 5, 3
2: 1, 3, 4
3: 2, 1, 4
4: 3, 2, 5
5: 4, 1

7.1.3 Paths, Connectivity, and Trees

Paths and cycles

A **path** in G is a sequence of nodes $v_1, v_2, \ldots, v_k$ such that $(v_i, v_{i+1}) \in E$ for every i; its **length** is $k - 1$, the number of edges. A path is **simple** if all its nodes are distinct. A **cycle** is a path with $v_1 = v_k$ and $k > 2$; it is **simple** if $v_1, \ldots, v_{k-1}$ are all distinct. An undirected graph is **connected** if every pair of nodes is joined by a path. A **tree** is a connected undirected graph with no cycle; a **forest** is a graph with no cycle (a collection of trees).



simple cycle: $1, 3, 2, 1$; nonsimple cycle: $2, 1, 3, 2, 4, 5, 3, 2$

Tree characterisation

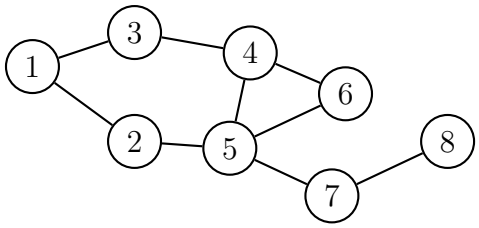
Let G be an undirected graph. Any two of the following statements imply the third, and hence G is a tree: (1) G is connected; (2) G contains no cycle; (3) $|E| = |V| - 1$. In particular, a connected graph always satisfies $|E| \geq |V| - 1$.

$|E| \geq |V| - 1$ for connected graphs

Start from a spanning tree on all $|V|$ nodes (it has exactly $|V| - 1$ edges) and add the remaining edges one by one: connectivity is preserved and every added edge creates a cycle. Since a forest on $|V|$ nodes has at most $|V| - 1$ edges, any connected graph has at least $|V| - 1$ edges, with equality iff it is a tree.

s–t connectivity and shortest paths

The s–t **connectivity problem** asks whether a path from s to t exists. In a directed graph this is called **reachability** and is not symmetric: s may reach t while t cannot reach s.
The s–t **shortest path problem** asks for the minimum number of edges (or, with weights, the minimum total weight) on an s–t path; that number is the **distance** from s to t. In an undirected graph $s \rightsquigarrow t$ and $t \rightsquigarrow s$ have the same distance, but in a directed graph they may differ.



path $6, 5, 7, 8$ has length 3 from 6 to 8

Wireless network connectivity

Problem. In a wireless network of n devices (n even), each device is within range of at least $\frac{n}{2}$ other devices. Show all devices are connected.

Solution. Model the network as an undirected graph G in which every node has degree $\geq \frac{n}{2}$. Take any two nodes u, v.

If $(u, v) \in E$ they are connected. Otherwise consider the $n - 2$ nodes other than u, v: each of u and v has at least $\frac{n}{2}$ neighbours among these $n - 2$ nodes, and

$$\frac{n}{2} + \frac{n}{2} - (n - 2) = 2 > 0,$$

so by the pigeonhole principle they share a common neighbour w, giving the path u, w, v. Since u, v were arbitrary, G is connected. ■

Follow-up. If the threshold is lowered to $\frac{n}{2} - 1$ the claim fails: take two disjoint cliques of $\frac{n}{2}$ nodes each. Every node has degree $\frac{n}{2} - 1$ but no edge crosses between the two cliques, so the graph is disconnected.

Rooted trees

Given a tree T, choose a **root** r and orient every edge away from r. The result is a **rooted tree**: every node $v \neq r$ has exactly one **parent** (its predecessor toward r) and zero or more **children**. A node with no children is a **leaf**. This is exactly the structure produced by the BFS and DFS trees below.

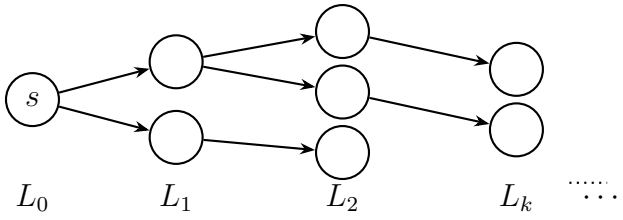
7.1.4 Breadth-First Search

BFS layers

Breadth-first search explores outward from a start node s in all directions, adding nodes one layer at a time:

$$L_0 = \{s\}, \qquad L_{i+1} = \{v \notin L_0 \cup \cdots \cup L_i : \ (u, v) \in E \text{ for some } u \in L_i\}.$$

L_1 is the set of neighbours of s; L_2 consists of the still unseen nodes with an edge into L_1; and so on. The **distance** from u to v is the number of edges on a shortest u–v path.



BFS layer theorem

For every i, the layer L_i consists of exactly the nodes at distance i from s. Consequently there is a path from s to t if and only if t appears in some layer.

Each node stores three fields:

- **color**: white = undiscovered, **gray** = discovered but neighbours not fully processed, black = discovered and fully processed;
- **distance** d: length of a shortest s–u path;
- **parent** p: predecessor of u on that shortest path.

Algorithm 49 BFS from a single source s

```
1:  procedure BFS(G, s)
2:      for each vertex u ∈ V − {s} do
3:          u.color ← white;   u.d ← ∞;   u.p ← nil
4:      end for
5:      s.color ← gray;   s.d ← 0
6:      initialize an empty queue Q;   ENQUEUE(Q, s)
7:      while Q ≠ ∅ do
8:          u ← DEQUEUE(Q)
9:          for each v ∈ Adj[u] do
10:             if v.color = white then
11:                 v.color ← gray;   v.d ← u.d + 1;   v.p ← u
12:                 ENQUEUE(Q, v)
13:             end if
14:         end for
15:         u.color ← black
16:     end while
17: end procedure
```

Running time and BFS tree

Each node enters and leaves the queue at most once, and its adjacency list is scanned exactly once when it is dequeued:

$$\sum_{u} (1 + \deg(u)) = \Theta(|V| + |E|),$$

which is $\Theta(|E|)$ when the graph is connected. The parent pointers form a **BFS tree** rooted at s: the parent of every node lies one layer earlier, and following parent pointers from v back to s yields a shortest path. Hence BFS computes the shortest path from s to every other node in unweighted graphs.

Queue contents

The nodes inside Q are exactly the gray nodes: already seen, but not yet fully processed. A node turns black only after all of its neighbours have been examined. Because Q is FIFO, nodes leave in nondecreasing order of distance, which is precisely why layers are completed one at a time.

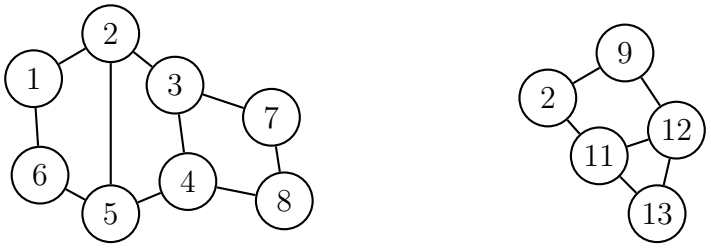
7.1.5 Connected Components and Reachability

Connected components via BFS

The **connected component** of s is the set of all nodes reachable from s; running BFS from s finds exactly this set. To find all components, rename the single-source procedure BFS-VISIT and repeatedly launch it from any still-white node.

Algorithm 50 Finding all connected components

```
1: procedure BFS-Visit(G, s)
       assumes s is white
2:     s.color ← gray;   s.d ← 0
3:     initialize an empty queue Q;   Enqueue(Q, s)
4:     while Q ≠ ∅ do
5:         u ← Dequeue(Q)
6:         for each v ∈ Adj[u] do
7:             if v.color = white then
8:                 v.color ← gray;   v.d ← u.d + 1;   v.p ← u;   Enqueue(Q, v)
9:             end if
10:        end for
11:        u.color ← black
12:    end while
13: end procedure
14: procedure BFS(G)
15:    for each vertex u ∈ V do
16:        u.color ← white;   u.d ← ∞;   u.p ← nil
17:    end for
18:    for each vertex u ∈ V do
19:        if u.color = white then
20:            BFS-Visit(G, u)
21:        end if
22:    end for
23: end procedure
```



BFS-Visit(1) blackens $\{1, \ldots, 8\}$, BFS-Visit(2) blackens $\{2, 9, 10, 11, 12, 13\}$ (the component containing 2), and every call from a node already blackened does nothing. Each vertex is initialised once and scanned once, so the total time is still $\Theta(|V| + |E|)$.

Directed reachability and strong connectivity

Problem. For a directed graph, decide whether every node can reach every other node.
Solution. Pick any node s and run BFS from s in G. Then build G^{rev} (all edges reversed) and run BFS from s in G^{rev}. Return true iff both searches reach all nodes. Why one source suffices: call s strong if s reaches every t and every t reaches s. If some strong vertex exists, then every vertex is strong (compose $u \to s \to v$ and $v \to s \to u$), so the graph is strongly connected iff a single fixed s passes the two tests. ■



strongly connected not strongly connected

Algorithm 51 Strongly connected components (simple version)

```
1: procedure Strongly-Connected-Components(G)
2:     create G^rev, the graph G with every edge reversed
3:     while there are nodes left do
4:         u ← any remaining node
5:         run BFS in G from u;   run BFS in G^rev from u
6:         C ← {nodes reached in both BFS runs}
7:         output C as a strongly connected component
8:         remove C and all its edges from G and G^rev
9:     end while
10: end procedure
```

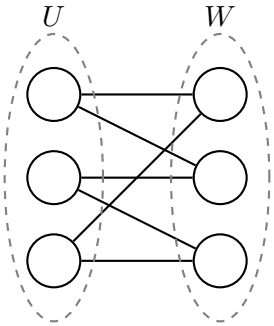
Each round removes at least one node and costs $O(|V| + |E|)$, so the procedure runs in $O(|V||E|)$; a $\Theta(|V| + |E|)$ algorithm exists but is outside the scope of this course.

Bipartite graphs

A graph is **bipartite** if V can be partitioned into two disjoint sets U and W such that every edge has one endpoint in U and the other in W. To test it on a connected undirected graph:

- run BFS from any vertex and let $d[v]$ be the distance computed;
- put the even-distance vertices in S and the odd-distance ones in $V - S$;
- the graph is bipartite iff every edge $\{u, v\}$ joins vertices of different parity.

Equivalently, a graph is bipartite iff it contains no odd cycle: during BFS, an edge whose two endpoints sit in the same layer immediately certifies an odd cycle and non-bipartiteness.



BFS modelling exercises

1. Knight moves. Find the minimum number of knight moves from (x, y) to (x', y') on a chessboard.
Solution. Build an implicit graph whose nodes are board squares, and whose edges join squares one knight move apart: from (x, y) the neighbours are $(x \pm 2, y \pm 1)$ and $(x \pm 1, y \pm 2)$ whenever they stay on the board. Run BFS from (x, y); the first time (x', y') is discovered its distance is the minimum number of moves. ■
2. Binary maze. Given a binary matrix, find the length of a shortest path from (x, y) to (x', y') using only cells equal to 1, moving up/down/left/right.
Solution. Same reduction: nodes are the cells with value 1, edges join vertically or horizontally adjacent cells. BFS from (x, y); the distance assigned to (x', y') is the answer. Both problems run in time proportional to the number of nodes plus edges of the implicit graph. ■

7.1.6 Worked BFS Example

Consider the undirected graph with the following adjacency lists.

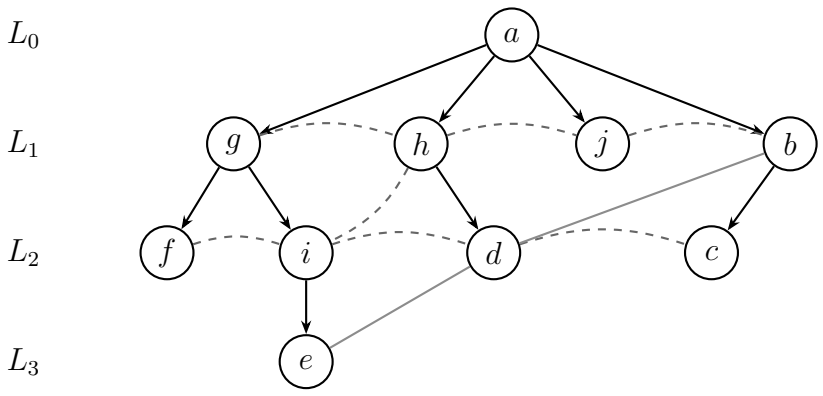
Vertex	Adjacency list
a	g, h, j, b
b	a, j, d, c
c	d, b
d	e, i, h, b, c
e	i, d
f	g, i
g	a, h, f, i
h	a, g, i, d, j
i	f, g, h, d, e
j	h, a, b

Run BFS from a. The queue evolves as follows.

Step	Dequeued u	Newly discovered (distance, parent)	Q afterwards
1	a	$g(1,a)$, $h(1,a)$, $j(1,a)$, $b(1,a)$	g, h, j, b
2	g	$f(2,g)$, $i(2,g)$	h, j, b, f, i
3	h	$d(2,h)$	j, b, f, i, d
4	j	—	b, f, i, d
5	b	$c(2,b)$	f, i, d, c
6	f	—	i, d, c
7	i	$e(3,i)$	d, c, e
8	d	—	c, e
9	c	—	e
10	e	—	$\emptyset$

Hence $L_0 = \{a\}$, $L_1 = \{g, h, j, b\}$, $L_2 = \{f, i, d, c\}$, $L_3 = \{e\}$, and the distances are

$$d(a) = 0, \quad d(g) = d(h) = d(j) = d(b) = 1, \quad d(f) = d(i) = d(d) = d(c) = 2, \quad d(e) = 3.$$



Solid arrows are the tree edges of the BFS tree ($9 = |V| - 1$ of them). Dashed grey arcs join nodes on the same level; the two lighter grey edges (b–d and d–e) join consecutive levels. Note that h–d is the tree edge, while b–d and d–e are non-tree edges.

Reading the figure

The first time a node u is seen is when a neighbour v is processed off the queue; then

u turns gray, receives $d(u) = d(v) + 1$ and parent $p(u) = v$. The edges $(u, p(u))$ form the BFS tree. All tree edges go from level i to level $i+1$, so a path in this tree realises the shortest distance.

7.1.7 Depth-First Search

BFS is broad: when a node is processed it scans all its neighbours before any other node is processed. Depth-first search instead goes deep: when u examines a neighbour v, it immediately starts processing v (which starts processing the children of v, and so on); only after the whole subtree below v is finished does u resume with its next neighbour.

Algorithm 52 Depth-first search

```
 1: procedure DFS(G)
 2:     for each vertex u ∈ V do
 3:         u.color ← white;   u.p ← nil
 4:     end for
 5:     for each vertex u ∈ V do
 6:         if u.color = white then
 7:             DFS-Visit(u)
 8:         end if
 9:     end for
10: end procedure
11: procedure DFS-Visit(u)
12:     u.color ← gray
13:     for each v ∈ Adj[u] do
14:         if v.color = white then
15:             v.p ← u;   DFS-Visit(v)
16:         end if
17:     end for
18:     u.color ← black
19: end procedure
```

Running time and DFS tree

Every vertex is visited exactly once and every adjacency entry is inspected exactly once, so the running time is $\Theta(|V|+|E|)$. The parent pointers form the **DFS tree**; its edges are exactly the pairs $(u, p(u))$. Before DFS-Visit(u) returns, all nodes of the connected component containing u are blackened, so DFS-Visit is entered exactly once per connected component of G.

Colors mean different things

Gray means on the current recursion stack; black means finished. A gray neighbour is an ancestor in the DFS tree (a back edge), whereas a black neighbour can belong to an unrelated subtree. This distinction drives the edge classification and the cycle-detection algorithms below.

7.1.8 Worked DFS Example

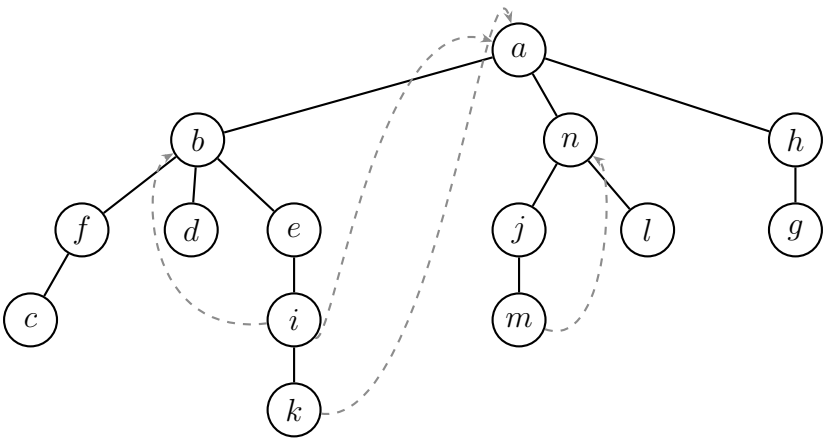
Use the following undirected graph (the same graph as the lecture example).

Vertex	Adjacency list
a	b, i, k, n, h
b	a, f, d, e, i
c	f
d	b
e	b, i
f	c, b
g	h
h	a, g
i	e, b, k, a
j	n, m
k	i, a
l	n
m	j, n
n	a, j, m, l

Starting at a and scanning each adjacency list from left to right, the recursion proceeds as follows:

$$a \to b \to f \to c, \quad b \to d, \quad b \to e \to i \to k,$$
$$a \to n \to j \to m, \quad n \to l, \quad a \to h \to g.$$

- discovery order: $a, b, f, c, d, e, i, k, n, j, m, l, h, g$;
- finish order: $c, f, d, k, i, e, b, m, j, l, n, g, h, a$.



Bold edges are the $13 = |V| - 1$ tree edges. The four grey dashed arcs are the **back edges**, each pointing from a node to a proper ancestor: (i, b), (i, a), (k, a) and (m, n). The first back edge encountered is (i, b); the DFS-tree path $b \to e \to i$ together with the edge (i, b) closes the cycle b, e, i, b.

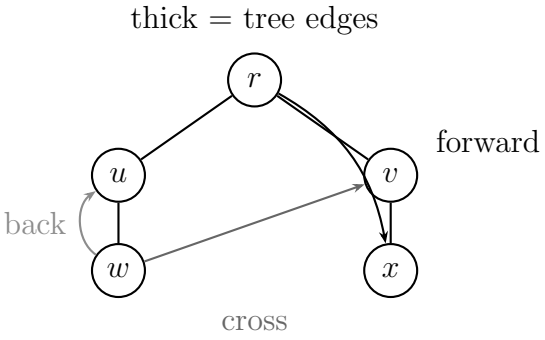
Why no other edge types appear

In an undirected DFS every non-tree edge joins a node to an ancestor, i.e. is a back edge. There are no forward or cross edges in undirected graphs — this is proved in the next subsection.

7.1.9 Classifying Edges

After a BFS or DFS, every edge of G falls into exactly one of four classes:

- **tree edges**: the edges traversed by the search (the parent pointers);
- **back edges**: from a node to a proper ancestor in the search tree;
- **forward edges**: from a node to a proper descendant (other than a child);
- **cross edges**: between two nodes with no ancestor/descendant relationship.



Theorem: undirected DFS has no cross edges

In a DFS of an undirected graph, every edge is a tree edge or a back edge.

Proof. Let (u, v) be any edge and assume without loss of generality that u is discovered first. When u's adjacency list is scanned, v cannot be black yet: if v were black, then u would have finished before v was discovered, so when u scanned (u, v) the node v was white and would have become a child of u — contradicting that u finished first. Hence at that moment v is either white or gray. If v is white, (u, v) becomes a tree edge. If v is gray, then v was discovered while u was still gray, so v lies in the DFS subtree of u; if $p(v) = u$ the edge is a tree edge, otherwise it joins v to a proper ancestor u and is a back edge. No edge can join two incomparable subtrees, so there are no cross edges. ■

Theorem: undirected BFS has no forward or back edges

In a BFS of an undirected graph, every non-tree edge is a cross edge; equivalently every edge joins two nodes on the same level or on consecutive levels.

Proof. Let $(u, v) \in E$. Because v is a neighbour of u, BFS gives

$$|d(u) - d(v)| \leq 1 :$$

whichever endpoint is dequeued first relaxes the other with distance at most one more. A forward or back edge would join a node to a proper ancestor or descendant, and in a BFS tree such nodes differ in level by at least 2, contradicting $|d(u) - d(v)| \leq 1$. Hence no forward or back edges exist, leaving only tree edges and cross edges. ■

7.1.10 Cycle Detection

Counting edges per component

A connected component is a tree iff it has exactly $|V_c| - 1$ edges. Run BFS/DFS to find all components and count the edges inside each one: if some component has at least as many edges as vertices, it contains a cycle. Time $\Theta(|V| + |E|)$.

Cycle detection by DFS back edges

Run DFS on each component. If (u, v) is a back edge, then v is a proper ancestor of u, so the DFS-tree path from v down to u plus the edge (u, v) is a cycle. If no back edge exists, the component consists only of tree edges, hence is a tree. Only tree edges

are traversed before the first non-tree edge is found, and there are at most $|V| - 1$ of them, so the search costs $\Theta(|V|)$.

Algorithm 53 Cycle detection with DFS

```
 1: procedure CYCLE-DETECTION(G)
 2:     for each vertex u ∈ V do
 3:         u.color ← white;   u.p ← nil
 4:     end for
 5:     for each vertex u ∈ V do
 6:         if u.color = white then
 7:             DFS-VISIT(u)
 8:         end if
 9:     end for
10:     return "No cycle"
11: end procedure
12: procedure DFS-VISIT(u)
13:     u.color ← gray
14:     for each v ∈ Adj[u] do
15:         if v.color = white then
16:             v.p ← u;   DFS-VISIT(v)
17:         else if v ≠ u.p then
            back edge (u, v) found
18:             output "Cycle found:"
19:             while u ≠ v do
20:                 output u;   u ← u.p
21:             end while
22:             output v;   return
23:         end if
24:     end for
25:     u.color ← black
26: end procedure
```

Cycle detection by BFS cross edges

Run BFS on each component. If (u, v) is a cross edge, let $\ell = \text{lca}(u, v)$ be the lowest common ancestor of u and v in the BFS tree; then the path $\ell \rightsquigarrow u$, the path $\ell \rightsquigarrow v$ and the edge (u, v) form a cycle. If no cross edge exists the component is a tree.
To find ℓ: if $d(u) = d(v)$ climb from both nodes simultaneously; if $d(u) > d(v)$, first climb $d(u) - d(v)$ steps from u, then climb together. This costs $O(|V|)$ per edge, so the whole procedure is $O(|V||E|)$ in the worst case.

Unique simple paths

Problem. Given a directed graph G and a node s, decide whether s has a unique simple path to every other node.
Solution. Run BFS/DFS from s and first check that every node is reachable. Then classify every edge. Tree edges and back edges are harmless (a back edge only closes a cycle, which a simple path cannot use, so it creates no second simple path). A forward edge or a cross edge (u, v) gives a second route from s to v: the tree path $s \rightsquigarrow u$ followed by (u, v), which differs from the tree path $s \rightsquigarrow v$ because u is not the parent of v. Hence the answer is "yes" iff no forward and no cross edge exists.
Classifying an edge (u, v) while scanning the adjacency list of u:

Colour of v	DFS	BFS
white	tree edge	tree edge
gray	back edge	back edge
black	forward or cross edge	cross edge or back edge (ambiguous)

BFS has no forward edges, but it cannot always separate cross from back edges; DFS is the right tool for this test. ■

7.1.11 Topological Sorting

DAGs and topological order

A **directed acyclic graph** (DAG) is a directed graph with no cycle. A **topological ordering** of a DAG is a linear ordering of its vertices such that for every edge (u, v), u appears before v. The order is usually not unique: a DAG with 10 vertices may admit many valid orders.

Key observation

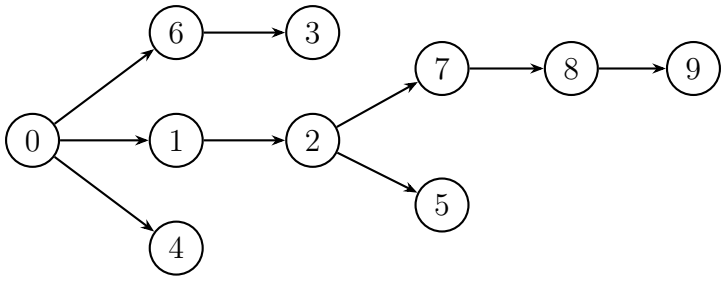
Every DAG has at least one vertex with in-degree 0. (Otherwise, following incoming edges backwards forever must repeat a vertex and hence produce a cycle.)

Algorithm 54 Topological sort (Kahn's algorithm)

```
1:  procedure TOPOLOGICAL-SORT(G)
2:      initialise an empty queue Q
3:      for each u ∈ V do
4:          if indegree(u) = 0 then
5:              ENQUEUE(Q, u)
6:          end if
7:      end for
8:      while Q is not empty do
9:          u ← DEQUEUE(Q);   output u
10:         for each v ∈ Adj(u) do
11:             indegree(v) ← indegree(v) − 1
12:             if indegree(v) = 0 then
13:                 ENQUEUE(Q, v)
14:             end if
15:         end for
16:     end while
17: end procedure
```

Correctness. Removing a vertex together with its outgoing edges from a DAG leaves a DAG, so at every stage the current graph has a vertex of in-degree 0 and the algorithm can always output another vertex; hence all vertices are output. Suppose the output order were not topological: then some edge (u, v) would have v output before u. But v could only enter the queue when its in-degree dropped to 0, which requires the edge (u, v) to have been removed, which requires u to have been output already — a contradiction. Therefore the output is a topological order.

If the graph is not acyclic, the process gets stuck before outputting all $|V|$ vertices; comparing the number of outputs with $|V|$ therefore also detects cycles. With adjacency lists the running time is $\Theta(|V| + |E|)$.



Dequeued	Q **afterwards**	**Output so far**
—	0	—
0	$6, 1, 4$	0
6	$1, 4, 3$	0,6
1	$4, 3, 2$	0,6,1
4	$3, 2$	0,6,1,4
3	2	0,6,1,4,3
2	$7, 5$	0,6,1,4,3,2
7	$5, 8$	0,6,1,4,3,2,7
5	8	0,6,1,4,3,2,7,5
8	9	0,6,1,4,3,2,7,5,8
9	$\emptyset$	0,6,1,4,3,2,7,5,8,9

The resulting order $0, 6, 1, 4, 3, 2, 7, 5, 8, 9$ is one valid topological order; for instance $0, 4, 1, 6, 2, 5, 3, 7, 8, 9$ is another.

Topological order requires a DAG

A directed graph has a topological ordering if and only if it is acyclic. On a graph containing a cycle, Kahn's algorithm outputs only the vertices that are never blocked, and the shortfall $|V| - \#\text{outputs}$ is exactly the set of vertices lying on or downstream of a cycle.

8 Graph Optimization: MST, Shortest Paths, Max Flow, Matchings and Stable Marriage

8.1 Minimum Spanning Trees

8.1.1 Useful Facts about Connected Graphs

Edge counts and tree characterizations

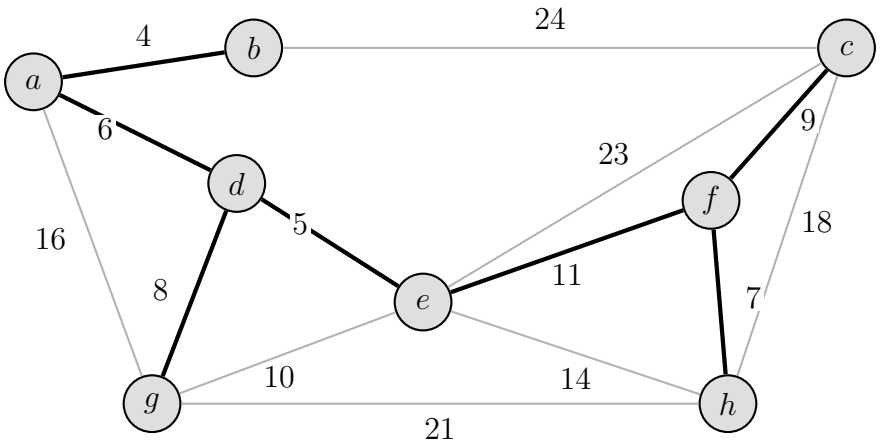
A connected undirected graph $G = (V, E)$ has between $|V|-1$ and $|V|\dfrac{(|V|-1)}{2}$ edges. Hence $|E| = O(|V|^2)$ and

$$\log|E| = O\big(\log|V|^2\big) = O(\log|V|).$$

- An undirected graph $G = (V, E)$ with fewer than $|V| - 1$ edges is not connected.
- A connected undirected graph $G = (V, E)$ with exactly $|V| - 1$ edges is a tree.
- An undirected graph $G = (V, E)$ with more than $|V| - 1$ edges contains at least one cycle.

8.1.2 The Minimum Spanning Tree Problem

Minimum spanning tree. Given a connected undirected graph $G = (V, E)$ with real-valued edge weights $w(e)$, a minimum spanning tree (MST) is a subset of the edges $T \subseteq E$ such that T is a tree that connects all nodes and $\sum_{e\in T} w(e)$ is minimized.



The bold edges form one minimum spanning tree, with total weight

$$4 + 6 + 8 + 5 + 11 + 9 + 7 = 50;$$

the grey edges are not used by this tree.

Applications

MSTs are used in telephone networks, electrical and hydraulic systems, TV cables, computer networks, and road systems. The abstract problem is solved once and then reused in all of these settings.

Shape of any spanning tree

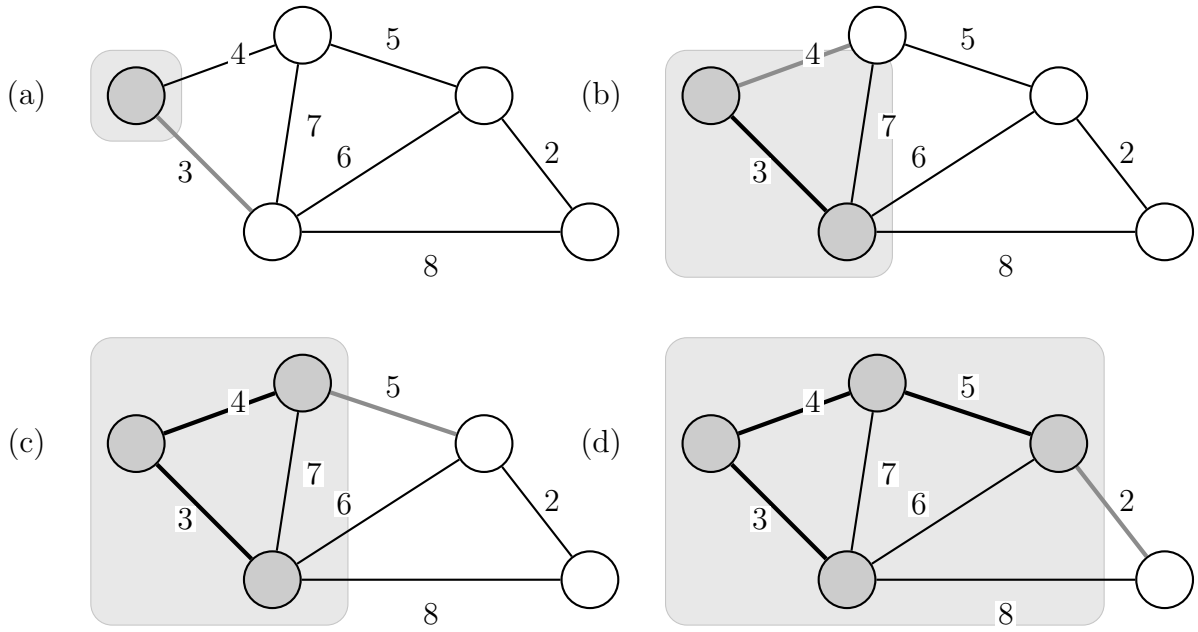
Every spanning tree of a connected graph on n vertices has exactly $n-1$ edges: it must be connected, so it needs at least $n-1$ edges, and it must be acyclic, so it cannot have more. Consequently an MST has $|V|-1$ edges, and a graph that is not connected has no spanning tree at all.

8.1.3 Prim's Algorithm

Idea. Grow a single tree out of one starting node.

1. Initialize $S = \{\text{any one node}\}$.
2. Add the minimum-cost edge $e = (u, v)$ with $u \in S$ and $v \in V - S$ to T.
3. Add v to S.
4. Repeat until $S = V$.

Because every added edge has exactly one endpoint outside S, no cycle is ever created, and when $S = V$ the chosen edges form a spanning tree.

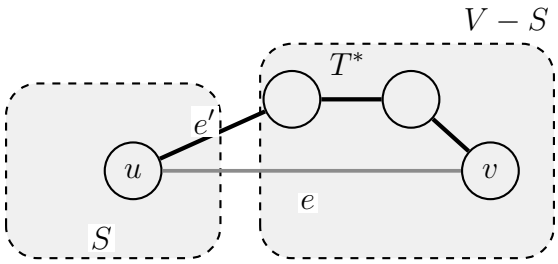


The shaded region marks the explored set S; bold edges are already in T; the grey edge is the lightest edge crossing the cut $(S, V - S)$. The last step adds 2, so the tree costs $3 + 4 + 5 + 2 = 14$.

8.1.4 Correctness: the Cut Lemma

Cut Lemma (distinct weights)

Assumption: all edge weights are distinct. Let S be any subset of nodes, and let e be the minimum-cost edge with exactly one endpoint in S. Then every MST must contain e.



Proof (exchange argument). Let T^* be any MST and let $e = (u, v)$ with $u \in S$ and $v \notin S$. Suppose $e \notin T^*$. Since T^* is a spanning tree, it contains a unique path from u to v; this path leaves S through some edge $e' \in T^*$ that also has exactly one endpoint in S. By the choice of e we have $w(e') > w(e)$. Replacing e' by e in T^* keeps a spanning tree and lowers its total weight, contradicting the fact that T^* is an MST. Hence e lies in every MST. ■

Cut Lemma without distinctness

If weights may repeat, the same argument only gives $w(e') \geq w(e)$; replacing e' by e then never increases the cost. The conclusion weakens to: there exists an MST that contains e.

Prim's algorithm is correct

At every step, the edge Prim adds is the cheapest edge crossing the cut $(S, V - S)$. With distinct weights, the Cut Lemma says that this edge belongs to the (unique) MST, so all edges chosen so far are safe and the final tree is the unique MST. Without distinct weights the same argument gives one MST.

8.1.5 Implementation of Prim's Algorithm

Implementation.

- Maintain the set of explored nodes S.
- For each unexplored node v, maintain its cheapest edge to a node in S.
- Keep all nodes in a priority queue with this cheapest-edge weight as the key.

When a node u becomes explored, every white neighbour v may find a cheaper connection to S, so its key is decreased.

Algorithm 55 PRIM(G, r) with Decrease-Key

```
1: for each v ∈ V do
2:     v.key ← ∞;  v.p ← NIL();  v.colour ← white
3: end for
4: r.key ← 0;  insert every vertex into a min-priority queue Q keyed by key
5: while Q ≠ ∅ do
6:     u ← EXTRACT-MIN(Q);  u.colour ← black
7:     for each v ∈ ADJ(u) do
8:         if v.colour = white and w(u, v) < v.key then
9:             v.p ← u;  v.key ← w(u, v)
10:            DECREASE-KEY(Q, v, w(u, v))
11:        end if
12:    end for
13: end while
```

At the end the parent pointers form the MST. Running time $O(|E| \log |V|)$ with a binary heap.

Avoiding Decrease-Key

Decrease-key has a large constant overhead in time and space, and most libraries do not support it (for example `std::priority_queue`). The alternative is lazy insertion: never decrease a key, simply insert the improved key as a new entry and skip stale entries when they are extracted.

Algorithm 56 PRIM(G, r) without Decrease-Key

```
1: for each v ∈ V do
2:     v.key ← ∞;   v.p ← NIL();   v.colour ← white
3: end for
4: r.key ← 0;   insert r into a min-priority queue Q
5: while Q ≠ ∅ do
6:     u ← EXTRACT-MIN(Q)
7:     if u.colour = black then
8:         continue
9:     end if
10:    output the edge (u, u.p);   u.colour ← black
11:    for each v ∈ ADJ(u) do
12:        if v.colour = white and w(u, v) < v.key then
13:            v.p ← u;   v.key ← w(u, v);   INSERT(Q,v)
14:        end if
15:    end for
16: end while
```

Every edge triggers at most one insertion, so $|Q| = O(|E|)$ and the running time is

$$O(|E| \log |E|) = O(|E| \log |V|^2) = O(|E| \log |V|),$$

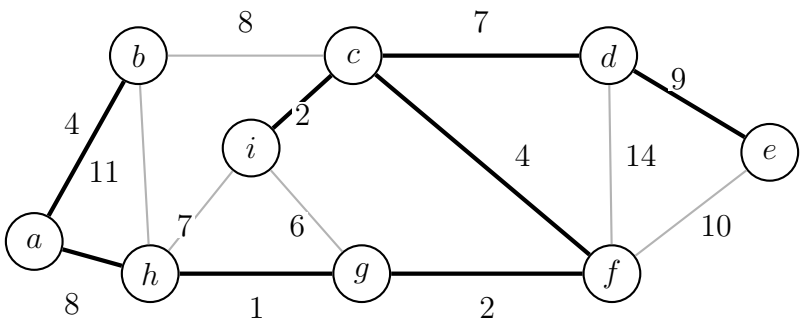
the same asymptotic bound as the decrease-key version.

Common pitfall

Without the colour test, a stale entry with a large key would print a wrong tree edge. Always skip an entry whose vertex is already black: the edge $(u, u.p)$ output for a black u is not part of the tree.

8.1.6 Worked Example for Prim and Kruskal

Both algorithms are now run on the nine-node graph below; their executions differ, but they output the same tree.



Bold edges are the MST found by either algorithm; its total weight is

$$4 + 8 + 1 + 2 + 4 + 2 + 7 + 9 = 37.$$

Each row of the table below extracts the minimum-key vertex, relaxes its white neighbours, and adds the resulting tree edge; keys that are not mentioned stay unchanged.

Worked Example: Prim from a

Step	Extract u	Key updates	Tree edge
1	a (key 0)	b:4, h:8	— (root)
2	b (4)	c:8 (h stays 8)	(a, b) 4
3	h (8)	g:1, i:7	(a, h) 8
4	g (1)	f:2	(h, g) 1
5	f (2)	c:4 (8→4), e:10, d:14	(g, f) 2
6	c (4)	i:2 (7→2), d:7 (14→7)	(f, c) 4
7	i (2)	none (h, g already black)	(c, i) 2
8	d (7)	e:9 (10→9)	(c, d) 7
9	e (9)	none	(d, e) 9

Total 37. Note the tie at step 3: c and h both had key 8; extracting h first is what makes (a, h) the tree edge, and either choice leads to the same final cost because the weights are not distinct.

Edges are scanned in increasing weight order; an edge is accepted exactly when its endpoints lie in different components.

Worked Example: Kruskal

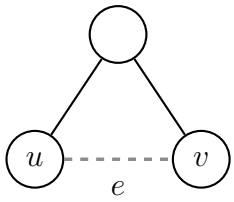
Weight	Edge	Sets of its endpoints	Decision
1	(g, h)	$\{g\}, \{h\}$	accept
2	(c, i)	$\{c\}, \{i\}$	accept
2	(f, g)	$\{f\}, \{g, h\}$	accept
4	(a, b)	$\{a\}, \{b\}$	accept
4	(c, f)	$\{c, i\}, \{f, g, h\}$	accept
6	(g, i)	same set	reject (cycle)
7	(c, d)	$\{c, i, f, g, h\}, \{d\}$	accept
7	(h, i)	same set	reject (cycle)
8	(a, h)	$\{a, b\}, \{c, i, f, g, h\}$	accept
8	(b, c)	same set	reject (cycle)
9	(d, e)	$\{a, \ldots, h\}, \{e\}$	accept
10	(e, f)	same set	reject (cycle)
11	(b, h)	same set	reject (cycle)
14	(d, f)	same set	reject (cycle)

Eight edges are accepted and the tree stops growing exactly when $|V| - 1 = 8$ edges are in T; total 37, the same tree as Prim. ■

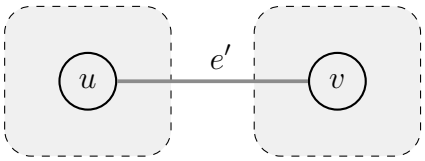
8.1.7 Kruskal's Algorithm

Idea. Build a forest and merge components.

1. Start with the empty tree T.
2. Consider edges in increasing order of weight.
3. Case 1: if adding e to T creates a cycle, discard e.
4. Case 2: otherwise insert $e = (u, v)$ into T, justified by the Cut Lemma.



Case 1: e closes a cycle, discard it



Case 2: e' joins two components, add it

Key question: how to test quickly whether adding e creates a cycle?

- Using DFS for each edge gives $O(|E| \cdot |V|)$ total time — too slow.
- Observation: the internal shape of each component of T does not matter. Each component is just a set of nodes, and adding an edge unions two such sets.
- Aim for $O(\log |V|)$ per test with a union-find structure.

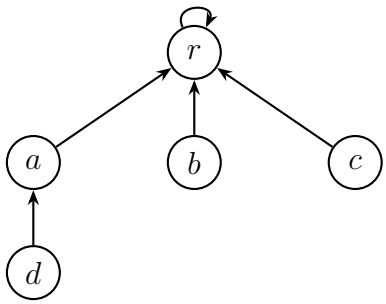
8.1.8 The Union-Find Data Structure

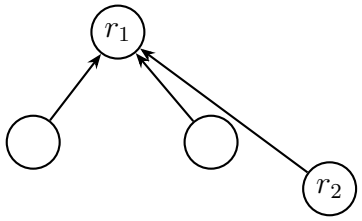
The structure maintains a collection of disjoint sets and supports

- FIND-SET(u): report the set containing u;
- UNION(u,v): merge the sets containing u and v.

Representation. Each set is a rooted tree whose root is the representative of the set; the root's ID is the set's ID. Every node except the root stores one parent pointer, and the root points to itself. There are no child pointers, so a node may have arbitrarily many children. These trees are not the partial MST trees: they only encode which nodes belong together.

root = representative





shorter root
becomes a child

Algorithm 57 Union-Find with Union by Height

```
1: procedure MAKE-SET(x)
2:     x.parent ← x;   x.height ← 0
3: end procedure
4: procedure FIND-SET(x)
5:     while x ≠ x.parent do
6:         x ← x.parent
7:     end while
8:     return x
9: end procedure
10: procedure UNION(x,y)
11:     a ← FIND-SET(x);   b ← FIND-SET(y)
12:     if a.height ≤ b.height then
13:         if a.height = b.height then
14:             b.height ← b.height + 1
15:         end if
16:         a.parent ← b
17:     else
18:         b.parent ← a
19:     end if
20: end procedure
```

Naively setting $x.parent \leftarrow y$ can build a long chain of height $|V|$, so every operation costs $O(|V|)$. **Union by height** makes the root of the taller tree the parent of the shorter one.

Union-Find runs in $O(\log n)$ per operation

Claim: before any UNION(x,y), a tree of height h contains at least 2^h nodes, and the claim is preserved by every union.

- Base: a new set has height 0 and size $1 \geq 2^0$.
- Case $h(x) < h(y)$ with $h(x') = h(y)$:

$$\text{size}(x') = \text{size}(x) + \text{size}(y) \geq 2^{h(x)} + 2^{h(y)} \geq 2^{h(y)} = 2^{h(x')}.$$

- Case $h(x) = h(y)$ with $h(x') = h(y) + 1$:

$$\text{size}(x') = \text{size}(x) + \text{size}(y) \geq 2^{h(x)} + 2^{h(y)} = 2^{h(y)+1} = 2^{h(x')}.$$

- Case $h(x) > h(y)$ is symmetric to the first case.

A tree with n nodes therefore has height h with $2^h \leq n$, that is $h \leq \log_2 n$, so FIND-SET and UNION both cost $O(\log n)$. ■

Path compression

After FIND-SET(x) we walk from x to the root r and now know that all these nodes belong to the set represented by r. Redirecting every visited node's parent pointer straight to r makes future operations faster. The running time becomes practically constant (though theoretically not constant); the details are in the textbook and are not required here.

Algorithm 58 MST-Kruskal(G)

```
1: for each vertex v ∈ V do
2:     Make-Set(v)
3: end for
4: sort the edges of G into increasing order by weight
5: for each edge (u, v) ∈ E in this order do
6:     if Find-Set(u) ≠ Find-Set(v) then
7:         output the edge (u, v)
8:         Union(u,v)
9:     end if
10: end for
```

Running time:

$$O\big(|E|\log|E| + |E|\log|V|\big) = O(|E|\log|V|),$$

the first term for sorting and the second for $|E|$ union-find operations. If the edges are already sorted and path compression is used, the running time is close to $O(|E|)$.

State of the art

An algorithm by Seth and Ramachandran (2002) has been shown to be optimal, but its running time is still unknown. In practice Kruskal's algorithm with union-find dominates on sparse graphs, while Prim with a binary heap is usually preferred on dense graphs.

8.1.9 Correctness of Kruskal's Algorithm

Simplifying assumption: all edge weights are distinct, so the Cut Lemma guarantees that the cheapest cut edge lies in every MST.

Case 1. Adding e creates a cycle: both endpoints already lie in one component, so e cannot belong to any spanning tree of G contained in the current forest plus remaining edges; discarding it is safe.

Case 2. Adding $e = (u, v)$ joins two different components. Let S be the vertex set of one of the two components, and consider the cut $(S, V - S)$. Every edge crossing this cut and cheaper than e was already considered and rejected because both of its endpoints were inside a single component; hence e is the cheapest edge crossing this cut. By the Cut Lemma, e belongs to the MST, so accepting it is safe.

Since each decision preserves the invariant "the chosen edges are contained in some MST", the final set of $|V| - 1$ edges is an MST. ■

8.1.10 Exercises on Minimum Spanning Trees

Exercise: Cheapest Edge at a Vertex

Problem. Suppose (u, v) is an edge of a graph G and it has the least cost among all edges incident to v. Assuming all edge weights are distinct, does every MST contain (u, v)?

Solution. Yes. Let T be an MST with $(u, v) \notin T$. Adding (u, v) to T creates a cycle. Let (z, v) be the other edge of this cycle incident to v. Since (u, v) is the cheapest edge at v, $w(z, v) > w(u, v)$. Replacing (z, v) by (u, v) keeps T a spanning tree and lowers its cost, contradicting optimality of T. ■

Exercise: Extremal Edges

Problem. Let G be a connected undirected graph with distinct edge weights, let $e_{\max}$ be the edge of maximum weight and $e_{\min}$ the edge of minimum weight. Which statements are false?
(1) Every MST of G must contain $e_{\min}$.
(2) No MST contains $e_{\max}$.
(3) If $e_{\max}$ lies in an MST, then its removal disconnects G.
(4) G has a unique MST.

Solution. Statements (1), (3) and (4) are true, so (2) is false.

- (1) $e_{\min}$ is the cheapest edge at both of its endpoints, so every MST contains it.
- (4) With distinct weights the MST is unique: if two MSTs differed, an exchange argument on the heaviest edge of their symmetric difference would strictly lower one of them.
- (3) $e_{\max}$ is a bridge of G; otherwise it lies on a cycle, and deleting it from that cycle would give a cheaper spanning tree not containing it.
- (2) is false because a graph can have a bridge of maximum weight, for example a bridge connecting two dense cheap components; then every spanning tree must use $e_{\max}$.

■

Exercise: Maximum Spanning Tree

Problem. Design an algorithm that finds a spanning tree maximizing the sum of edge weights.

Solution. Three equivalent approaches:

- **Prim.** Start from any node and, at each step, add the heaviest edge leaving the explored set.
- **Kruskal.** Sort the edges in decreasing order of weight and keep adding edges that do not create a cycle.
- **Reduction.** Negate every weight and run any MST algorithm; the tree that is minimum for $-w$ is maximum for w.

All three run in $O(|E| \log |V|)$. ■

Exercise: Bottleneck Weight

Problem. Let $G = (V, E)$ be a connected undirected graph with edge weights. The bottleneck weight of a spanning tree T is the maximum weight of an edge in T. Prove that the MST minimizes the bottleneck weight over all spanning trees.

Solution. By contradiction. Let T be an MST and let (a, b) be its heaviest edge. Removing (a, b) from T splits T into components A (containing a) and B (containing b). Suppose some spanning tree T' has a smaller bottleneck weight. In T' there is a unique path from a to b, so that path contains an edge (a', b') with $a' \in A$ and $b' \in B$. Since the heaviest edge of T outweighs every edge of T', we have $w(a', b') < w(a, b)$. Adding (a', b') to T and deleting (a, b) yields a spanning tree of strictly smaller total weight, contradicting that T is an MST. Hence no spanning tree has a smaller bottleneck weight. ■

8.2 Shortest Paths

8.2.1 The Shortest Path Problem

Input. A directed graph $G = (V, E)$, a real-valued weight $w(e)$ on every edge (an undirected edge is viewed as two directed edges), a source s and a destination t. Weights are allowed to be negative.

Output. The shortest s–t path. In the single-source version we want a shortest path from s to every vertex.

Definition. $\delta(u, v)$ denotes the total weight of a shortest path from u to v, with $\delta(u, v) = \infty$ when v is unreachable from u.

Negative-weight cycles

If G contains a cycle C of total weight $w(C) < 0$ that lies on some s–t path, the shortest path problem is not well defined: walking around C again and again produces s–t walks of arbitrarily negative weight. We therefore always assume that the graph has no negative-weight cycle. In that case every shortest path is simple and has at most $|V| - 1$ edges, so $\delta(s, v)$ is always attained by a simple path.

Algorithm	Problem	Time	Space
Bellman–Ford	single source	$O(\|V\|\|E\|)$	$O(\|V\|)$
DAG shortest path	single source	$O(\|V\| + \|E\|)$	$O(\|V\|)$
Dijkstra	single source, $w \geq 0$	$O(\|E\| \log \|V\|)$	$O(\|V\|)$
Repeated Dijkstra	all pairs, $w \geq 0$	$O(\|V\|\|E\| \log \|V\|)$	$O(\|V\|^2)$
Repeated Bellman–Ford	all pairs	$O(\|V\|^2\|E\|)$	$O(\|V\|^2)$
Floyd–Warshall	all pairs	$O(\|V\|^3)$	$O(\|V\|^2)$

All single-source algorithms use adjacency lists; all-pairs algorithms use an adjacency matrix. Only additional space is listed; the graph itself is not counted.

8.2.2 Subpath Optimality and Relaxation

Lemma (cut-and-paste). Let $P = (s, \ldots, u, \ldots, t)$ be a shortest s–t path. Then the subpath $P_1 = (s, \ldots, u)$ is a shortest s–u path, and the subpath $P_2 = (u, \ldots, t)$ is a shortest u–t path.

Proof. Suppose P_1 is not a shortest s–u path, so some path P_1' from s to u is strictly shorter. Replacing P_1 by P_1' inside P produces an s–t path $P_1'P_2$ that is strictly shorter than P, contradicting the choice of P. The same argument applies to P_2. ■

Relaxation. Maintain estimates $v.d$, the length of the best s–v path found so far, and $v.p$, the predecessor of v on that path. Relaxing the edge (u, v) tests whether the best known path to u followed by (u, v) improves the best known path to v:

```
Relax(u, v)
if u.d + w(u, v) < v.d then
    v.d ← u.d + w(u, v)
    v.p ← u
```

Initially $v.d = \infty$ for every v, except $s.d = 0$. Distances only ever decrease, and at every moment $v.d \geq \delta(s, v)$; a vertex is settled once $v.d = \delta(s, v)$.

8.2.3 Bellman–Ford: Relax Everything $|V|-1$ Times

Idea. Relax all edges once, in an arbitrary order. Afterwards $v.d$ is correct for all vertices reachable by a path of at most 1 edge. Relaxing all edges a second time makes $v.d$ correct for all paths of at most 2 edges, and in general after the i-th pass $v.d$ is the shortest length of a path from s to v using at most i edges: the $(i-1)$-th pass has already computed the correct value of every predecessor used by the i-th pass. Since a shortest path never needs more than $|V|-1$ edges, $|V|-1$ passes suffice.

Algorithm 59 BELLMAN-FORD(G, s)

```
1: for each v ∈ V do
2:     v.d ← ∞; v.p ← NIL
3: end for
4: s.d ← 0
5: for i = 1 to |V| − 1 do
6:     for each (u, v) ∈ E do
7:         if u.d + w(u, v) < v.d then
8:             v.d ← u.d + w(u, v); v.p ← u
9:         end if
10:    end for
11: end for
```

Analysis: $\Theta(|V||E|)$ time and $\Theta(|V|)$ space.

Worked example. Vertices s, a, b, c with the path $s \to a \to b \to c$ of weights $1, 2, 3$, and the edges scanned in the order $(b, c), (a, b), (s, a)$. One pass fixes only a, the second pass fixes b, and the third pass fixes c:

After	$s.d$	$a.d$	$b.d$	$c.d$
initialisation	0	∞	∞	∞
pass 1	0	1	∞	∞
pass 2	0	1	3	∞
pass 3	0	1	3	6

Bellman–Ford as dynamic programming. Define $v.d^{(i)}$ as the shortest length of an s–v path with at most i edges. If (u, v) is the last edge of such a path, the prefix is a shortest s–u path with at most $i-1$ edges, hence

$$v.d^{(i)} = \min_{(u,v)\in E} \big\{ u.d^{(i-1)} + w(u, v) \big\}, \qquad v.d^{(0)} = \infty.$$

The final answer is $v.d^{(|V|-1)}$. The code above stores a single array $v.d$ instead of all layers; after pass i we have $v.d \le v.d^{(i)}$, which is enough because distances only decrease and the values are correct at the end.

Faster practical implementation

Scan only the outgoing edges of vertices whose distance changed in the previous pass, and stop early when a whole pass changes nothing. The worst case is still $O(|V||E|)$, but on real graphs the algorithm is often much faster. It can be run in parallel, it is used on very large graphs even when all weights are nonnegative, and it also supports negative weights.

Detecting a negative-weight cycle

Run Bellman–Ford for the usual $|V|-1$ passes, then perform one extra relaxation

pass over all edges. If some edge (u, v) still satisfies $u.d + w(u, v) < v.d$, no shortest path exists and the graph contains a negative-weight cycle; otherwise the computed distances are correct.

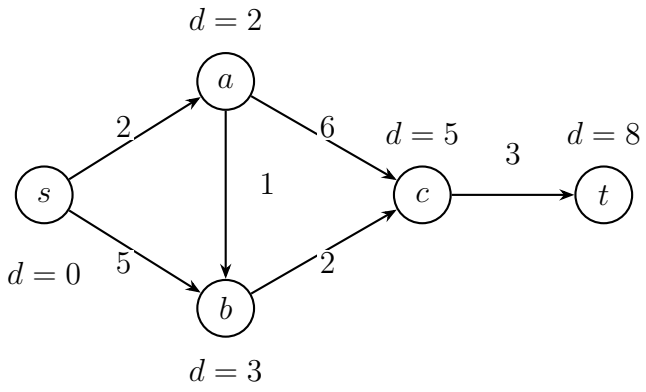
8.2.4 Shortest Paths in a DAG

If G is acyclic we can settle each vertex once, provided all of its in-neighbours are settled first. A topological order does exactly that, because every edge points forward in that order.

Algorithm 60 DAG-Shortest-Paths(G, s)

```
topologically sort the vertices of G
for each v ∈ V do
    v.d ← ∞; v.p ← NIL
end for
s.d ← 0
for each u in topological order do
    for each v ∈ Adj[u] do
        if v.d > u.d + w(u, v) then
            v.d ← u.d + w(u, v); v.p ← u
        end if
    end for
end for
```

Every edge is relaxed exactly once, so the running time is $\Theta(|V| + |E|)$ including the topological sort. The actual path is recovered by following the parent pointers, and if only s–t is wanted one may stop at t without changing the asymptotic bound.



Relaxing in the topological order s, a, b, c, t gives $a.d = 2$, $b.d = \min(5,\ 2 + 1) = 3$, $c.d = \min(2 + 6,\ 3 + 2) = 5$ and $t.d = 5 + 3 = 8$.

Exercise: Longest Path in a DAG

Problem. Given a DAG with real weights and vertices s, t, find a longest simple path from s to t.
Solution. On a DAG every path is simple, so the same topological-order argument applies with max instead of min. Let $ld[v]$ be the weight of the longest s–v path, with $ld[s] = 0$ and $ld[v] = -\infty$ initially. Processing vertices in topological order,

$$ld[v] = \max\big\{ld[u] + w(u, v) \ :\ (u, v) \in E\big\},$$

and the answer is $ld[t]$. The running time is $\Theta(|V| + |E|)$. ■

Exercise: Critical Paths

Problem. A DAG models jobs: an edge (u, v) means that job u must finish before job v starts, and vertex v carries the positive time $t(v)$ needed to complete it. Find the minimum total time to finish all jobs.
Solution. Build G' by adding a new source s with an edge to every vertex of in-degree 0, and give every edge (u, v) the weight $t(v)$. The earliest finishing time of v is then the longest s–v path in G', and the minimum total time is the longest path from s. Computing it in topological order takes $\Theta(|V| + |E|)$ time. For the jobs $a(5)$, $b(7)$, $c(3)$, $d(4)$, $f(6)$ with $a \to d$, $c \to d$, $d \to f$, the longest path is $s \to a \to d \to f$ with weight $5 + 4 + 6 = 15$; job b finishes earlier and does not delay anything. ■

Exercise: Counting Distinct Paths

Problem. In a DAG whose topological order starts at s and ends at t, count the number of distinct s–t paths without listing them.
Solution. Let $cnt[u]$ be the number of distinct s–u paths, with $cnt[s] = 1$ and $cnt[u] = 0$ otherwise. In topological order, for every edge (u, v) set $cnt[v] \leftarrow cnt[v] + cnt[u]$; each path to v is obtained from a unique path to some predecessor followed by the last edge. The answer is $cnt[t]$ and the running time is $\Theta(|V| + |E|)$. ■

8.2.5 Dijkstra's Algorithm

Dijkstra's algorithm solves the single-source problem when all weights are nonnegative. It grows a set S of settled vertices out of s, exactly like Prim's MST algorithm but with cumulative distances as keys instead of edge weights.

Key lemma. Assume $u.d = \delta(s, u)$ for every $u \in S$, and that all edges leaving S have already been relaxed. Let v be the vertex of $V - S$ with the smallest $v.d$. Then $v.d = \delta(s, v)$.

Proof. Since $v.d$ is always the length of some real s–v path, $v.d \geq \delta(s, v)$. Suppose $v.d > \delta(s, v)$ and let P be a shortest s–v path. Let $x \to y$ be the first edge of P that leaves S, so $x \in S$ and $x.d = \delta(s, x)$. That edge has been relaxed, hence $y.d \leq x.d + w(x, y) = \delta(s, x) + w(x, y)$. Because P is shortest, its prefix to y is shortest too, so the last expression equals $\delta(s, y)$. Since all weights are nonnegative, $\delta(s, y) \leq \delta(s, v)$. Therefore

$$v.d > \delta(s, v) \geq \delta(s, y) \geq y.d,$$

contradicting the choice of v as the vertex of $V - S$ with minimum $v.d$. ■

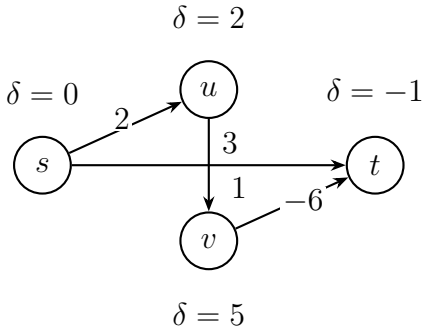
Algorithm 61 DIJKSTRA(G, s) (with Decrease-Key)

```
 1: for each v ∈ V do
 2:     v.d ← ∞; v.p ← NIL; v.color ← white
 3: end for
 4: s.d ← 0
 5: insert all vertices into a min-priority queue Q with key v.d
 6: while Q ≠ ∅ do
 7:     u ← EXTRACT-MIN(Q); u.color ← black
 8:     for each v ∈ Adj[u] do
 9:         if v.color = white and u.d + w(u, v) < v.d then
10:             v.d ← u.d + w(u, v); v.p ← u
11:             DECREASE-KEY(Q, v, v.d)
12:         end if
13:     end for
14: end while
```

Running time: $O(|E| \log |V|)$ with a binary heap, plus $O(|V|)$ for initialisation. A

lazy variant inserts a new heap entry whenever a distance improves and discards an extracted vertex whose colour is already black; the bound is $O(|E| \log |V|)$ as well and no DECREASE-KEY is needed.

Why the distances form a tree. Every vertex except s receives its final distance together with a parent pointer, and the parent is always settled earlier. Hence the parent pointers form a shortest-path tree rooted at s: for each v the unique tree path from s to v has weight $\delta(s, v)$.



Dijkstra settles s, then u with 2, then v with 5, then t with 1, and reports $\delta(s, t) = 1$; the true shortest path $s \to u \to v \to t$ has weight -1. Negative weights break the key lemma because $\delta(s, y) \le \delta(s, v)$ may fail.

Re-weighting does not repair this. Adding a constant c to every edge weight preserves the ordering of paths with the same number of edges only. Paths with different edge counts change by different amounts, so the shortest path can change: in the example above, adding 3 to every weight favours the two-edge path $s \to t$ and makes Dijkstra return the wrong pair.

Most reliable path

Let $r(u, v)$ be the probability that the channel (u, v) works, with independent failures. The reliability of a path is the product of its edge probabilities. Replace add by multiply, min by max, and use a max-heap with $d[s] = 1$, $d[u] = 0$ otherwise:

$$d[v] \leftarrow \max\bigl\{d[u] \cdot r(u, v)\bigr\}.$$

The same correctness proof works because probabilities multiply monotonically along a path.

A* search

If the edge weights are lengths and $E(u, t)$ is the Euclidean distance from u to t, then $E(u, t) \le \delta(u, t)$. A* orders the heap by $f(u) = d(u) + E(u, t)$ instead of $d(u)$, which guides the search towards t and usually settles far fewer vertices while keeping the same asymptotic bound. Bidirectional Dijkstra runs two searches, from s and from t, and combines them at a common vertex; it can be combined with A*.

8.2.6 All-Pairs Shortest Paths

Input. A directed graph with an adjacency matrix representation, so that $w(u, v)$ is available in $O(1)$ time; $w(u, u) = 0$ and $w(u, v) = \infty$ if there is no edge. **Output.** $\delta(u, v)$ for all pairs, plus enough information to reconstruct each path. Storing all paths explicitly would need $\Theta(|V|^3)$ space.

- If $w \ge 0$, run Dijkstra once per source: $O(|V||E| \log |V|)$ time, which is $O(n^3 \log n)$ on a dense graph.

- If weights may be negative, run Bellman–Ford once per source: $O(|V|^2|E|)$ time, which is $O(n^4)$ on a dense graph.

DP by the number of edges. Let $d_{ij}^{(m)}$ be the shortest length of a path from i to j using at most m edges, and let $D^{(m)} = (d_{ij}^{(m)})$. Looking at the last edge gives

$$d_{ij}^{(m)} = \min_{1 \le k \le n} \bigl\{ d_{ik}^{(m-1)} + w(k,j) \bigr\}, \qquad d_{ij}^{(1)} = w(i,j).$$

Computing all $n-1$ layers costs $O(n^4)$ time and $O(n^3)$ space (easily reduced to $O(n^2)$ by keeping two layers). The goal is $D^{(n-1)}$.

DP by doubling. Instead of extending paths by one edge, concatenate two halves:

$$d_{ij}^{(2s)} = \min_{1 \le k \le n} \bigl\{ d_{ik}^{(s)} + d_{kj}^{(s)} \bigr\}.$$

Only $O(\log n)$ layers $D^{(1)}, D^{(2)}, D^{(4)}, \ldots$ are needed, each costing $O(n^3)$, for $O(n^3 \log n)$ time. Overshooting $n-1$ is harmless because no shortest path has more than $n-1$ edges, so $D^{(m)} = D^{(n-1)}$ for every $m \ge n-1$.

8.2.7 Floyd–Warshall

Definition. $d_{ij}^{(k)}$ is the shortest length of a path from i to j whose intermediate vertices all lie in $\{1, 2, \ldots, k\}$ (the endpoints i, j are unrestricted).

Recurrence. For the pair (i, j) there are two cases with respect to vertex k:

- k is not an intermediate vertex: the path only uses $\{1, \ldots, k-1\}$, giving $d_{ij}^{(k-1)}$;
- k is an intermediate vertex: the path splits at k into a shortest i–k path and a shortest k–j path, both using only $\{1, \ldots, k-1\}$, giving $d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$.

Hence

$$d_{ij}^{(k)} = \min\bigl\{ d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \bigr\}, \qquad d_{ij}^{(0)} = w(i,j),$$

and the answer is $D^{(n)}$.

Algorithm 62 FLOYD-WARSHALL(G)

```
1: for all 1 ≤ i, j ≤ n do
2:     d_ij ← w(i, j); intermed[i, j] ← NIL
3: end for
4: for k = 1 to n do
5:     for i = 1 to n do
6:         for j = 1 to n do
7:             if d_ik + d_kj < d_ij then
8:                 d_ij ← d_ik + d_kj; intermed[i, j] ← k
9:             end if
10:        end for
11:    end for
12: end for
```

In-place computation. The superscripts may be dropped and a single $n \times n$ matrix D used. During iteration k, the values d_{ik} and d_{kj} read by the update are already the values of layer k, but for those entries vertex k is an endpoint rather than an intermediate vertex, so the two layers agree: $d_{ik}^{(k)} = d_{ik}^{(k-1)}$ and $d_{kj}^{(k)} = d_{kj}^{(k-1)}$. Thus overwriting D is safe. The running time is $O(n^3)$ and the space is $O(n^2)$.

Worked example. Five vertices with the weight matrix

$$W = D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}.$$

After all five iterations,

$$D^{(5)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}.$$

For instance $d_{3,5}$ develops as follows: $d^{(0)}_{3,5} = d^{(1)}_{3,5} = \infty$; with $k = 2$ the path $3 \to 2 \to 5$ appears, $d^{(2)}_{3,5} = 4 + 7 = 11$; $k = 3$ changes nothing; with $k = 4$ the path $3 \to 2 \to 4 \to 1 \to 5$ gives $4 + 1 + 2 - 4 = 3$, so $d^{(4)}_{3,5} = 3$ and $d^{(5)}_{3,5} = 3$. Negative edges are therefore handled correctly even though Dijkstra could not be used.

Reconstructing a shortest path

$intermed[i, j]$ stores one intermediate vertex of the chosen shortest path, or NIL when the path is the single edge (i, j). Then PATH(i, j) prints (i, j) if $intermed[i, j]$ is NIL, and otherwise recurses on $(i, intermed[i, j])$ and $(intermed[i, j], j)$. The reconstruction takes time proportional to the number of edges of the path.

Negative cycles and transitive closure

- **Negative cycles.** If the graph has a negative cycle C, let k be its largest-numbered vertex and i another vertex of C. Then $d^{(k)}_{ii} < 0$, so a negative cycle exists exactly when some diagonal entry of D becomes negative during Floyd–Warshall. Abort as soon as this happens.
- **Transitive closure.** For an unweighted graph set $w(i, j) = 1$ on edges and $w(i, j) = \infty$ otherwise; then $a^*_{ij} = 1$ iff $d^{(n)}_{ij} < \infty$. The same $O(n^3)$ algorithm works with Boolean operations, $T^{(k)}_{ij} = T^{(k-1)}_{ij} \vee \left(T^{(k-1)}_{ik} \wedge T^{(k-1)}_{kj}\right)$, which is faster in practice and needs less space.

8.2.8 Exercises on Shortest Paths

Exercise: Negative Cycle Detection

Problem. Show how Bellman–Ford detects a negative-weight cycle.
Solution. Without a negative cycle every shortest path has at most $|V| - 1$ edges, so $|V| - 1$ passes are enough. If a cycle of negative weight exists, distances keep decreasing forever; adding one extra pass over all edges and testing $u.d + w(u, v) < v.d$ therefore reports a negative cycle whenever such an edge is still improvable. ■

Exercise: Most Reliable Path

Problem. In a communication network every edge (u, v) has an independent reliability $r(u, v) \in (0, 1)$. Find the most reliable path from s to every vertex.
Solution. Set $d[s] = 1$ and $d[u] = 0$ for $u \neq s$, use a max-heap, and replace the

relaxation by $d[v] \leftarrow \max\{d[u] \cdot r(u,v)\}$ with an increase-key. The proof is identical to Dijkstra's, with products in place of sums: along any path the reliability only decreases, and the first edge leaving the settled set is the one that fixes the value of the extracted vertex. ■

Exercise: Transitive Closure

Problem. Given the adjacency matrix of an unweighted directed graph, compute G^* in which $a^*_{ij} = 1$ iff there is a path from i to j.

Solution. Either run Floyd–Warshall with $w(i,j) = 1$ for edges and ∞ otherwise and set $a^*_{ij} = 1$ iff $d_{ij} < \infty$, or use the Boolean recurrence $T^{(k)}_{ij} = T^{(k-1)}_{ij} \vee (T^{(k-1)}_{ik} \wedge T^{(k-1)}_{kj})$; both take $\Theta(n^3)$ time. ■

8.3 Maximum Flow

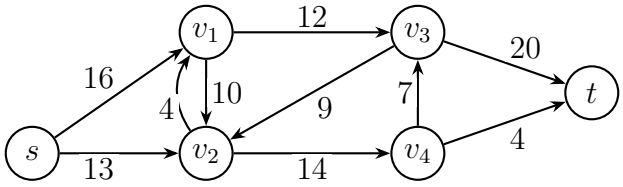
8.3.1 Flows and Cuts

Input. A directed graph $G = (V, E)$ with a nonnegative integer capacity $c(e)$ on every edge, a source s and a sink t. All capacities are integers; the algorithms below then produce integer flows.

Definition (flow). A flow is a function $f : E \to \mathbb{R}$ with

- **capacity constraints**: $0 \le f(e) \le c(e)$ for every edge e;
- **conservation**: for every $v \in V - \{s, t\}$, the total flow entering v equals the total flow leaving v.

The value of a flow is $|f| = \sum_{e \text{ out of } s} f(e) - \sum_{e \text{ into } s} f(e)$; by conservation this equals the net flow into t.



A network with capacities. The maximum flow from s to t has value 23.

Cuts. An s–t cut is a partition (S, T) of V with $s \in S$ and $t \in T$. Its capacity is

$$\text{cap}(S, T) = \sum_{\substack{(u,v) \in E \\ u \in S,\ v \in T}} c(u, v),$$

the sum of the capacities of the edges that go from S to T; edges crossing from T back to S are not counted.

Net flow lemma. For any flow f and any cut (S, T),

$$f(S, T) = \sum_{\substack{(u,v) \in E \\ u \in S,\ v \in T}} f(u, v) - \sum_{\substack{(u,v) \in E \\ u \in T,\ v \in S}} f(u, v) = |f|.$$

Proof. Sum the conservation equations over all $v \in S - \{s\}$. Every edge inside S cancels (flow out of one vertex is flow into another), so only edges crossing the cut survive, and the sum equals the net flow out of s, namely $|f|$. ∎

Corollary (weak duality). For every flow f and every cut (S, T),

$$|f| = f(S, T) \le \text{cap}(S, T),$$

because each crossing edge contributes at most its capacity. Hence max flow ≤ min cut.

8.3.2 Ford–Fulkerson

Residual graph. Given f, the residual capacity of an edge (u, v) is $c_f(u, v) = c(u, v) - f(u, v)$: the extra flow it can still carry forward. Pushing flow back along (u, v) is equivalent to cancelling existing flow, so the residual graph also contains the reverse edge (v, u) with residual capacity $c_f(v, u) = f(u, v)$. A residual edge may therefore have capacity even when G has no corresponding forward edge.

Augmenting path. An s–t path in the residual graph. Its bottleneck is the minimum residual capacity on the path, $\beta = \min_{e \in P} c_f(e)$, and pushing β units along P increases $|f|$ by β while preserving all constraints.

Algorithm 63 FORD-FULKERSON(G, s, t)

```
for each e ∈ E do
    f(e) ← 0
end for
while there is an s–t path P in the residual graph G_f do
    β ← min_{e∈P} c_f(e)
    augment β units of flow along P
    update the residual graph
end while
```

Worked example. On the network above, three augmentations already reach the maximum:

Step	Augmenting path	Bottleneck	$\lvert f\rvert$ after
1	$s \to v_1 \to v_3 \to t$	12	12
2	$s \to v_2 \to v_4 \to v_3 \to t$	7	19
3	$s \to v_2 \to v_4 \to t$	4	23

The resulting flow values (flow / capacity) are

(s,v_1)	(s,v_2)	(v_1,v_3)	(v_2,v_4)	(v_4,v_3)	(v_3,t)	(v_4,t)
12/16	11/13	12/12	11/14	7/7	19/20	4/4

while (v_1,v_2), (v_4,v_1) and (v_3,v_2) carry 0. The cut $S = \{s, v_1, v_2, v_4\}$, $T = \{v_3, t\}$ has capacity $12 + 7 + 4 = 23$, so no flow can be larger.

Max-flow min-cut theorem

The following statements are equivalent:
1. f is a maximum flow;
2. there is no augmenting path in the residual graph G_f;
3. $\lvert f\rvert = \operatorname{cap}(S,T)$ for some cut (S,T).

Consequently $\max_f \lvert f\rvert = \min_{(S,T)} \operatorname{cap}(S,T)$.

Proof of (2) $\Rightarrow$ (3)**.** Suppose no s–t path exists in G_f and let S be the set of vertices reachable from s in G_f, with $T = V - S$. For every edge (u,v) with $u \in S$ and $v \in T$, the residual capacity is 0, i.e. $f(u,v) = c(u,v)$ (the edge is saturated). For every edge (u,v) with $u \in T$ and $v \in S$, the fact that v is reachable but u is not forces $f(u,v) = 0$. Therefore

$$\lvert f\rvert = \sum_{u \in S, v \in T} f(u,v) - \sum_{u \in T, v \in S} f(u,v) = \sum_{u \in S, v \in T} c(u,v) = \operatorname{cap}(S,T).$$

■

8.3.3 Running Time and Edmonds–Karp

With integer capacities, Ford–Fulkerson maintains an integral flow throughout, because every bottleneck is an integer. If all capacities are integers bounded by U, the number of augmentations is at most $\lvert f^*\rvert \le \lvert E\rvert U$, so the running time with BFS/DFS per augmentation is $O(\lvert E\rvert\,\lvert f^*\rvert) = O(\lvert E\rvert^2 U)$.

Ford–Fulkerson can be very slow

Capacities can be exponentially large in the input size, so $O(|E||f^*|)$ is not polynomial in the input length. Even with small capacities the choice of augmenting paths matters: take capacities $c(s,u) = c(s,v) = c(u,t) = c(v,t) = U$ and $c(u,v) = 1$. Alternating the paths $s \to u \to v \to t$ and $s \to v \to u \to t$ augments only 1 unit each time and needs $2U$ augmentations, although the maximum flow $2U$ could be found in two steps.

Edmonds–Karp. Always choose an augmenting path with the fewest edges, found by BFS in the residual graph. Then the shortest augmenting-path distance strictly increases after $O(|E|)$ augmentations, which bounds the total number of augmentations by $O(|V||E|)$ and the running time by $O(|V||E|^2)$. This bound holds for arbitrary real capacities.

Better algorithms

Push–relabel labels vertices with heights and runs in $O(|V|^2|E|)$; the highest-label variant runs in $O(|V|^2\sqrt{|E|})$; Dinic's algorithm runs in $O(|V|^2|E|)$ and in $O(|E||V|^{2/3})$ on unit-capacity graphs. Modern algorithms solve max flow in nearly $O(|E|^{1+o(1)})$ time, but they are complicated and rarely used in practice.

8.3.4 Applications of Maximum Flow

Edge-disjoint paths

Given G with s, t, assign capacity 1 to every edge and compute a maximum flow. Integrality forces every edge to carry 0 or 1, so the edges with flow 1 form a set of edge-disjoint s–t paths, and the flow value is their number. The largest number of edge-disjoint s–t paths therefore equals the maximum flow with unit capacities.

Circulations with demands

If each vertex v demands $d(v)$ units (negative $d(v)$ means supply), add an edge (t, s) of infinite capacity and ask for a feasible circulation: conservation at every vertex including s, t. A feasible circulation exists iff the maximum flow saturates all edges out of a super-source attached to the suppliers; the details follow by reducing the problem to an ordinary max-flow instance with lower bounds.

Bipartite matching and scheduling

Maximum bipartite matching, feasible course scheduling and assignment problems all reduce to max flow with unit capacities; this is developed in the next chapter. Baseball elimination can also be phrased as a max-flow feasibility question: a team is eliminated exactly when the remaining games form a cut whose capacity is smaller than the number of wins that team still needs.

8.3.5 Exercises on Maximum Flow

Exercise: Recovering a Minimum Cut

Problem. Given a maximum flow f, find a minimum s–t cut.
Solution. Build the residual graph G_f, run BFS/DFS from s, and let S be the reachable set. Then $(S, V - S)$ is a cut with $\text{cap}(S, V - S) = |f|$, hence it is a minimum cut by weak duality. The computation takes $O(|V| + |E|)$ time after the flow has been found. ■

Exercise: Edge-Disjoint Paths

Problem. Given an undirected graph G with two vertices s, t, find the maximum number of edge-disjoint s–t paths.
Solution. Replace each undirected edge by two opposite directed edges, assign capacity 1 to each directed edge, and compute a maximum flow. Any integral flow uses each edge in at most one direction and decomposes into edge-disjoint paths, and conversely any family of k edge-disjoint paths yields a flow of value k. Hence the answer is the max-flow value, computed in $O(|V||E|^2)$ time with Edmonds–Karp. ■

Exercise: Integrality

Problem. Show that if all capacities are integers, there is a maximum flow that is integral on every edge.
Solution. Ford–Fulkerson starts from the zero flow and always augments by an integer bottleneck, so every intermediate flow is integral. When the algorithm stops, the flow is maximum by the max-flow min-cut theorem. ■

Exercise: Flow Decomposition

Problem. Show that every flow of value k in an acyclic network can be written as the sum of k unit flows along s–t paths, plus a circulation if the network contains cycles.
Solution. Repeatedly trace a path from s to t using only edges with positive flow, decrease the flow on those edges by the bottleneck $\beta > 0$, and repeat. Because flow is conserved, the trace cannot get stuck before reaching t, and each round removes at least one edge from the support, so the process ends after $O(|E|)$ rounds and produces flows along s–t paths. Any remaining flow satisfies conservation at all vertices including s and t, i.e. it is a circulation. ■

8.4 Matchings in Bipartite Graphs

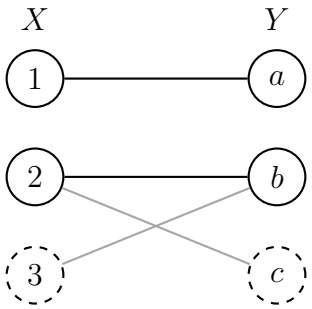
8.4.1 Definitions

A graph $G = (V, E)$ is bipartite if V can be partitioned as $V = X \cup Y$ with $X \cap Y = \emptyset$ and every edge having one endpoint in X and the other in Y. Equivalently, G is bipartite iff it contains no odd cycle. We usually write $n = |X|$, $m = |Y|$ and draw X on the left and Y on the right.

Matchings

- A matching $M \subseteq E$ is a set of edges no two of which share an endpoint.
- Its size is $|M|$; a vertex is matched if it is incident to an edge of M and free otherwise.
- A maximum matching has maximum size among all matchings.
- A perfect matching matches every vertex; it exists only if $|X| = |Y|$ and then $|M| = |X|$.
- A matching is maximal if no edge can be added to it. Every maximum matching is maximal, but the converse is false.

Applications. Assigning jobs to people, tasks to machines, courses to classrooms, medical students to hospital places, and matching players in tournaments are all bipartite matching problems.

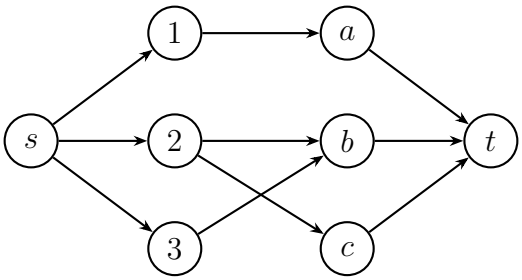


Matching $M = \{(1, a), (2, b)\}$ of size 2 (bold). The free vertices are 3 and c, and $3 \to b \to 2 \to c$ is an augmenting path. Augmenting gives the perfect matching $\{(1, a), (3, b), (2, c)\}$.

8.4.2 Reduction to Maximum Flow

Given a bipartite graph $G = (X \cup Y, E)$, construct a flow network G':

- direct every edge of E from X to Y and give it capacity 1;
- add a source s with an edge of capacity 1 to every vertex of X;
- add a sink t with an edge of capacity 1 from every vertex of Y.



Every edge has capacity 1. A maximum flow of value k corresponds to a matching of size k.

Theorem. The maximum matching size of G equals the maximum flow value in G'.

Proof ($\Rightarrow$). Let M be a matching of size k. Send one unit along each of the $3k$ edges $s \to x$, $x \to y$, $y \to t$ for $(x, y) \in M$. Every capacity constraint holds because each vertex is incident to at most one matching edge, and conservation holds at every vertex of $X \cup Y$; the value is k.

Proof ($\Leftarrow$). Let f be an integral maximum flow of value k, which exists because all capacities are integers. Set $M = \{(x, y) \in E : f(x, y) = 1\}$. If two edges of M shared the vertex x, conservation at x would force inflow ≥ 2 while the edge $s \to x$ carries at most 1, impossible; the same holds for shared y. Hence M is a matching, and $|M| = k$ because every unit of flow enters X through a distinct s-edge and leaves through a distinct t-edge. ■

Running time. Ford–Fulkerson augments at least 1 unit each time, and the flow value is at most $\min(|X|, |Y|) \leq |V|/2$. Finding an augmenting path takes $O(|E|)$ time with BFS, so the total is $O(|V||E|)$.

8.4.3 Augmenting Paths and Berge's Theorem

Let M be a matching. An alternating path is a path whose edges alternate between M and $E - M$. An augmenting path is an alternating path that starts and ends at free vertices; for a bipartite graph it starts at a free vertex of X and ends at a free vertex of Y.

Lemma (augmenting increases the size). If P is an augmenting path, then $M' = M \oplus P$ (the symmetric difference) is a matching and $|M'| = |M| + 1$.

Proof. Along P the edges alternate between unmatched and matched, and both endpoints are free, so P has one more unmatched edge than matched edges. Removing the matched edges of P frees exactly the vertices once matched internally, and adding the unmatched edges matches them again; internal vertices still have exactly one incident edge of M', and the endpoints become matched. No conflict arises outside P because M' agrees with M there. ■

Theorem (Berge). A matching M is maximum if and only if there is no M-augmenting path.

Proof ($\Rightarrow$). If an augmenting path exists, the lemma produces a larger matching.

Proof ($\Leftarrow$). Suppose M is not maximum and let M' be a matching with $|M'| > |M|$. Consider $M \oplus M'$: every vertex is incident to at most one edge of M and at most one of M', so the symmetric difference is a disjoint union of alternating paths and alternating cycles. Cycles contain the same number of edges of M and M'; since $|M'| > |M|$, some path contains more edges of M' than of M, and such a path must start and end with edges of M', i.e. at vertices that are free in M. That path is M-augmenting, a contradiction. ■

Finding an augmenting path. Build the directed graph $D(M)$: orient unmatched edges from X to Y and matched edges from Y to X. Then M-augmenting paths are exactly directed paths from a free vertex of X to a free vertex of Y. A BFS from all free vertices of X finds one in $O(|E|)$ time, and repeating at most $\min(|X|, |Y|)$ times gives the $O(|V||E|)$ algorithm again. This is the same augmentation process as Ford–Fulkerson on G', without building G' explicitly.

Faster matching algorithms

Hopcroft–Karp augments along a maximal set of shortest augmenting paths in each phase and runs in $O(|E|\sqrt{|V|})$. For dense graphs, algebraic algorithms based on matrix multiplication run in $O(|V|^{2.376})$ time; the fastest known combinatorial algorithms run in about $O(|E|\sqrt{|V|})$ and near-linear-time algorithms exist for special graph classes.

8.4.4 Hall's Theorem and Vertex Covers

Theorem (Hall, 1935). A bipartite graph $G = (X \cup Y, E)$ has a matching saturating every vertex of X if and only if for every $S \subseteq X$,

$$|N(S)| \geq |S|,$$

where $N(S)$ is the set of neighbours of S in Y.

Proof ($\Rightarrow$). The edges of a matching saturating X match distinct vertices of $N(S)$ to the $|S|$ vertices of S, so $|N(S)| \geq |S|$.

Proof ($\Leftarrow$). Use the flow network G' above with infinite capacity on the edges $X \to Y$ and compute a minimum cut (A, B). Let $S = X \cap A$. Because edges $X \to Y$ have infinite capacity, no such edge can cross from A to B, so every neighbour of S lies in A. The cut consists of the edges $s \to x$ for $x \in X - S$ plus the edges $y \to t$ for $y \in N(S)$, hence

$$\operatorname{cap}(A, B) = |X - S| + |N(S)| \geq |X| - |S| + |S| = |X|.$$

By max-flow min-cut, the maximum flow has value $|X|$, i.e. there is a matching saturating X. ■

Defect version. For a general bipartite graph, the maximum matching size is

$$|X| - \max_{S \subseteq X} \big(|S| - |N(S)|\big).$$

The same cut argument applies: the minimum cut has capacity $|X| - |S| + |N(S)|$, minimized over S.

König's theorem

In a bipartite graph, the maximum matching size equals the minimum vertex-cover size. A vertex cover is a set of vertices touching every edge. Given a minimum cut (A, B) of G', the set $(X - A) \cup (Y \cap A)$ is a vertex cover of size $\operatorname{cap}(A, B)$, which equals the maximum matching; conversely every vertex cover of size c yields a cut of capacity c. The theorem fails for general graphs: a triangle has maximum matching 1 but minimum vertex cover 2.

8.4.5 Worked Applications

Exercise: Feasible Cooking Schedule

Problem. There are n roommates $r_1, \ldots, r_n$ and n days $d_1, \ldots, d_n$. Roommate r_i is available on day d_j iff $c_{i,j} =$ true. Decide whether a schedule exists in which every roommate cooks exactly once and every day has exactly one cook.
Solution. Build a bipartite graph with $X = \{r_1, \ldots, r_n\}$, $Y = \{d_1, \ldots, d_n\}$ and the edge (r_i, d_j) iff $c_{i,j}$ is true. A feasible schedule is exactly a perfect matching: each roommate is matched to the single day on which he cooks, and each day is matched to its cook. Compute a maximum matching with the flow reduction; a perfect matching exists iff its size is n. The running time is $O(n|E|) \leq O(n^3)$ with the augmenting-path algorithm, or $O(|E|\sqrt{n})$ with Hopcroft–Karp. ■

Exercise: Balanced Assignment

Problem. A company assigns n customers $c_1, \ldots, c_n$ to k facilities $f_1, \ldots, f_k$. Customer c_i can be served only by facilities in his vicinity, and facility f_j can serve at most b_j customers. Decide whether all customers can be served.
Solution. Build a bipartite graph between customers and facilities, with the edge (c_i, f_j) iff f_j can serve c_i. Add a source s with capacity-1 edges to all customers and a sink t with capacity-b_j edges from facility f_j. Each customer-facility edge has capacity

1. All customers can be served iff the maximum flow equals n; by integrality, a flow of value n assigns each customer to exactly one facility and respects every capacity b_j. The running time is $O(|V||E|^2)$ with Edmonds–Karp, or polynomial in the capacities with a scaling algorithm. ■

Exercise: Constrained Assignment

Problem. Each customer c_i must be assigned to exactly a_i facilities among those that can serve him, and facility f_j can serve at most b_j customers. Decide whether this is possible.
Solution. Use the same network with capacity a_i on the edge $s \to c_i$ instead of capacity 1, capacity b_j on $f_j \to t$, and capacity 1 on each customer-facility edge. A feasible assignment exists iff the maximum flow equals $\sum_i a_i$. Integrality guarantees that each customer gets exactly a_i distinct facilities and no facility is overloaded. ■

8.4.6 Exercises on Matchings

Exercise: Maximal but Not Maximum

Problem. Give a bipartite graph and a maximal matching that is not maximum.
Solution. Take $X = \{x_1, x_2\}$, $Y = \{y_1, y_2\}$ and the edges (x_1, y_1), (x_1, y_2), (x_2, y_1). The matching $\{(x_1, y_1)\}$ is maximal because no further edge can be added, but it has size 1 while $\{(x_1, y_2), (x_2, y_1)\}$ has size 2. Hence maximality does not imply maximality of size. ■

Exercise: Hall's Condition Fails

Problem. Show that the bipartite graph with $X = \{x_1, x_2, x_3\}$, $Y = \{y_1, y_2\}$ and the edges (x_1, y_1), (x_2, y_1), (x_3, y_2) has no matching saturating X, and identify the violating set S.
Solution. Take $S = \{x_1, x_2\}$; then $N(S) = \{y_1\}$ and $|N(S)| = 1 < 2 = |S|$, so Hall's condition fails and no matching can saturate X. Indeed at most one of x_1, x_2 can be matched, so every matching leaves a vertex of X free. ■

Exercise: Vertex Cover from a Matching

Problem. In the graph $X = \{1, 2, 3\}$, $Y = \{a, b, c\}$ with edges $(1, a)$, $(2, a)$, $(2, b)$, $(3, b)$, $(3, c)$, find a maximum matching and a minimum vertex cover of the same size.
Solution. The matching $\{(1, a), (2, b), (3, c)\}$ is perfect, so its size is 3. The set $\{a, b, c\}$ is a vertex cover of size 3, and no cover of size 2 can touch all five edges (removing any two of the three vertices of Y leaves an edge uncovered). Thus the maximum matching and minimum vertex cover both have size 3, as König's theorem predicts. ■

8.5 Stable Marriage

8.5.1 The Problem

Input. n men and n women. Each man ranks all women from most to least preferred, and each woman ranks all men from most to least preferred. Preferences are strict: no ties.

Output. A set of n man–woman pairs forming a perfect and stable matching.

- **Perfect**: every man is matched to exactly one woman, and every woman to exactly one man.
- **Stable**: there is no unstable pair, i.e. no unmatched pair (m, w) such that m prefers w to his current partner and w prefers m to her current partner.

An unstable pair could both improve by leaving their partners and pairing with each other, which is exactly the kind of outcome a matching mechanism must avoid.

Applications. The problem was stated in terms of men and women, but the same model describes the matching of medical students to hospital residencies in the US, the assignment of students to programmes in JUPAS, auction mechanisms for sponsored search, and school-choice systems. The Gale–Shapley algorithm earned Shapley the Nobel Prize in Economics in 2012.

Man	1st	2nd	3rd
X	A	B	C
Y	B	A	C
Z	A	B	C

Woman	1st	2nd	3rd
A	Y	X	Z
B	X	Y	Z
C	X	Y	Z

Preference lists used throughout this chapter.

Example: Stable or Not?

Question 1. Is the matching X–C, Y–B, Z–A stable?
Answer. No. The pair X–B is unstable: X ranks B above his partner C, and B ranks X above her partner Y.
Question 2. Is X–A, Y–B, Z–C stable?
Answer. Yes. The only pair that could break it is Z–B; Z ranks B above C, but B prefers Y (and X) to Z, so B would not switch. All other blocking pairs fail for the same reason.
Question 3. Is the stable matching unique?
Answer. No. For the same lists, X–B, Y–A, Z–C is also stable, so an instance may have several stable matchings.

Two facts proved below

- A stable matching always exists for the stable marriage problem, for every choice of preference lists. This is not obvious, since not every matching problem has one (see the roommates problem at the end).
- A stable matching need not be unique, but the Gale–Shapley algorithm always returns the same one, the one optimal for the proposing side.

8.5.2 The Propose-and-Reject Algorithm

Algorithm 64 GALE-SHAPLEY$(M, W, \text{preferences})$

```
initialise every man and every woman to be free
while some man m is free and has not proposed to every woman do
    choose such a man m
    w ← the first woman on m's list to whom m has not yet proposed
    if w is free then
        engage m and w
    else if w prefers m to her current fiancé m′ then
        engage m and w, and set m′ free
    else
        w rejects m
    end if
end while
return the set of engagements
```

Two observations drive the proofs:

1. Men propose in decreasing order of preference: a man walks down his list and never returns to a woman who has rejected him.
2. Once a woman is matched she never becomes free again; she only trades up, replacing her fiancé by a man she prefers.

8.5.3 Correctness

Claim 1 (termination). The algorithm terminates after at most n^2 iterations of the while loop.

Proof. Each man proposes to a given woman at most once, because he always moves forward on his list; there are n men and n women, hence at most $n \cdot n = n^2$ proposals. Every iteration produces one proposal that is never repeated, so the loop runs at most n^2 times. ■

Claim 2 (perfection). When the algorithm terminates, every man and every woman is matched.

Proof. By contradiction, suppose a man m is free at termination. Then m has proposed to every woman. By Observation 2 a woman who has ever received a proposal is matched forever, so every woman is matched. Since there are n women and n men, all men are matched as well, contradicting that m is free. ■

Claim 3 (stability). The returned matching has no unstable pair.

Proof. Suppose m–w is an unstable pair: m prefers w to his final partner, and w prefers m to her final partner. Consider whether m ever proposed to w.

- **Case 1: m never proposed to w.** Since m proposes in decreasing order and stops at his final partner, a woman to whom he never proposed is ranked below his final partner; this contradicts that m prefers w.
- **Case 2: m proposed to w.** Then w rejected m (immediately or later). At the moment of rejection w preferred someone else to m, and by Observation 2 she only trades up afterwards. Hence w's final partner is preferred to m, contradicting that w prefers m to her final partner.

Both cases contradict the assumption, so no unstable pair exists. ■

8.5.4 An $O(n^2)$ Implementation

- Keep the free men in a queue (or stack) and a pointer $next[m]$ to the next woman on m's list.

- Keep arrays $wife[m]$ and $husband[w]$, with value 0 meaning "free"; an engagement updates both entries in $O(1)$ time.
- For each woman, precompute the inverse of her preference list: $rank[w][m]$ is the position of m on w's list. Then the comparison "does w prefer m to m'?" becomes $rank[w][m] < rank[w][m']$, in $O(1)$ time.

Amy's preference	8	3	7	1	4	5	6	2
Inverse list	1	2	3	4	5	6	7	8
Rank of the man	4th	8th	2nd	5th	6th	7th	3rd	1st

Amy prefers man 3 to man 6 because $rank[3] = 2 < 7 = rank[6]$.

Every proposal costs $O(1)$ time, there are at most n^2 proposals, and the preprocessing of the inverse lists costs $O(n^2)$. The total running time is therefore $O(n^2)$, with $O(n^2)$ space for the preference lists.

8.5.5 Optimality for the Proposing Side

Several stable matchings may exist, and Gale–Shapley gives us the freedom to choose which free man proposes next. Remarkably, the order never matters.

Theorem (optimal for the men). For every instance, Gale–Shapley returns a stable matching that is optimal for the men: every man is matched to the best partner he could obtain in any stable matching. Equivalently, it is pessimal for the women: every woman obtains the worst partner she could have in any stable matching.

Proof sketch. Let S^* be the Gale–Shapley matching and let M be any stable matching. Suppose some man is worse off in S^* than in M, and among all pairs (m, w) with m matched to w in M but rejected by w during Gale–Shapley, choose the one whose rejection happens first. When w rejects m, she is engaged to some m', whom she prefers to m. In M, w is matched to m and prefers m' to m, so stability of M forces m' to prefer his M-partner w'' to w. Since m' proposes in decreasing order and ends with a woman worse than w, he must have proposed to w'' and been rejected before he proposed to w; that rejection happens strictly earlier than the rejection of m by w, contradicting the choice of (m, w). Hence no man is worse off in S^* than in M. ■

Woman-optimal matching

The algorithm is not symmetric between the sexes: men propose and women accept or reject. Running Gale–Shapley with the women proposing and the men accepting produces the woman-optimal stable matching, in which every woman obtains her best possible partner over all stable matchings. In general the man-optimal and woman-optimal matchings are different, and every stable matching lies "between" them: each man's partner ranges from his best (man-optimal) to his worst (woman-optimal) possible stable partner.

Example: Man-Optimal versus Woman-Optimal

Problem. For m_1: w_1, w_2; m_2: w_2, w_1; w_1: m_2, m_1; w_2: m_1, m_2, find both optimal stable matchings.
Solution. Men proposing: both men start with their first choices, m_1 proposes to w_1 and m_2 to w_2; both women accept, giving $\{(m_1, w_1), (m_2, w_2)\}$, the man-optimal matching. Women proposing: both women start with their first choices, w_1 proposes to m_2 and w_2 to m_1; both men accept, giving $\{(m_1, w_2), (m_2, w_1)\}$, the woman-optimal matching. Both are stable: in the first matching, m_2 prefers w_2, but w_2 ranks m_1 above m_2; in the second, m_1 prefers w_1, but w_1 ranks m_2 above m_1.

8.5.6 Exercises on Stable Marriage

Exercise: Mutually First Choices

Problem. (a) Must there always exist, in some stable matching, a pair (m, w) such that w is m's first choice and m is w's first choice? (b) If such a mutually first pair exists in an instance, must it belong to every stable matching?

Solution. (a) No. In the lists used above, X's first choice is A but A's first choice is Y; Y's first choice is B but B's first choice is X; Z's first choice is A; C's first choice is X. Hence there is no mutually first pair, and every instance with these lists has no such pair in any matching. (b) Yes. If m and w rank each other first and are not matched to each other in some matching M, then each is matched to someone else and each prefers the other to that partner, so (m, w) is an unstable pair and M is not stable. Hence every stable matching contains (m, w). ■

Exercise: Number of Rounds

Problem. (a) If every man has a different favourite woman, how many rounds until the algorithm converges? (b) If all men have identical preference lists, how many rounds?

Solution. (a) One round. The n men propose to n different women, each woman accepts her only suitor, and from then on no woman has any reason to trade: every man is matched to his first choice, so no man ever proposes again. (b) n rounds. In round 1 all men propose to the same first-choice woman, who keeps her favourite and frees the rest; in round 2 the remaining $n - 1$ men all propose to the second-choice woman, and so on. After $n - 1$ rounds a single man is left and proposes to the last woman, so the process takes n rounds (and $\binom{n}{2}$ proposals). ■

Exercise: The Roommates Problem

Problem. $2n$ people must be paired into n rooms. Each person ranks all $2n - 1$ other people, and the graph is not bipartite because anyone can share with anyone. Is a stable matching always possible?

Solution. No. For 4 people A, B, C, D with A: B, C, D; B: C, A, D; C: A, B, D; D: any, there are only three matchings. For $(A, B), (C, D)$ the pair B–C is unstable; for $(A, C), (B, D)$ the pair A–B is unstable; for $(A, D), (B, C)$ the pair A–C is unstable. Every matching contains an unstable pair, so no stable matching exists. This shows why the bipartite structure, not the proposal mechanism alone, is what guarantees existence for stable marriage. ■

9 Intractability, Approximation and Local Search

9.1 Intractability

Some problems have no known polynomial-time algorithm, and we strongly believe that none exists. **NP-completeness theory** gives a precise way to say this: it classifies problems so that a polynomial-time algorithm for any one NP-complete problem would imply a polynomial-time algorithm for all of them.

9.1.1 Decision Problems versus Optimisation Problems

An **optimisation problem** asks for the best value (shortest path, largest clique, minimum cover). NP-completeness is defined only for **decision problems**, whose answer is "yes" or "no". Every optimisation problem has a natural decision version: for example,

Optimisation	**Decision version**
shortest path from s to t	is there an s–t path of length $\leq k$?
maximum clique	does G contain a clique of size $\geq k$?
minimum vertex cover	does G have a vertex cover of size $\leq k$?

If the decision version is hard, the optimisation version is at least as hard; the theory is stated for decision problems for technical convenience.

9.1.2 Polynomial Time and the Class P

An **alphabet** Σ is a finite set of symbols; a **language** L over Σ is any set of strings built from Σ. A decision problem corresponds to the language of its "yes" instances.

Class P

$P = \{ L : L \text{ can be decided by a polynomial-time algorithm} \}$. A problem is polynomial-time solvable if its decision version belongs to P.

Encodings matter. A number written in unary instead of binary can change the running time drastically, so complexity statements always assume a reasonable encoding (binary for integers). Polynomial-time algorithms are closed under composition, which is why the class P is robust.

9.1.3 Verification and the Class NP

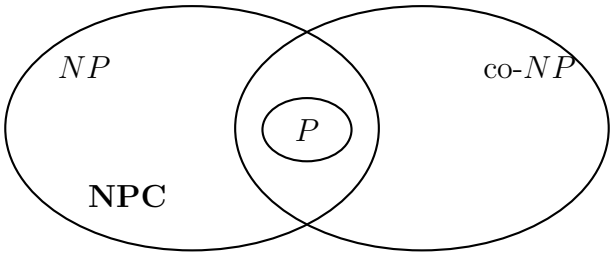
Verification Algorithm and NP

A **verification algorithm** $A(x, y)$ takes an input string x (the instance) and a binary string y (the **certificate**, or witness). It verifies x if there exists a certificate y with $A(x, y) = 1$.

$$NP = \{ L : \text{there is a polynomial-time verification algorithm } A \text{ such that } L = \{x : \exists y,\ A(x, y) = 1\} \}.$$

Equivalently, NP is the class of languages decidable in polynomial time by a non-deterministic Turing machine: the machine "guesses" y and then checks it deterministically in polynomial time.

For example, a Hamiltonian cycle is certified by the cycle itself, a clique of size k by its k vertices, and a satisfying assignment by the assignment. Each certificate has polynomial length and is checked in polynomial time.



$P \subseteq NP \cap \text{co-}NP$; the NP-complete problems lie inside NP but outside co-NP.

Basic Inclusions

- $P \subseteq NP$: any polynomial-time algorithm is a verification algorithm that ignores the certificate.
- co-NP is the class of languages L with $\bar{L} \in NP$; it contains the problems whose no answers have short certificates.
- $P \subseteq NP \cap \text{co-}NP$. Whether $P = NP$ and whether $P = NP \cap \text{co-}NP$ are both open.

"Solvable by dynamic programming" does not imply polynomial time

A DP whose state space is exponential (the $\frac{0}{1}$ knapsack with capacity W encoded in binary has $O(nW)$ states, and W can be 2^n) is only pseudo-polynomial. The statement "if a problem can be solved by DP then it is polynomial-time solvable" is false.

9.1.4 Reductions and NP-Completeness

Polynomial-Time Reduction

L_1 **polynomially reduces** to L_2, written $L_1 \leq_P L_2$, if there is a polynomial-time computable function $f : \{0,1\}^* \to \{0,1\}^*$ such that for every x,

$$x \in L_1 \iff f(x) \in L_2.$$

The algorithm computing f is a **reduction algorithm**. If $L_1 \leq_P L_2$ and $L_2 \in P$, then $L_1 \in P$; dually, if $L_1 \leq_P L_2$ and $L_1 \notin P$, then $L_2 \notin P$.

NP-Complete and NP-Hard

A language $L \subseteq \{0,1\}^*$ is **NP-complete** if
1. $L \in NP$, and
2. $L' \leq_P L$ for every $L' \in NP$.

If L satisfies (2) but not necessarily (1), it is **NP-hard**. Hence L is NP-complete iff $L \in NP$ and L is NP-hard.

The Central Theorem

If any NP-complete problem is polynomial-time solvable, then $P = NP$. Equivalently, if any problem in NP is not polynomial-time solvable, then no NP-complete problem is.

Two common misreadings

- Every NP-complete problem is in NP (property 1 of the definition).
- A reduction goes from an unknown problem to a known NP-complete problem to prove hardness, and from the NP-complete problem to a new problem to prove the

new problem is hard. The direction matters: if $L_1 \leq_P L_2$ and $L_2 \in NP$, then $L_1 \in NP$; but $L_2 \in NP$ plus $L_1 \leq_P L_2$ says nothing about whether L_2 is hard.

9.1.5 The First NP-Complete Problems

Circuit Satisfiability, SAT and 3-SAT

- **Circuit SAT:** given a Boolean circuit of AND, OR and NOT gates, is there an assignment of its inputs that makes the output 1? It is in NP and NP-hard, hence NP-complete; in fact every NP problem reduces to it.
- **SAT:** given a Boolean formula in conjunctive normal form, is it satisfiable? It is the historically first NP-complete problem (Cook–Levin).
- **3-SAT:** SAT restricted to clauses of exactly three literals. Still NP-complete, and it is the standard starting point for most hardness proofs.

How a hardness proof works. Show (i) the problem is in NP (give a certificate and a polynomial-time verifier), and (ii) some known NP-complete problem L' reduces to it. The two standard techniques are:

- **restriction:** show that the new problem, restricted to a special case, is exactly a known NP-complete problem;
- **local replacement:** transform each component of a known problem instance into a small gadget of the new problem, so that solutions correspond one-to-one.

9.1.6 A Catalogue of NP-Complete Problems

Problem	Question	Reduction from
CLIQUE	clique of size $\geq k$?	3-SAT
VERTEX-COVER	vertex cover of size $\leq k$?	CLIQUE
HAMILTONIAN-CYCLE	cycle through all vertices?	VERTEX-COVER
TSP	tour of length $\leq k$?	HAMILTONIAN-CYCLE
SUBSET-SUM	$S' \subseteq S$ with $\sum_{s \in S'} s = t$?	3-SAT

Clique to vertex cover. Let $G = (V, E)$ and $\bar{G}$ its complement. $S \subseteq V$ is a clique of G of size k iff $V \setminus S$ is a vertex cover of $\bar{G}$ of size $|V| - k$. Indeed, a clique in G has no edge of $\bar{G}$ inside it, so every edge of $\bar{G}$ has at least one endpoint outside the clique, i.e. in $V \setminus S$. Therefore

$$\langle G, k \rangle \in \text{CLIQUE} \iff \langle \bar{G}, |V| - k \rangle \in \text{VERTEX-COVER}.$$

Exercise: Which Is Not NP-Complete?

Problem. Among the following, which is not an NP-complete problem: (A) vertex cover; (B) Hamiltonian cycle; (C) the halting problem; (D) satisfiability?
Solution. (C). The halting problem is undecidable — not even a decidable decision problem — so it cannot be in NP and cannot be NP-complete. ■

Exercise: A Problem in NP

Problem. Q is in NP but not necessarily NP-complete. Which statement is false: (A) a polynomial-time algorithm for SAT implies one for Q; (B) a polynomial-time algorithm for Q implies one for SAT; (C) if $Q \notin P$ then $P \neq NP$; (D) if Q is NP-hard

then Q is NP-complete?
Solution. (B) is false. Every NP problem reduces to SAT, so SAT is at least as hard as Q: solving SAT solves Q, but solving Q need not solve SAT. (A) holds because $Q \leq_P$ SAT. (C) holds: a problem in NP outside P already separates the two classes. (D) holds by the definition of NP-completeness. ■

Exercise: Reduction Direction

Problem. If $L_1 \leq_P L_2$ and $L_2 \in NP$, is $L_1 \in NP$?
Solution. Yes. Compose the reduction algorithm with the polynomial-time verifier for L_2: on input x, compute $f(x)$ and verify it. Both steps are polynomial, and $x \in L_1 \iff f(x) \in L_2$, so $L_1 \in NP$. ■

9.2 Approximation Algorithms

Three ways to cope with NP-hardness.

1. If the inputs are small, an exponential-time exact algorithm is acceptable.
2. Some special cases (trees, planar graphs, small treewidth) admit polynomial-time exact algorithms.
3. Return a provably near-optimal solution in polynomial time: an **approximation algorithm**.

9.2.1 Approximation Ratio and Schemes

Approximation Ratio

Let C be the cost of the solution produced by an algorithm on an input of size n, and C^* the cost of an optimal solution. The algorithm has **approximation ratio** $\rho(n)$ if for every input

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n),$$

where $0 < C \leq C^*$ for a maximisation problem and $0 < C^* \leq C$ for a minimisation problem. Such an algorithm is a $\rho(n)$**-approximation algorithm**; a 1-approximation algorithm is exact.

PTAS and FPTAS

An **approximation scheme** takes the instance and a parameter $\epsilon > 0$ as input and, for every fixed ϵ, is a $(1+\epsilon)$-approximation algorithm.

- **PTAS** (polynomial-time approximation scheme): running time is polynomial in n for every fixed ϵ (it may be exponential in $\frac{1}{\epsilon}$).
- **FPTAS** (fully polynomial-time approximation scheme): running time is polynomial in both n and $\frac{1}{\epsilon}$, e.g. $O(\left(\frac{1}{\epsilon}\right)^2 n^3)$.

Tightness does not transfer

"ALG is an α-approximation algorithm with tight ratio, so for every $\epsilon > 0$ there is no $(\alpha-\epsilon)$-approximation algorithm unless $P = NP$" is false in general. Tightness means the analysis cannot be improved, not that no better algorithm exists; proving a lower bound requires a separate reduction.

9.2.2 Bin Packing

Problem. Given items of sizes $s_1, \ldots, s_n \in (0, 1]$ and bins of capacity 1, pack all items using as few bins as possible. Bin packing is NP-hard.

Heuristic	Rule	Ratio
Next Fit (NF)	newest bin only	$\leq 2 \cdot OPT - 1$
First Fit (FF)	first bin that fits	$\leq \frac{17}{10} OPT$
Best Fit (BF)	fullest bin that fits	$\leq \frac{17}{10} OPT$
First Fit Decreasing	decreasing order, then FF	$\leq \frac{11}{9} OPT + \frac{6}{9}$

Why Next Fit is a 2-approximation. In a Next-Fit packing any two consecutive bins have total size greater than 1 (otherwise the second bin would never have been opened). Suppose NF uses $2M$ bins; pairing them gives $S > M$, and since at least S bins are necessary, $OPT \geq S > M$, i.e. $2M < 2\,OPT$. As $2M$ is even, $2M \leq 2\,OPT - 1$: NF uses at most $2\,OPT - 1$ bins. The same argument gives the weaker but useful bound $NF \leq \lceil 2S \rceil$.

Structural facts. Next Fit can leave two bins of size less than $\frac{1}{2}$ that are not adjacent (e.g. items 0.4, 0.8, 0.4), so "Next Fit leaves at most one bin less than half full" is false. First Fit, on the other hand, leaves at most one bin less than half full: if two such bins existed, the second item would have been placed in the first one, since it had enough free space.

Online versus offline. An **online** algorithm must place each item before seeing the next one and can never move it. No online algorithm can be optimal: the adversary can always make the last items fill the gaps that the algorithm has already committed. An **offline** algorithm sees the whole input; sorting it in decreasing order before applying FF or BF gives the decreasing variants, which are substantially better.

9.2.3 Knapsack

Fractional knapsack (items may be split) is solved optimally by the greedy algorithm: sort by value density $\frac{p_i}{w_i}$ and take items in that order, splitting the last one. The ratio is 1.

0/1 knapsack (each item is taken whole) is NP-hard and has a 2-approximation: run the density greedy to obtain a solution A, and compare it with the single most valuable item B that fits; return the better one. If the greedy fills the knapsack completely then $C \geq C^*$, otherwise the item that did not fit has value at least the remaining capacity's worth, and a case analysis gives $C \geq C^*/2$.

Algorithm 65 KNAPSACK2(n, p, w, W)

1: sort items so that $\frac{p_1}{w_1} \geq \frac{p_2}{w_2} \geq \cdots \geq \frac{p_n}{w_n}$
2: greedily take items $1, 2, \ldots$ while they fit; let the result have value V_g
3: $V_m \leftarrow$ maximum p_i over all i with $w_i \leq W$
4: **return** $\max(V_g, V_m)$

9.2.4 The k-Center Problem

Problem. Given n sites in the plane and an integer k, choose k centers so that the maximum distance from any site to its nearest center (the covering radius) is minimised.

Greedy 2-approximation. Pick any site as the first center. Repeatedly pick the site that is farthest from the set of centers already chosen, until k centers are chosen and return them.

Why the ratio is 2. Let r^* be the optimal radius. When the greedy algorithm is about to choose its k-th center, it has found k sites that are pairwise at distance greater than $2r^*$ (each was farther than the current radius from all previous centers). If the optimum used k balls of radius r^*, two of these k sites would have to lie in the same ball and would then be at distance at most $2r^*$ — a contradiction. Hence the greedy radius is at most $2r^*$.

Lower bound. Unless $P = NP$, no ρ-approximation with $\rho < 2$ exists for the center-selection problem: a $(2 - \epsilon)$-approximation would decide whether k centers of radius 1 cover all sites, which is NP-hard.

9.2.5 Vertex Cover

Problem. Given an undirected graph $G = (V, E)$, find a smallest set $C \subseteq V$ such that every edge has at least one endpoint in C.

Algorithm 66 VertexCoverApprox(G)

```
C ← ∅;  E' ← E
while E' ≠ ∅ do
    pick an edge (u, v) ∈ E'
    C ← C ∪ {u, v}
    delete from E' every edge incident to u or v
end while
return C
```

This is a 2-approximation. Let A be the set of edges chosen in the loop. No two edges of A share an endpoint, because after choosing (u, v) all edges incident to u or v are deleted. Every vertex cover — in particular an optimal one C^* — must contain at least one endpoint of each edge in A, and since the edges of A are disjoint, one vertex of C^* covers at most one of them; hence $|C^*| \geq |A|$. The algorithm returns $|C| = 2|A|$, so

$$|C| = 2|A| \leq 2|C^*|.$$

A 2-approximation for vertex cover gives nothing for clique

Clique reduces to vertex cover by complementing the graph, but the reduction does not preserve approximation ratios: the mapping is $k_{\text{cover}} = |V| - k_{\text{clique}}$, so a factor 2 in the cover size becomes

$$\rho_{\text{clique}} = \frac{|V| - k^*}{|V| - k} = 1 + \frac{1}{|V|/k^* - 2},$$

which is unbounded as k^* grows. Approximation ratios are not preserved under polynomial reductions.

9.2.6 The Travelling Salesman Problem

Problem. Given a complete graph with edge costs, find a Hamiltonian cycle of minimum total cost. TSP is NP-hard.

With the triangle inequality, i.e. $c(u, w) \leq c(u, v) + c(v, w)$ for all u, v, w, there is a 2-approximation:

1. compute a minimum spanning tree T of the graph;
2. root T at any vertex and list the vertices in preorder, forming the tour H.

Why it works. Deleting one edge from an optimal tour H^* leaves a spanning tree, so $c(T) \leq c(H^*)$. A full traversal W of T uses every edge twice, hence $c(W) = 2c(T) \leq 2c(H^*)$. Preorder is obtained from W by skipping repeated visits; by the triangle inequality, skipping a vertex never increases the cost, so $c(H) \leq c(W) \leq 2c(H^*)$. (Postorder also satisfies the bound after removing repetitions.)

Without the triangle inequality there is no constant-factor approximation at all (unless $P = NP$): given an unweighted graph, assign cost 1 to its edges and cost $n + 1$ to every missing edge. A tour of cost $\leq n$ exists iff the graph has a Hamiltonian cycle, so any ρ-approximation would decide Hamiltonicity.

Exercise: Approximating a Maximum Spanning Tree

Problem. Let S be the set of maximum-weight edges incident to each vertex, and T a maximum spanning tree. Which statement is true: (A) $S = T$ always; (B) $S \neq T$

always; (C) $w(T) \geq w\dfrac{(S)}{2}$; (D) none?

Solution. (C). S may or may not equal T (a path $a - b - c - d$ with $S = \{ab, cd\}$ and $T = \{ab, bc, cd\}$ shows $S \neq T$; a triangle with $S = T$ shows equality is possible), so (A) and (B) fail. Since every edge of T is dominated by an incident maximum edge and each edge of S can be charged at most twice, $w(T) \geq w\dfrac{(S)}{2}$. ■

Exercise: Traversals That Give the 2-Approximation

Problem. In the MST-based TSP algorithm, which traversals satisfy the 2-approximation guarantee: level-order, preorder, postorder?

Solution. Preorder and postorder (two of the three). Both walk the tree in a way whose shortcut version respects the triangle inequality; level-order can jump between distant parts of the tree and does not satisfy the bound. ■

9.3 Local Search

Idea. When the full solution space is astronomically large, start from an arbitrary feasible solution and repeatedly move to a neighbouring solution that is better. When no better neighbour exists, the algorithm stops at a **local optimum**. Local search is fast and simple, but a local optimum need not be a global optimum — the algorithm has no way to climb out of a valley.

The Generic Scheme

$S \leftarrow$ some feasible solution;
while $\exists\, S' \in N(S)$ with $cost(S') < cost(S)$: $S \leftarrow S'$.

Here $N(S)$ is the **neighbourhood** of S, defined by a fixed kind of local move. The two questions that must be answered for every application are (i) does the algorithm terminate, and (ii) how far can the local optimum be from the global optimum?

9.3.1 Local Search for Vertex Cover

Start with $S = V$ (trivially a vertex cover). Repeatedly try to delete a vertex v from S; if $S \setminus \{v\}$ is still a vertex cover, keep the deletion, otherwise restore v and try another vertex. When no single vertex can be deleted, S is a minimal vertex cover: a local optimum with respect to the "delete one vertex" move.

Minimal is not minimum. On a path of four vertices, the cover $\{v_2, v_3\}$ is minimal but the optimum is $\{v_2, v_4\}$ only if v_4 is adjacent to v_3 — the algorithm's output depends on the order of the deletions. Nevertheless, for some graphs every order reaches the optimum: starting from deleting an isolated corner vertex in a small graph, every subsequent deletion can always be completed to a minimum cover.

9.3.2 Hopfield Neural Networks

A Hopfield network has a node for every edge of a weighted graph, with edge weights w_e (positive for "agreement", negative for "disagreement"). A node is **unstable** if the total weight of edges on which it disagrees exceeds the total weight of edges on which it agrees:

$$\sum_{e \text{ disagree}} |w_e| > \sum_{e \text{ agree}} |w_e|.$$

The local search flips one unstable node at a time.

Termination Bound

Each flip strictly decreases the total "disagreement weight" by at least 1 (weights are integers), and this quantity is bounded below by $-\sum_e |w_e|$. Hence the process terminates after at most

$$\sum_e |w_e|$$

flips, regardless of the order in which unstable nodes are chosen.

9.3.3 The Maximum Cut Problem

Problem. Given an undirected graph with positive edge weights, partition the vertices into (A, B) so that the total weight $w(A, B)$ of edges crossing the cut is maximised. Max-cut is NP-hard.

Local search. Start from any partition. If moving a single vertex from one side to the other increases $w(A, B)$, do it; stop when no single-vertex move improves the cut.

A $\frac{1}{2}$-Approximation

At a local optimum, for every vertex v the total weight of v's incident edges on the same side is at most the total weight on the other side; otherwise moving v would improve the cut. Let $d(v)$ be the weighted degree of v, and let $W = \sum_e w_e = \frac{1}{2}\sum_v d(v)$ be the total edge weight. For each v at least $d\frac{(v)}{2}$ of its incident weight crosses the cut, and summing over all vertices counts every crossing edge twice:

$$2\,w(A,B) \geq \sum_v \frac{d(v)}{2} = W \quad \Longrightarrow \quad w(A,B) \geq \frac{W}{2}.$$

Since $w(A^*, B^*) \leq W$, the local optimum satisfies $w(A,B) \geq \frac{1}{2} w(A^*, B^*)$.

Greedy version (sequential). Process the vertices one at a time; put each vertex on the side that currently gives the larger cut contribution. When vertex v is placed, at least half of its incident weight becomes a crossing edge, and no later vertex can change that, so again $w(A,B) \geq \frac{W}{2}$.

Local Optimum versus Global Optimum

- Local search for max-cut is a $\frac{1}{2}$-approximation; the bound is tight (a triangle gives 2 of the 3 total weight).
- Local search for vertex cover gives only a minimal cover, with no constant-factor guarantee.
- Local search can fail badly when the neighbourhood is too small: the k-center variant that recomputes cluster centres converges to a solution whose radius may exceed 2 times the optimum, because the moves are not the same as the farthest-point greedy moves that give the 2-approximation.

Exercise: Local Search on a Small Graph

Problem. In the graph of the figure, if we start by deleting the black vertex, can local search always find the minimum vertex cover?
Solution. Yes. After deleting the black vertex the remaining white vertices form a vertex cover, and no white vertex is redundant: deleting any white vertex uncovers one of its incident edges. Every order of attempted deletions therefore ends at the same minimum cover. ■

Exercise: Maximum Cut with a Large Gain

Problem. Max-cut with positive integer weights. A node is flipped only if it increases the cut by at least $\frac{w(A,B)}{|V|}$. Which statement is true: (A) at termination $2.5\,w(A,B) \geq w(A^*,B^*)$; (B) the algorithm terminates after $O(\log|V|\log W)$ flips; (C) at termination $2\,w(A,B) \geq w(A^*,B^*)$; (D) the algorithm terminates after $O(|V|^2)$ flips?

Solution. (A). With the threshold $\frac{w(A,B)}{|V|}$ per flip, the total cut increases by at least $w(A,B)$ over $|V|$ flips, so the final cut is at least $\frac{1}{2.5}$ of any cut, including the

optimum. The gain threshold also gives the termination bound, which is not $O(|V|^2)$ in general. ■

Exercise: Partitioning Jobs Between Two Machines

Problem. n jobs with times t_j must be split into two sets to minimise $|T_1 - T_2|$. Local search moves one job from one machine to the other whenever this decreases the difference. Which statement is true: (A) it never terminates; (B) if we always move the job with maximum t_j among those that reduce the difference, it terminates in at most n moves; (C) it always returns an optimal solution; (D) it always returns $\frac{T_1}{2} \le T_2 \le 2T_1$?

Solution. (B). The difference strictly decreases with each move and, choosing the largest available job, no job is moved twice, so there are at most n moves. (C) fails on $\{5, 6, 8\}$ versus $\{7, 7, 9\}$, where the local optimum $\{5, 6, 7, 8\}$ versus $\{7, 9\}$ cannot be improved although $\{6, 7, 8\}$ versus $\{5, 7, 9\}$ is better. (D) fails on $\{1, 1\}$ versus $\{10\}$. ■

10 Parallel Algorithms

10.1 Parallel Algorithms

A parallel algorithm is measured by two quantities rather than one: the total amount of work and the length of the longest chain of dependencies. This model separates what is inherently sequential from what is merely expensive.

10.1.1 The PRAM Model and Work–Depth

Parallel Random Access Machine (PRAM)

A PRAM consists of p processors sharing one unbounded memory. In one parallel step, every processor may read one cell, compute, and write one cell. The variants differ in how simultaneous accesses to the same cell are handled:

- **EREW** (exclusive read, exclusive write): no two processors may touch the same cell in the same step;
- **CREW** (concurrent read, exclusive write): simultaneous reads are allowed, simultaneous writes are not;
- **CRCW** (concurrent read, concurrent write): simultaneous writes are allowed, with an arbitration rule (common, arbitrary, priority).

Work and Depth

For an algorithm whose computation is a DAG of unit-time operations:

- **work** $W = T_1$: the total number of operations, i.e. the running time on one processor;
- **depth** $D = T_\infty$: the length of the longest path in the DAG, i.e. the running time on infinitely many processors.

For a balanced binary reduction tree over n leaves, $W = \Theta(n)$ (one operation per internal node) and $D = \Theta(\log n)$ (the height of the tree).

Brent's Theorem

For any parallel algorithm with work W, depth D and p processors,

$$\frac{W}{p} \le T_p \le \frac{W}{p} + D.$$

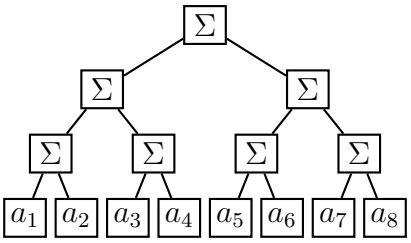
The lower bound is "each processor must do its share of the work"; the upper bound is obtained by scheduling the DAG greedily (always run a ready operation when a processor is free).

10.1.2 The Summation Problem

Sum n numbers on a balanced binary tree. In round 1, $\frac{n}{2}$ processors add adjacent pairs; in round 2, $\frac{n}{4}$ processors add the pairwise sums; after $\log n$ rounds one processor holds the total. Hence

$$W = \Theta(n), \qquad D = \Theta(\log n), \qquad \Theta\left(\frac{n}{p}\right) \le T_p \le \Theta\left(\frac{n}{p} + \log n\right).$$

With $p = n$ processors the running time is $\Theta(\log n)$, which is optimal; the work is still $\Theta(n)$, so no work is wasted.



rounds $= \log_2 8 = 3$

The classic true/false question "B(1,6) is computed before B(2,1)" is true: bottom-up evaluation computes B(1,6) in round 1 and B(2,1) in round 2.

10.1.3 Prefix Sums

Given $a_1, \ldots, a_n$, compute $c_i = a_1 + \cdots + a_i$ for every i. The parallel algorithm uses the same balanced tree twice:

1. **Bottom-up pass:** compute the sum $B(h, i)$ of each block of size 2^h — exactly the summation tree above.
2. **Top-down pass:** set $C(\text{root}) = B(\text{root})$, and for each node propagate partial sums: if i is a left child then $C(i) = C(\text{parent})$; if i is a right child then $C(i) = C(\text{parent}) + B(\text{left sibling})$.

Both passes take $\Theta(\log n)$ rounds, and each round performs $O(n)$ operations, so

$$W = \Theta(n), \qquad D = \Theta(\log n).$$

Because the top-down pass runs from the root downwards, $C(4, 1)$ is computed before $C(2, 2)$ — the classic true/false question is true. The same template solves prefix-min, which therefore also has $O(\log n)$ depth and $O(n)$ work.

10.1.4 Merging and Ranking

Ranking problem. Given two sorted arrays A and B, each of n elements, for every element determine its position in the merged sorted array.

Parallel binary search. Every element of A binary-searches for its position in B, and vice versa. One search is $O(\log n)$ sequential work, so

$$W = O(n \log n), \qquad D = O(\log n).$$

A serial merge takes $O(n)$ time, so the parallel version trades more total work for a shorter critical path. There is a parallel algorithm with $O(\log n)$ depth and $O(n)$ work (ranking by counting), and no serial algorithm can do better than $\Theta(n)$ — hence the answer to "which statement about ranking is true" is that a parallel algorithm exists with $O(\log n)$ time and $O(n)$ work.

10.1.5 Maximum Finding

Method 1: compare all pairs. Let n^2 processors compare every pair (i, j). Mark i as "not maximum" whenever $a_j > a_i$; the unmarked element is the maximum. This takes $O(1)$ time but $W = \Theta(n^2)$. The statement "to solve maximum finding with $T(n) = O(1)$ we need $W(n) = \Omega(n^2)$" is **false**.

Method 2: random sampling. Draw $\dfrac{n}{\log} n$ elements at random, find their maximum m sequentially, then use n processors to check in parallel whether any element exceeds

m. With high probability no element outside the sample exceeds m (the probability that the sample misses the true maximum is about $\frac{(\log n)}{n}$), so

$$T(n) = O(1), \qquad W(n) = O(n)$$

with very high probability, using $O(n)$ processors. This is the best of both worlds: constant depth without quadratic work.

Bounds. A serial algorithm needs $\Omega(n)$ time; no algorithm can achieve $O(\log n)$ time by itself. When the problem is split into subproblems solved in parallel, decreasing the subproblem size from $\sqrt{N}$ to $\log \log N$ does not reduce the depth below $O(\log \log n)$, so the claim that it reduces both work and worst-case time is false.

10.1.6 Sorting by Merging

Merge sort parallelises naturally:

MERGE-SORT(A, n) : sort $A[1..n/2]$ and $A[n/2 + 1..n]$ in parallel, then merge.

Merge implementation	**Work**	**Depth**
sequential merge	$O(n \log n)$	$O(n)$
binary search for every element	$O(n \log^2 n)$	$O(\log^2 n)$
ranking by counting	$O(n \log n)$	$O(\log n)$

With the binary-search merge, each level of recursion costs $O(n \log n)$ work and $O(\log n)$ depth, and there are $\log n$ levels, giving $W = O(n \log^2 n)$ and $D = O(\log^2 n)$. This is the standard comparison point for the classic multiple-choice question.

Exercise: Depth and Work of Prefix Min

Problem. The parallel prefix-min algorithm performs four loops: the initialisation, the bottom-up tree pass, the top-down propagation, and the output. What are its depth and work: (A) $O(n), O(n)$; (B) $O(\log n), O(\log n)$; (C) $O(\log n), O(n)$; (D) $O(n), O(\log n)$?
Solution. (C). The initialisation and output are $O(1)$ depth and $O(n)$ work each; the two tree passes are $O(\log n)$ depth and $O(n)$ work each. Depth adds to $O(\log n)$, work adds to $O(n)$. ■

Exercise: Random Sampling for the Maximum

Problem. To solve maximum finding with $T(n) = O(1)$ and $W(n) = O(n)$ with very high probability, are $O(n)$ processors required?
Solution. Yes (true). The random-sampling algorithm spends $O(1)$ time on the sample, and checking all n elements against the sampled maximum in one parallel step needs $O(n)$ processors; the total work is $O(n)$. ■

Exercise: Serial versus Parallel

Problem. When comparing a serial algorithm with its parallel counterpart, is it enough to concentrate on reducing the work?
Solution. No (false). Work is only one of the two measures; the depth (worst-case parallel running time) is equally important, and some parallel algorithms reduce work while leaving a long critical path. ■